Computing and Numerical Methods 2
C++ Programming

Dr. Chris Cantwell

2023/2024

v1.7

Contents

I C++ 3

1 Introduction to C++ 5

2 Variables and Data Types 15

3 Operators and Arithmetic 23

4 Input and Output 29

5 Control Structures 39

6 Functions 49

7 Arrays, Pointers and Dynamic Memory 61

8 The Standard Template Library (STL) 77

9 C++ Classes 91

10 C++ Classes - Polymorphism and Inheritance (Optional) 109

11 Templates (Optional) 115

II Supplementary Material 127

A Unix/Linux Operating System 129

B List of C++ File Types 137

C The C++ Compiler and Linker 139

Preface

In this part of the course, we will learn how to write computer programs using the C++ programming language. These lecture notes cover the various concepts and programming syntax, providing a reference for what is covered in the course. There are examples to illustrate the different aspects of the programming language being discussed and how they can be used.

Throughout the notes the material is presented in various formats:

Definition: Concepts

Important concepts are defined in coloured framed boxes to highlight them.



Note

Information to which your attention should be drawn are highlighted in notes. This might suggest where particular ideas or syntaxes are particularly useful, for example.



Warning

Warnings draw your attention to where common mistakes might be made, or where there are exceptions to the rules outlined previously.



Tip

Tips are intended to point out things which might make your life easier.

How to use these notes

The course is run in a *flipped-classroom* style. Each session will cover a particular topic or set of topics. You should familiarise yourself with these topics *before* the lab session by reading the relevant sections of the notes, watching any supplementary videos and attempting the exercises. The lab sessions will be devoted to practising programming. This will include class demonstrations and the opportunity to continue working on the exercises and ask questions if you are struggling.

Remember, this is a practical course! The best way to learn a (computer) language is through *practice*! You are therefore strongly encouraged to try the programming exercises provided at the end of each chapter.

Part I

C++

Chapter 1

Introduction to C++

Objectives

In this chapter you will learn about:

- the nature of the C++ programming language;
- how C++ compares with other types of languages;
- the basic syntax of a C++ program;
- running the compiler from the command-line and using Integrated Development Environments (IDEs).

1.1 What is C++?

C++ is a programming language created by Bjarne Stroustrup in 1985, originally as an extension to the C programming language to include object-oriented programming through *Classes*. The language has evolved significantly since then and its specification is periodically discussed, updated and published by the International Organization for Standardization (ISO). The version of the C++ language specification we will learn about is C++14. This is the current default standard used by the software tools we will be using.

C++ is a general-purpose language used to instruct a computer to perform a sequence of tasks or calculations. It is an intermediate-level language in the sense that instructions, or *source code* are given using human-readable text, yet it does contain some low-level capabilities which, for example, allow direct access to, and manipulation of, computer memory. As a consequence, it can be used to write very efficient, high-performance code.

It is a *compiled language*, which means the source code must be converted or *compiled* into a machine-readable file of instructions, called an *executable* that the computer processor can run directly; this is in contrast to an *interpreted* language, such as MATLAB or Python, where each statement in the source code is converted to machine instructions on-the-fly as the script is processed. C++ is *statically typed*, which means that the type of data must be known, and is checked for consistency, at compile time. C++ is also a procedural language, whereby a program is composed of steps (or *statements*) which are processed in a systematic order. It uses a

block-structured syntax which enables groups of statements to be grouped together as a single unit (using curly braces).

Like many other modern languages, C++ supports the concept of *object-oriented* programming and the *encapsulation* of data, which helps programmers create robust software. In fact, this is one of the key characteristics which originally differentiated it from the C language when it was conceived. Object-oriented programming is achieved through the concepts of *classes*. Inheritance and polymorphism allow programmers to save time by building on and extending existing classes, adding and changing their functionality. C++ also supports template meta-programming which improves our ability to reuse code with different data types by writing type-agnostic code that is made type-specific by the compiler rather than the programmer, while still retaining the strengths of a statically typed language.

The Standard Template Library (STL) exploits all of these capabilities and, in modern C++, now provides a rich collection of data structures and algorithms which remove many of the laborious aspects of working with an intermediate-level language, enabling sophisticated code to be more rapidly developed.

1.2 Creating a C++ program

To convert an algorithm into an executable program a number of steps must occur:

- The programmer writes the C++ program in text files using a text editor, or integrated development environment.

  These text files are called *source files* or *source code*. For C++ these files have the extension `.cpp` or sometimes `.cxx`. In large projects, components of the program will often be declared separately in *header files*, which have the extension `.h`, `.hpp` or `.hxx`; their use will be discussed in Chapter 9.

- A *Preprocessor* manipulates the source files, actioning any *preprocessor commands* in the source code.

  These commands may instruct the preprocessor to include another (source) file when processing your program, or substitute a preprocessor label with a defined value.

- A *Compiler* translates the processed high-level instructions into low-level binary instructions and creates an *object file*, or *object code*.

  These files contain instructions the computers CPU can directly understand, but these files are not directly executable as they may depend on other libraries of object code available elsewhere on the system.

- A *Linker* connects the programmers object code to existing libraries of object code (e.g. as provided by the Operating System) and produces an *executable file*.

  The executable file can now be run directly by the computer.

For more details on this process see Section C.1.

1.3 A first program

As is traditional with most programming languages, we introduce the C++ syntax through an example program which will print the phrase *Hello World!* to the terminal. Such an example is given in Example 1.1.

Example 1.1: An example program

```
1 #include <iostream>
2 using namespace std;
3
4 // Prints hello world to the screen.
5 int main() {
6     cout << "Hello World!" << endl;
7 }
```

We will introduce a number of aspects of the C++ syntax by examining the code line-by-line:

1. `#include <iostream>`

   This is a *pre-processor* command. The `#include` command makes a set of features available for us to use. The result is as if we had copy-and-pasted the contents of the `iostream` header file into our source code. The pre-processor is discussed further in Section 2.4.

2. `using namespace std;`

   This is a C++ *statement*. It tells the compiler to look in the namespace called `std` for features we will use later in the program. Namespaces just group together related functionality. The functionality we use in this course will almost exclusively come from the `std` namespace.

3. This is a blank line. Its only purpose is to make the code easier to read and understand by humans and is ignored by the compiler.

4. This is a comment, written in plain English, and is used to help other humans understand your code.

5. `int main() {`

   This begins a new program.

   - It defines a group of instructions called a *function* or *routine*, which are discussed in Chapter 6.
   - A C++ program must have exactly one routine called `main`. It must be called `main`, not `Main` or `MAIN`. C++ is a case-sensitive language.
   - `int` is short for *integer* (whole number) and is a data type. Its use here indicates that the function `main` produces an integer as its output. Data types are discussed in Section 2.1.
   - The left curly-brace, `{`, indicates the beginning of a block of code.
     C++ is a block-structured language which groups related statements together. These blocks are started with a `{` and must be terminated with a `}`.
     Their use will be more apparent when we discuss control structures in Chapter 5.

6. `cout << "Hello World!" << endl;`

   Prints the words *Hello World!* to the terminal.

   - `cout` is a *stream object* and represents output to the terminal window. See Chapter 4.
   - Remember C++ is case-sensitive; `Cout` is completely different to the compiler.
   - `<<` is a *stream operator*. See Chapter 3
   - `endl` (end line) denotes a new line.
   - The semi-colon indicates the end of a statement.



Warning

Don't forget the semi-colon!

The C++ language uses a semi-colon to indicate the end of each statement.

Whitespace (spaces, tabs, new lines) is ignored by the compiler and is purely to make the code more readable.

Pre-processor statements do not end in a semi-colon.

7. `}`

   Indicates the end of a block of code which began with the corresponding `{` (in this case the function `main`).

1.4 Code Layout

Layout of code is important, for both your own benefit and for others who will look at your code. Bad code layout makes it difficult (and thus unlikely) for others to help you solve your coding problems. e.g. below is a **BAD** example:

```
#include <iostream>
using namespace std;int main(){
cout<<"Hello World!"<<
endl;}
```

While this code compiles ok, it is difficult to read because there are multiple statements on a line and indentation has not been used to make it clear which statements are nested within the block denoted by the curly braces.

There is no single standard for code layout; often individual projects will specify a *coding standard* for all developers to follow. For the purposes of this course, please follow these guidelines at a minimum:

- Begin each statement on a new line;
- Use spaces to indent and align nested statements within blocks;
- Put spaces either side of operators.

Most importantly, be consistent!

1.5 Compiled and Interpreted Languages

C++ is a compiled language, which differs from an interpreted language, such as MATLAB. Interpreted languages progress through the original source code and *interpret* each line of code as they are encountered. In contrast, compiled languages convert the entire source code into binary instructions which can be directly executed by the processor. The conversion process is called *compilation*. Compiled codes are often more performant than interpreted codes, as interpreting source code takes time to process. However, this is always down to how well the code was written in the first place! Some other languages, such as Python, are called *byte-compiled* languages. The source code is compiled into a binary representation which is then interpreted by the Python interpreter, giving some performance improvement.



Note

While the term *compile* actually has a specific meaning in terms of the conversion process from human-readable source code to machine code (see Section C.1), we will in general use it more loosely to refer to the entire process of converting source code to an executable.

1.5.1 Command-line

To compile the code in Example 1.1, we type it into our favourite text editor and save the code as a text file, say `hello.cpp`, and then compile it using a compiler. The compiler we will use is the GNU C++ compiler which is invoked with the command `g++`.

```
g++ hello.cpp
```

This will produce an executable code with the name `a.out`, by default. More details on the compiler and the various options which can be used to modify the compilation process are given in Chapter C.

To execute our program, we simply run:

```
./a.out
```

Note that the `./` before the `a.out` is important. This tells the operating system to look for `a.out` in the current directory. Otherwise it will be looking for it in one of the standard system directories, such as `/usr/bin`.

1.5.2 Integrated Development Environments (IDEs)

Writing large amounts of code in a simple text editor and compiling it from a terminal can become laborious. Integrated development environments (IDEs) are software tools designed to help accelerate code development by providing access to these editors and tools through a single interface. Some features often provided by IDEs include:

- Syntax highlighting
- Auto-completion
- Direct integration with compilers and debuggers
- On-the-fly syntax checking

There are many such IDEs for C++ ranging from simple to highly complex software packages. For the purposes of this course, we will suggest the use of the CodeLite IDE, shown in Figure 1.1 since it provides all the important features of an IDE, is cross-platform (and can therefore provide a consistent environment on Linux, Mac and Windows computers) and is free software.

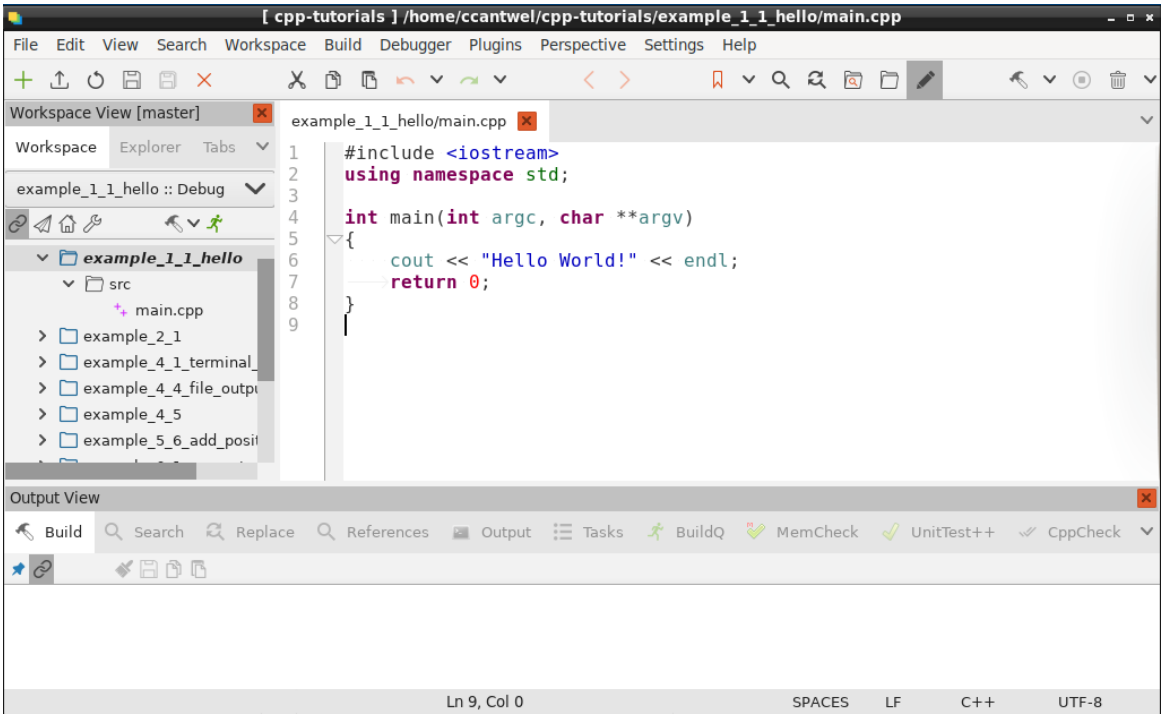


Figure 1.1: Screenshot of the CodeLite integrated development environment.

Exercise 1.1

For this course, we will be using the tools and libraries available under the Linux operating system.

1. Create a directory called `cpp-exercises` in your home directory and another directory called `exercise1-1` within it.
2. Using a web browser download the example files from Blackboard Learn to the `cpp-exercises/exercise1-1` directory.
3. Open `hello.cpp` in a text editor (e.g. leafpad). This is the program from Example 1.1.
4. Now open a terminal and compile the program by typing the commands

   ```
   cd $HOME/cpp-exercises/exercise1-1
   g++ hello.cpp
   ```

   Press ENTER after entering each line. If successful, you should see no output and be returned to the prompt.
5. A file `a.out` should have been created. Print a directory listing to check this by running

   ```
   ls
   ```

6. Run the program by typing

   ```
   ./a.out
   ```

   It should print the phrase:

   ```
   Hello World.
   ```

Exercise 1.2

Repeatedly compiling your code manually on the command-line, particularly for large projects, is tedious. Integrated Development Environments (IDEs) typically provide a useful way of

- managing multiple source code files in multiple projects
- quickly and easily compiling and running your code
- helping you get the syntax correct

We recommend using the IDE called **CodeLite**.

1. Launch *CodeLite* of the applications menu on the task bar. It can be found under the *Programming* sub-menu. If prompted, select the *GNU GCC compiler*.
2. Click the button to create a new workspace and select the *C++* workspace type. In the next dialog box, enter the workspace name *cpp-exercises*. Set the workspace path to the *cpp-exercises* directory you created in Exercise 1.1.
3. Select File, New, New Project. Choose *Simple executable (g++)* and click *Next*
4. Enter the project title *exercise1-2* (note that only characters, numbers and hyphens are allowed). Ensure the path is set to the `cpp-exercises` directory you set above.
5. Accept the default options in the following screens. A project will be created and a main.cpp file will appear under "exercise1-2/src" in the tree browser on the left side of the screen. Double-click it to open it, which contains a "Hello World" program, similar to the version seen in Exercise 1.1.
6. Build the program by clicking on the *Build* button (the green rectangle with the white arrow near the middle of the toolbar). The build process can be seen in the console window at the bottom. CodeLite runs g++ behind the scenes as can be seen in the Build window at the bottom.
7. Select *Run* from the *Build* menu — or press CTRL+F5 — to execute the compiled program normally. You should see the output in a terminal window. Note that the *Play* button on the toolbar runs the program through a program called a debugger and will not show output in this case.

```
Hello World!
```

Exercise 1.3

The aim of this exercise is to introduce you to the compiler.

1. Create another project called `exercise1-3` in the `cpp-exercises` workspace. There are now two independent projects in the workspace.

   Double-click `exercise1-3` to make it the active project (bold typeface).

   Delete the default `main.cpp` file, provided by CodeLite, from the project. This avoids us having two `main()` routines in our project.

2. Download the `compiler_error1.cpp` source code from Blackboard Learn, save it to `cpp-exercises/exercise1-3`).

Right-click on `exercise1-3/src` and select `Add an Existing File...` to choose the downloaded file and add it to the project.

3. Compile this code like you did in the previous exercise. Note that this time, the compiler does not complete successfully. It produces output in the *Build Log* tab similar to:

```
1  compiler_error1.cpp: In function 'int main(int, char**)':
2  compiler_error1.cpp:6:35: error: expected ';' before 'return'
3      cout << "Hello World." << endl
4      ^
5      ;
6      return 0;
7      ~~~~~~
```

Note: The wording the compiler uses may vary slightly according to the version being used.

Line 1 tells us which part of the code the error is located. Line 2 has the form:

`<file>:<line>:<column>: error: <message>`

The compiler is telling us that we are missing a semi-colon at the end of line 6. You may need to look at the lines surrounding the line the compiler reports, to find the real error.

Add a semicolon to the end of line 6; the code should now compile and run.

4. Download `compiler_error2.cpp`. Remove `compiler_error1.cpp` from the project and add `compiler_error2.cpp`. Compile the code and examine the *Build Log* messages. They should look similar to:

```
1  compiler_error2.cpp: In function 'int main(int, char**)':
2  compiler_error2.cpp:6:35: error: expected ';' before 'for'
3      cout << "Hello World." << endl
4                                   ^
5                                   ;
6      for (unsigned int i = 0; i < 10; ++i) {
7      ~~~
8  compiler_error2.cpp:7:30: error: 'i' was not declared in this scope
9      for (unsigned int i = 0; i < 10; ++i) {
10                              ^
11 compiler_error2.cpp:9:5: error: expected '}' at end of input
12     }
13     ^
14 compiler_error2.cpp:5:34: note: to match this '{'
15   int main (int argc, char** argv) {
16                                   ^
```

It is important to fix errors in order. The error on line 7 of the compiler output above is not actually an error; it is a consequence of a previous error (on line 6), and it will break your code further if you try and fix it! Add the semicolon again to the end of line 6 of the code and recompile. The compiler now reports:

```
1  compiler_error2.cpp: In function 'int main(int, char**)':
2  compiler_error2.cpp:9:5: error: expected '}' at end of input
3      }
4      ^
5  compiler_error2.cpp:5:34: note: to match this '{'
6    int main (int argc, char** argv) {
7                                    ^
```

C++ is known as a block-structured language; that is, code is contained within blocks. If a block is longer than one statement it must be surrounded by a matching pair of curly braces (`{, }`). The remaining error is a common error when the closing brace is missing. In the code, insert a new line after line 9 and place a closing curly brace (`}`). The code should now compile successfully and run, printing "Hello World" and the numbers 0 to 9.

Some things to check when your code doesn't compile:

- Have you forgotten a semi-colon at the end of a statement near the line-number at which the compiler reports an error?
- Do all curly braces match correctly?
- Are variable names spelt correctly? Remember C++ is *case-sensitive* and all keywords are written in lowercase.

Chapter 2

Variables and Data Types

Objectives

In this chapter you will learn about:

- the different data types that are used in C++;
- variables, how to name them, initialise them and about their scope;
- how to define constants;
- the preprocessor.

All computer programs work with data in one way or another. In this chapter we will examine how we store data in memory. Before we examine the different fundamental data types in C++, we should first understand how data is represented in memory.

2.1 Data types

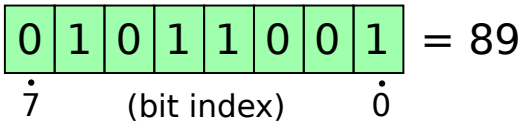
Computer memory, or random access memory (RAM) is volatile storage and its contents is lost when the computer is switched off. This is in contrast to disk (magnetic or solid state) which retains data when the power is removed.

Memory is a set of locations, arranged in a linear fashion and each with a unique *address*. Each location can store a single *binary* number. A binary number is 1 byte = 8 bits, where each bit is either a 0 or a 1.

Definition: Data Type

A *data type* defines how data is represented in memory, the values a particular item of data can take and the operations which can be performed on that data item.

A *datatype* tells the computer how many bytes are used to represent each piece of data and how to interpret the binary number(s). Positive whole numbers in the range 0-255 can be represented by 1 byte as shown below:

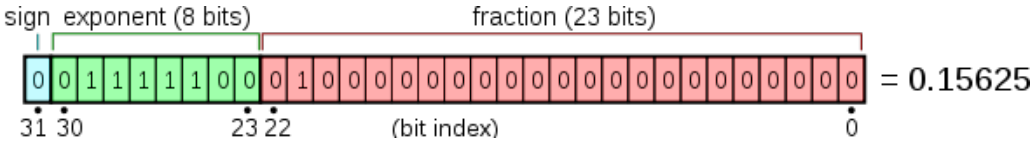


Note that the 8 bits are indexed from right, the least significant bit (LSB), to left, the most significant bit (MSB). Each bit represents a multiple of 2 (hence binary), so bit with index n represents 2^n. The decimal number is found by summing the values of those bits which are 1. In the example above, the 0th, 3rd, 4th and 6th bits are 1, so this binary number would have value:

$$1 + 8 + 16 + 64 = 89 \tag{2.1.1}$$

By using two bytes, we could represent positive whole numbers in the range 0-65535. If we used one of the bits to indicate the *sign* of the number we could use those same two bytes to represent -32768 to 32767. The standard C++ `int` datatype, discussed below, uses 4 bytes with one bit as the sign and so can store numbers in the range -2,147,483,648 to 2,147,483,647.

Other data types may use the bits in different ways. For example, a single-precision floating-point number, as defined in the IEEE 754 standard, uses 4 bytes and is interpreted as shown below:



This can be converted to decimal using the formula:

$$\begin{aligned} x &= (-1)^s \times 2^{E-127} \times \left(1 + \sum_{1}^{23} b_{23-i} 2^{-i}\right) \\ &= (+1) \times 2^{-3} \times 1.25 \\ &= 0.15625 \end{aligned}$$



Note

Different computers (e.g. 32-bit and 64-bit) may represent the same data type in different ways.

Finally, text is stored by representing each character as a number using a look-up table. Different sets of characters use different look-up tables, but the most ubiquitous one is the American Standard Code for Information Interchange (ASCII), although the Unicode Transformation Format (UTF) is now the preferred standard (which is backwards-compatible with ASCII). In ASCII, each character is represented by one byte. For example, 'A' is represented by the number 65 (or 01000001, in binary) and 'h' is represented by the number 104 (01101000).

Some of the most basic data types in C++ are:

- Numbers:
  - `int`: Integer (whole number, positive or negative)
  - `long`: Long (big) integer.
  - `float`: Single-precision floating-point number (32 bits).
  - `double`: Double-precision floating-point number (64 bits).

Integer datatypes may optionally be prefixed with `unsigned`, which means they can only take positive values ≥ 0.

```
unsigned int vIndex;
```

- Others:
  - `bool`: A boolean (true or false).
  - `char`: A single character (letter, digit, symbol, etc).

These are known as *fundamental data types*. Later in the course, we will encounter richer data types (such as strings, vectors and maps) (see Chapter 8), as well as how to define our own custom data types in the form of classes (see Chapter 9).

2.2 Variables

Definition: Variable

A variable is an identifier associated with a specific memory location.

A variable has the following properties:

- **Name**: An identifier which we use in our code in order to access or modify the data in the associated memory location;
- **Type**: The data type of data stored in that location. This tells the computer how to interpret the 1's and 0's in the memory.
- **Scope**: Define which parts of the program can use the variable.

Every variable must be declared before use. A simple variable declaration looks like the following:

```
DATATYPE IDENTIFIER;
```

For example, the following declares a variable which references an integer (whole) number in memory:

```
int vNumber;
```

and this example declares a variable which stores a single character and sets it to hold the character 'a':

```
char vInitial = 'a';
```

2.2.1 Naming Variables

The choice of identifier is entirely up to the programmer, with a few exceptions. Identifiers must

- begin with a letter. They **can** use numbers after the first character;
- not include spaces;
- not include any non-alphanumeric symbols (except the underscore).
- not match any existing C++ keywords (such as `int` or `if` or `for`);



Warning

Remember that **variable names are case-sensitive!**

`vmyvariable` is not the same as `vMyVariable`.

Different people have different practices when it comes to naming variables and so this course will not enforce one. However, whatever standard you adopt, it should be:

- Easily readable (e.g. `MyLongVariableName` rather than `mylongvariablename`).
- Consistent throughout your code.
- Descriptive (your identifier should reflect the variable's purpose!).



Tip

Here is a naming scheme which helps meet the above criteria and which also emphasises scope of a variable:

- Begin local identifiers with a single lowercase letter indicating its function scope: `v` for (local) variable, `m` for class member, `p` for function parameter, `g` for global variable.
- Begin each word with a capital letter (camel-case).

For example: `vMyName`, `pInputNumber`.

2.2.2 Initialising Variables

When a variable is declared, it can also be assigned a value.

There are three ways to do this:

- `DATATYPE IDENTIFIER = VALUE`
- `DATATYPE IDENTIFIER ( VALUE )`
- `DATATYPE IDENTIFIER { VALUE }`

The first and second approaches are the more conventional syntax typically used in older C++ codes — the second more frequently with complex datatypes, namely *Classes* (see Chapter 9). The third approach is called *braced initialisation* and was introduced more recently in the C++11 standard. The curly braces represent an *initialisation list* with one item in it. Their advantage is that they do not allow *narrowing conversions* (truncating a number to a less precise representation), which can help avoid coding errors.

Initialising variables at declaration is essential when a variable is defined as a *constant* (see Section 2.3).

2.2.3 Variable scope

Definition: Scope

Scope describes which parts of the program can *use* a variable. More concretely, a variable is only accessible in the block (and any nested sub-blocks) in which it is defined.

For example, a variable might be:

- Local to a control structure (Chapter 5).
- Local to a function or routine (Chapter 6).
- Local to a class (*member* or *class variable*) (Chapter 9).
- Global to the entire program.



Warning

Use of global variables:

One of the defining principles of C++ is to **encapsulate** data so only the parts of the program which need the variable can modify it. If you *need* to use a global variable, you are usually doing something wrong!

Example 2.1: Which variable is used outside of its declared scope, and where?

```
1 bool vTest = true;
2 if ( vTest == true ) {
3     int vNum = 3;
4     for (int i = 0; i < 10; ++i) {
5         vNum++;
6     }
7 }
8 cout << "The boolean is: " << vTest
9      << " and the number is " << vNum << endl;
```

If we try and use a variable outside of the scope in which it is declared, the compiler will produce an error message. The code in Example 2.1 contains such an error. It has three variables, each with different scope:

- `vTest` has a scope which extends for the entire example.
- `vNum` is only available inside the `if` block.
- `i` is only available inside the `for` loop.

The `if` and `for` statements are control structures (see Chapter 5). As we can see, in lines 8-9, the variable `vNum` is used outside of its scope. To fix this, we would instead *declare* the variable before the `if` statement. We can still modify the value of the variable inside the `if` statement if we need to – variable scope is defined by where it is declared.

2.2.4 Deduced data types

The keyword `auto` instructs the compiler to deduce the datatype of a variable from its initialising value. For example, for the code

```
int x = 5;
auto y = x;
```

the compiler would deduce that the datatype of `y` should be `int`.



Note

At the moment, `auto` is not particularly useful to us. It becomes more useful when dealing with more complex datatypes in the Standard Template Library (see Chapter 8) or templated functions (see Chapter 11) to avoid the need to write out the explicit type name.

2.2.5 Data type aliases

Sometimes it is helpful to define an alias for a data type to make code using complex data types more readable. This can be achieved using the `using` keyword.

```
using Num = long long;
Num a = 1000000000000000;
Num b = 2000000000000000;
```



Note

The use of type aliases is particularly helpful when using the Standard Template Library (see Chapter 8) and other templated types. It should generally be avoided for fundamental data types as it detracts from readability.

2.3 Constants

In some cases a variable should remain constant throughout its lifetime. For example the value of π or the speed of light c.

It is a good practice to tell the compiler if an identifier is constant. It can then tell you at compile time if you accidentally try to change it and therefore avoid a subtle functional error in your program. Moreover, the compiler can sometimes produce more efficient code when it knows the value of a variable at compile-time and that it will never change.

Marking an identifier as constant is achieved by prepending the data type with the keyword `const` as shown in Example 2.2.

Example 2.2: Using `const` to mark a variable as constant.

```
1 const double vPi = 3.14159;
2 const double vLight = 299792458; // m/s
```

2.3.1 Compile-time constants

A variable declared as `const` indicates that the value of the variable cannot be changed after it has been declared. However the (constant) value of the variable is still determined at run-time. It could, for example, be the result of calling a function, or prompting the user for input.

In contrast, a `constexpr` (introduced in C++11) indicates that the variable should be constant and *possible to evaluate at compile-time*. Valid values are themselves `constexpr` and must be known at compile-time.

For example,

```
constexpr double vPi = 3.14159;            // valid as known at compile-time
constexpr double vTime = getCurrentTime(); // almost certainly not valid
```

The result of a function can be used to initialise a `constexpr`, but the function itself must be declared as `constexpr`.

Compile-time constants allow us, in some instances, to make code more readable and reduce code duplication. See Section 6.5.

2.4 The Preprocessor

Statements beginning with a `#` are executed by the **Preprocessor**. Preprocessing happens just before the code is compiled. The preprocessor manipulates the source code producing an expanded code.



Note

Preprocessor statements are not terminated with a semi-colon.

Two common preprocessor commands are :

Command	Description
`#include`	Inserts the contents of another file.
`#define`	Defines a preprocessor variable..

An alternative way to define a constant is using a preprocessor variable (See Example 2.3). The variable uses the preprocessor command `#define`. This is often seen in some (standard) libraries. It is a common convention to use uppercase when naming identifiers with `#define`, although this is not required.

Example 2.3: Using the `#define` pre-processor directive.

```
#define G 9.81
...
double vForce = m * G;
...
```

However, typically the constant variable approach is preferred, leads to more readable code and errors are more readily identifiable.

For more information on the preprocessor see Section C.1.1.

2.5 Enumerations

Definition: Enumeration List

An enumeration list or `enum` is a type which is restricted to a specific range of named constants, whose values are of integer type.

Enumeration lists are a simple custom data type and are useful when a variable might take one of a fixed number of options (e.g. when a user chooses an option from a menu). By associating names with the options (instead of using arbitrary integers directly), it makes your code more readable.

In its most basic form, an `enum` can be created using the following syntax:

```
enum NAME { ENUMERATOR-LIST }
```

An example of an enum declaration and use is given in Example 2.4. The integer values associated with each name are implicitly chosen to be the value of the previous name, plus one. The first name is given the value zero. Values of some or all names can be explicitly chosen by assigning values to the names in the `enum` declaration.

Example 2.4: Using the `enum` keyword.

```
1  enum Colour {
2      red,            // red    -> 0 (implicitly chosen)
3      green = 5,      // green  -> 5
4      blue,           // blue   -> 6 (implicitly chosen)
5      yellow = 10     // yellow -> 10
6  };
7
8  Colour c = blue;
9  int    x = c        // x == 6
```

Chapter 3

Operators and Arithmetic

Objectives

In this chapter you will learn about:

- the different operators used in C++;
- the precedence of different operators;
- possible pitfalls with operators and arithmetic.

Once variables are defined a programmer will want to manipulate them. We can manipulate variables using *operators*.

Definition: Operator

An operator is a symbol denoting an operation (arithmetic, logical, or otherwise) on one or two variables.

The main type of operators are:

- **Unitary operator**: Acts on a single variable (e.g. `++`, `!`)
- **Binary operator**: Acts on two variables (e.g. `+`, `-` , `&&`, `<<`).

3.1 Arithmetic Operators

There are five arithmetical operations that are supported in C++. These are `+`, `-`, `/`, `*`, `%` and correspond to their respective mathematical equivalents. Note that the result of the modulus operation, `b % c`, is defined as the remainder a, such that $b = kc + a$ for some k.

The following table lists the operators and their descriptions. For the examples provided, assume variable a holds 5 and variable b holds 10:

Operator	Description	Example
`+`	Adds two operands.	`a + b` will be 15.
`-`	Subtracts second operand from the first.	`a - b` will be -5.
`*`	Multiplies two operands.	`a * b` will be 50.
`/`	Divides numerator by de-numerator.	`b / a` will be 2.
`%`	modulo operator (the remainder of a division).	`b % a` will be 0.

3.2 Assignment and compound assignment

The following compound assignment operators are supported by C++ (`=`, `+=`, `-=`, `*-`, `/=`, `%=`) and they provide the combined operation of performing an operation and assigning the result to the first operand.

Operator	Description	Example
`=`	Assigns values from right side operand to left side operand.	`c = a + b` will assign value of `a+b` into `c`.
`+=`	It adds right operand to the left operand and assign the result to left operand.	`c += a` is the same as `c = c + a` will assign value of`c+a` into `c`.
`-=`	It subtracts right operand to the left operand and assign the result to left operand.	`c -= a` is the same as `c = c - a` will assign value of `c-a` into `c`.
`*=`	It multiplies right operand with the left operand and assign the result to left operand.	`c *= a` is the same as `c = c * a` will assign value of `c*a` into `c`.
`/=`	It divides right operand with the left operand and assign the result to left operand.	`c /= a` is the same as `c = c / a` will assign value of `c/a` into `c`.

3.3 Increment/Decrement

The increment or decrement (`++`, `--`) increase or decrease the value stored in a variable by one, respectively.

Operator	Description	Example
`++`	Increment operator (increases the value stored in a variable by one).	`a++` is the same as `a = a + 1`.
`--`	Increment operator (decreases the value stored in a variable by one).	`a--` is the same as `a = a - 1`.

The increment and decrement can be used both as pre-increment and post-increment. The difference between them is:

`a = b++` is the same as `a = b; b = b + 1;` and `a = ++b` is the same as `b = b + 1; a = b;`.

3.4 Relational and comparison operators

Two expressions of the same data type can be compared using relational and comparison operators (`==`, `!=`, `>`, `>=`, `<`, `<=`), the result of which will be a boolean value (true or false).

For the examples provided in the table below, assume variable `a=5` and variable `b=10`:

Operator	Description	Example
`==`	Tests if the values of two operands are equal or not, if they are equal then condition becomes true.	`a == b` is false.
`!=`	Tests if the values of two operands are equal or not, if they are not equal then condition becomes true.	`a != b` is true.
`>`	Tests if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	`a > b` is false.
`>=`	Tests if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	`a >= b` is false.
`<`	Tests if the value of left operand is less than the value of right operand, if yes then condition becomes true.	`a < b` is true.
`<=`	Tests if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	`a <= b` is true.

3.5 Logical operators

Logical operators are used to combine multiple logical tests.

In the following examples, assume variable `a=true` and variable `b=false`:

Operator	Description	Example
`!`	Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true, then Logical NOT operator will make it false and vice-versa.	`!a` is false.
`&&`	Logical AND operator. If both the operands are non-zero (true), then condition becomes true.	`(a && b)` is false.
`||`	Logical OR Operator. If any of the two operands is non-zero (true), then condition becomes true.	`(a || b)` is true.

3.6 Stream operators

They are used to extract input from files or terminal or to perform formatted output to files or terminal. (for more details see Chapter 4)

Operator	Description
<<	It is applied to an output stream, is known as insertion operator, and performs formatted output.
>>	Extracts as many characters as possible from the stream and inserts them into the output sequence controlled by the stream buffer object.

3.7 Operator Precedence

What is the result of

`x = 4 * 3 + 8 / 2 - 1 ?`

In order to evaluate this, you need a rule to decide in which order to perform the operations.

If the precedence is not explicitly set with parenthesis or if the operations are of the same precedence, then the execution is by default from left to right.

The precedence of common operators, that is, which are evaluated first, is given in the table below. A full table of all operators is available on www.cplusplus.com.

Operator	Description	Order/grouping
`++ --`	Postfix increment/decrement	Left-to-right
`++ --`	Prefix increment/decrement	Right-to-left
`+ -`	Unary prefix	Right-to-left
`& *`	Reference/dereference	Right-to-left
`* / %`	Arithmetic scaling	Left-to-right
`+ -`	Arithmetic addition	Left-to-right
`<< >>`	Bitwise shift	Left-to-right
`< <= > >=`	Relational	Left-to-right
`== !=`	Equality	Left-to-right
`&&`	Logical AND	Left-to-right
`||`	Logical OR	Left-to-right

A more complicated example:

```
a = 1;
a = 2 * ++ a + 4 / 2;
```

This can be seen to be equivalent to the following, where we have used parentheses to explicitly define the precedence:

```
a = 1;
a = (2*(++a)) + (4/2);
```



Note

Use round brackets "()" in complicated expressions to make them more readable, avoid confusion and reduce the chance of subtle errors.

3.8 Operators and Arithmetic



Warning

Integer and floating-point arithmetic are different. The compiler will look at the operands, and choose what it thinks you mean. You should help the compiler make the correct choice.

For example:

```
double vHalf = 1/2;
```

will produce zero! It thinks you are doing integer arithmetic.

Instead do:

```
double vHalf = 1.0/2.0;
```

Exercise 3.1

If $a = 2$, $b = 5$ and $c = 4$ are integers, what is the result of the following mathematical expression, when following the C++ rules of operator precedence?

```
a + b * c / a * ( b - c ) + b % c
```

Write a program which evaluates the above expression to check your answer.

Exercise 3.2

Without using the compiler, identify the syntax errors in the following C++ program:

```
include iostream

Main();
{
  Float x,y,z;
  x = 2;
  y=3;
  z = x + y
  cout << "Result is " < z << end;
  return 0.
}
```

Write a corrected version of the program and check it compiles.

Chapter 4

Input and Output

Objectives

In this chapter you will learn about:

- how to read input and write output from/to the terminal;
- how to read input and write output from/to files;
- how to format text.

A program is useless unless it can accept input from the user and produce an output. Usual inputs/outputs include reading from, and printing to, the terminal, and reading from, and writing data to, a file on disk.

Input and output in C++ is easily achieved using an abstraction called **streams**. These can be conceptually thought of as pipes through which data flows. The stream operators are << (insertion) and >> (extraction).

In C++, terminal manipulation is usually done using the `iostream` objects `cout` and `cin`. To use these, you need to add `#include <iostream>` to your program.



Note

An *object* is a variable, having a complex type, which can contain both data and functions which act on that data (see Chapter 9).

4.1 Terminal I/O

To output data to the terminal we use the `cout` object followed by the insertion operator >> and the variable we wish to output, as follows.

```
cout << "Some text. ";
cout << "Some more text on the same line";
cout << endl;
int a = 42;
```

```
cout << a << endl;
```

The `endl` token indicates we wish to start a new line. It is identical to writing `"\n"`.

The token `flush` can be used to force output to be produced immediately when a new line is not required.

To read data input into the terminal use the `cin` object followed by the extraction operator `>>`, for example

```
int vNumber;
cin >> vNumber;
```

A full example on how to ask the user for input and then write back the supplied value to the terminal is shown in Example 4.1.

Example 4.1: Terminal I/0

```
#include <iostream>

using namespace std;

int main()
{
    string vName;
    unsigned int vAge;
    unsigned int vOption;

    cout << "Enter your name: ";
    cin >> vName;

    cout << "Enter your age: ";
    cin >> vAge;

    cout << "Please select an option:" << endl;
    cout << " 1. Option 1" << endl;
    cout << " 2. Option 2" << endl;
    cout << " 3. Quit" << endl;
    cout << "Enter option:";
    cin >> vOption;

    cout << "Name: " << vName << ", Age: " << vAge << ", Option: " <<
    vOption
<< endl;
}
```

4.1.1 Strings

Example 4.1 introduced a new data type called `string`. Conceptually, a string is a sequence of characters. The `string` datatype is defined in the standard library and to use it we should include the header file `<string>`. Often this is already included by other header files we are using, such as `<iostream>` in the example.

Strings are useful for storing in memory text read from the terminal or from a file.

4.1.2 Formatting text on terminal

Using the insertion operator with C++ streams is generally straight forward; controlling the format of the output requires additional commands. An output stream has a number of member variables that control the placement of the output on the terminal. The placement can be controlled by *member functions* of the `cout` object itself, or special tokens called *manipulators*, which are added to the stream.

The default output for integers is to print them using decimal (base 10), with no leading zeros and a leading minus sign if negative, using as many characters as needed, but no more. Floating-point numbers will be printed using a general format that is effective in representing a number in a small amount of space. The default precision is 6, i.e. up to 6 significant digits are used to represent the number. Very large floating-point numbers may be printed in *scientific notation*, e.g. `1.23456e+08`.

Member functions of `cout`

Functions	Description	Example	Result

`width()`	Sets the field width for subsequent items in the stream.	`cout << "*";`
		`cout.width(4);`
		`cout << 12 << "*" << endl;`
			`* 12*`

`fill()`	Changes the padding character used when the number occupies fewer characters than the field width.	`cout.fill('.');`
		`cout << "*";`
		`cout.width(4);`
		`cout << 12 << "*" << endl;`
			`*..12*`

`precision()`	Sets the precision of floating-point numbers.	`cout.precision(4);`
		`cout << 3.14159 << endl;`
		`cout << 1234.56789 << endl;`
		`cout << 245.678 << endl;`
		`cout << 12345.0 << endl;`
			`3.142`
			`1235`
			`245.7`
			`1.234e+04`

Manipulators

Manipulators are inserted into the stream (with the insertion operator) to modify the formatting of the next item in the stream.

Those manipulators with no arguments, such as `endl`, are included in `iostream` header. However, to use the manipulators in the table below you need to `#include <iomanip>`.

Functions	Description	Example	Result
`setw()`	Adjusts the field with for the item to be printed.	`cout << "*" << setw(4) << 12 << "*" << endl;`	`* 12*`
`setfill()`	Changes the padding character.	`cout << setfill('.') << "*" << setw(4) << 12 << "*" << endl;`	`*..12*`

`setprecision()`	Changes the precision of floating-point numbers.	`cout << setprecision(4) << 3.14159 << endl << 1234.56789 << endl << 245.678 << endl << 12345.0 << endl;`	`3.142`
			`1235`
			`245.7`
			`1.234e+04`

4.1.3 Reading lines of data

The stream input operator extracts characters from a stream up to the next whitespace character. When reading strings, this operator will only extract one word at a time.

Entire lines of text can be read from a stream using the `getline()` function. This will read characters up to the next newline character (or the end of the file) into a string variable `str`. For example,

```
getline(cin, str);
```

Optionally, the delimiter can be specified explicitly. The following would read all characters up until the next comma:

```
getline(cin, str, ',');
```

A complete example comparing these different methods is given in Example 4.2. The `ignore` member function is used to clear any remaining characters in the input buffer after extraction in the case where the newline character alone is not the delimiter.

4.1.4 Reading and writing different types of data

A program may prompt the user for information of various data types. For example, requesting strings for their name or address, whole numbers (integers) for specifying a menu item, or floating point numbers for reading in the radius of a circle.

All input to a program is initially in the form of a sequence of characters (a `string`). Sometimes these characters actually represent a number, e.g. `"42"` or `"3.14159"`, and need to be stored in a variable of a numeric data type for use in calculations later in the program. However, if input is directly read into such a variable, any non-numeric characters entered may cause the program to behave unpredictably.

Example 4.2: Comparison of different approaches to reading text from a terminal.

```cpp
#include <iostream>
#include <limits>
using namespace std;

int main() {
    string input;
    cout << "Enter text (whitespace-delimited): ";
    cin >> input;
    cout << "Using stream operators: " << input << endl;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    cout << "Enter text (newline-delimited): ";
    getline(cin, input);
    cout << "Using default getline: " << input << endl;

    cout << "Enter text (comma-delimited): ";
    getline(cin, input, ',');
    cout << "Using comma separator: " << input << endl;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    return 0;
}
```

To safely convert user input to a variable of a numeric data type, we can use one of several conversion functions provided by the standard library. This is achieved in two stages:

- Read the data into a string, since a string can store all characters entered by the user;
- Use an appropriate conversation function, for example `std::stod`, to convert the string to the desired data type.

This is illustrated in Example 4.3.

Example 4.3: Converting a string to numerical data types.

```cpp
#include <string>

int main() {
    std::string s    = "42";
    int         num  = std::stoi(s);    // convert to an integer
    float       numf = std::stof(s);    // convert to a float
    double      numd = std::stod(s);    // convert to a double
}
```

If the conversion cannot be performed (for example, if the string is not actually a number), these functions will throw an exception (see Section 6.10 for how to handle these).

4.1.5 Splitting strings

Sometimes it is necessary to split a string into words and store each in a different variable. This is more generally known as *tokenising*. In the case of splitting a string into words, the token *separator* would be the space character, but one might also use a comma as the separator if parsing, for example, a comma-separated values (CSV) file. C++ provides the `stringstream` class, which

provides a simple mechanism for tokenising strings with spaces. It also allows for implicit conversation of strings to numeric data types. In combination with `getline()` (instead of the extraction operator), it can be used to tokenise strings with a different separator.

Example 4.4: Tokenising a string using `stringstream` with implicit type conversion.

```
#include <iostream>
#include <sstream>
using namespace std;

int main() {
        stringstream s;
        s.str("1 2 3 4 5");
        cout << "The stringstream holds: " << s.str() << endl;
        cout << "Read at position: "       << s.tellg() << endl;

        int one, two, three;
        s >> one >> two >> three;

        cout << one << ", " << two << ", " << three << endl;
        cout << "The stringstream holds: " << s.str() << endl;
        cout << "Read at position: "       << s.tellg() << endl;

        return 0;
}
```

A `stringstream` behaves in a very similar way to `cin`; in fact they are both based on the same underlying `iostream` code. Example 4.4 shows a code to split a string and implicitly converting the tokens into integers. Only the first three tokens are extracted and we can use `tellg()` to see which character position we are at in the string. The code in this example produces the following output:

```
The stringstream holds: 1 2 3 4 5
Read at position: 0
1, 2, 3
The stringstream holds: 1 2 3 4 5
Read at position: 5
```

4.2 File I/O

Reading and writing to files on disk is also achieved using specialised streams:

- Input from file is performed using `ifstream`
- Output to file is performed using `ofstream`
- Both input and output can be performed using `fstream`

A file is opened and closed using the `open()` and the `close()` member functions of the stream object, respectively. If a filename is passed to the stream object when it is declared, the file will be automatically opened. An example showing how to write a file called `data.txt` with tabulated output is shown in Example 4.5. Remember to include the necessary header files.

4.2.1 File access modes

When opening files, one or more modes can be selected. Modes control how you access the file (read/write), the formatting of the file (binary mode/text mode) and whether existing file content should be kept (append), or discarded (truncate). Modes are applied when calling `open()`. Multiple modes can be combined using the **bitwise "OR" operator,** `|`.

If no modes are entered, defaults are used (in the case of `ofstream`, for example, the default is `ios::out`).

Mode	Description
`ios::in`	File is open for input only.
`ios::out`	File is open for output only.
`ios::binary`	File is formatted as binary (as opposed to text).
`ios::atet`	File is opened with the initial position at the end of the file.
`ios::app`	Append data to the end of the file if it already exists.
`ios::trunc`	Delete the file and start afresh if the file already exists.

Line 8 of Example 4.5 shows how to open an existing file for output, replacing (truncating) any data in the file (if it exists).

Example 4.5: Writing data to a file. The file `data.txt` is opened for output and any existing data is discarded. A header line and a line of data are written to the file using fixed width fields.

```
1  #include <fstream>
2  #include <iomanip>
3  using namespace std;
4
5  int main() {
6      int x = 5;
7      ofstream vOut("data.txt", ios::out | ios::trunc);
8      vOut.precision(5);
9      vOut << setw(15) << "# x"
10          << setw(15) << "x^2"
11          << setw(15) << "x^3" << endl;
12     vOut << setw(15) << x
13          << setw(15) << x*x
14          << setw(15) << x*x*x << endl;
15     vOut.close();
16 }
```

By default files are opened as text files, rather than binary. You will only need to use this mode during this course. The differences between binary and text modes are:

- Binary files are unformatted binary data.
- Text files can be more easily manipulated.
- Text files are frequently readable by plotting programs (e.g. MATLAB).
- Text files are easily readable by humans in a text editor (useful for debugging).
- The same data represented as a text file typically occupies significantly more disk-space than a corresponding binary file.



Note

Both text files and binary files may not be compatible across platforms, for different reasons. Text files may use different character sets and special characters (e.g end-of-line LF and CR conventions). Binary files may use different byte-orderings to represent data types (little-endian, big-endian). However, text files are generally more likely to be portable across platforms.

4.2.2 Stream states

Problems can occur when accessing files. For example, an input file might not exist, the end of the file is reached or the output file might not be writeable (disk full, insufficient permissions). To ensure your program runs reliably, the `state` of the stream can be tested after performing operations on file streams using the stream object member functions in the following table. Example 4.6 illustrate the use of these tests while reading data from a file.

Member Function	Description
`good()`	Tests if a file is ready for reading or writing (as appropriate).
`eof()`	Tests if the end of the file has been reached (when reading).

Example 4.6: Testing stream state while reading data from a file. The `if` and `while` keywords are control structures, discussed in Chapter 5.

```
ifstream vMyFile("data.txt");
string vTemp;
if (vMyFile.good()) {                  // Check file opened ok
    while (true) {                     // Keep trying...
        vMyFile >> vTemp;              // ... to read file
        if (vMyFile.eof()) {           // Test if end of file reached
            break;                     // ... and stop if it is
        }
        cout << vTemp << endl;         // Print what we read
    }
    vMyFile.close();
}
else {
    cout << "Failed to open file" << endl;
}
```

4.2.3 Navigating Files

The position in a file can be determined using the member functions in the table below. Example 4.7 illustrates their use by printing the position of the next read operation after reading the first word from the file `data.txt`.

Member Function	Description
`tellg()`	Position of the next read operation (get).
`tellp()`	Position of the next write operation (put).

We can move to another location in the file (character position) by using the following functions. Example 4.8 uses these to navigate a file in order to get the size of a file.

Example 4.7: Determining position in a file

```
string vBuffer;
ifstream vMyFile ("data.txt");
vMyFile >> vBuffer;                  // Read one word from the file
cout << vMyFile.tellg() << endl;     // Print the position in file
```

Member Function	Description
`seekg(position)` `seekg(offset, position)`	Seek to an absolute position. Seek an offset from a position where `position` is one of `ios::beg, ios::end, ios::cur.`

Example 4.8: File size

```
unsigned int vBegin, vEnd;

// Create input file stream (implicitly opens file)
ifstream vMyFile ("data.txt");

// Get the current position and save it
vBegin = vMyFile.tellg();

// Go to the end of the file and get position
vMyFile.seekg(0, ios::end);
vEnd = vMyFile.tellg();

// Print out the difference to show file length
cout << "File length is " << (vEnd - vBegin)
     << " bytes long." << endl;

vMyFile.close();
```

Exercise 4.1

Write a program in C++ which computes the surface area *and* volume of a sphere, defining the constant $\pi = 3.1415926$ yourself. Your solution should prompt the user to enter the sphere's radius, and it should print out the result. The volume, V, and surface area, A, are computed as:

$$V = \frac{4\pi r^3}{3} \qquad A = 4\pi r^2$$

Exercise 4.2

The motion of a free falling object in a vacuum is governed by Newton's second law of motion, such that acceleration is given by

$$a\,\hat{\mathbf{k}} = \frac{d^2y}{dt^2}\hat{\mathbf{k}} = -g\,\hat{\mathbf{k}}$$

1. Prompt the user to enter an initial height y_0 and a time T.
2. Determine the velocity $v(t) = \frac{dy}{dt}$ and position $y(t)$ of a body, initially at rest and at a height of y_0, after $t = T$s.

   Assume the constant acceleration due to gravity is $g = 9.8\,\text{m/s}^2$.
3. Print the velocity and position to the screen.

Exercise 4.3

A parachutist jumps from a plane moving horizontally. She is slowed by an upward force due to air resistance. Her vertical motion can be described by the following equation:

$$v(t) = \frac{mg}{c}\left(e^{-\frac{ct}{m}} - 1\right)$$

where m is her mass, c is the drag coefficient and g is the acceleration due to gravity.

1. Prompt the user to enter the mass (in kg) and drag coefficient (in kg/s) and compute her velocity at $3\,$s, $5\,$s and $10\,$s.
2. Calculate her terminal velocity.

Test your code with a mass of $65\,$kg and a drag coefficient $c = 13\,$kg/s.

Exercise 4.4

The deflection of a simply supported beam loaded with a uniformly distributed load is:

$$y = -\frac{w}{24EI}\left(x^4 - 2Lx^3 + L^3x\right)$$

where L is the length of the beam, E is the Young's modulus, I is the second moment of area and w is the magnitude of the distributed load.

Write a program which prompts the user for the parameters L, E, I, w and computes the deflection at the midpoint of the beam.

Test your code with $E = 210 \times 10^9\,$Pa, $I = 0.0016\,\text{m}^4$, $w = 10 \times 10^3\,$N/m and $L = 2\,$m.

Chapter 5

Control Structures

Objectives

In this chapter you will learn about:

- conditional structures and their use;
- iterative structures and their use.

Programs usually require the computer to make decisions based on the input data. They may also require the computer to perform instructions repeatedly. Control structures enable both of these capabilities. There are two main categories of control structures: **conditional** structures and **iterative** structures.

5.1 Conditional Structures

Definition: Conditional Structures

Conditional structures execute a *block* of statements if a condition is met.

Conditional structures include `if-else` statements and `switch` statements.

5.1.1 if statement

The `if` statement has the following form:

```
if ( CONDITION ) {
    STATEMENTS
}
```

The `if` statement evaluates the `CONDITION` (test expression) inside the parenthesis. If it evaluates to `true`, `STATEMENTS` inside the body of the block are executed. Otherwise they are skipped.



Note

If a block consists of a single statement, the curly braces are not essential, but are good to include anyway for code clarity.

Notice that `CONDITION` is an expression which evaluates to `true` or `false`. For example:

```
(a == b)
(a > b)
(vFlag == true)
(vFlag)
```

The use of the `if` statement is illustrated in Example 5.1.

Example 5.1: `if` statement

```
1 if (a == b) {
2     a = b + 1;
3 }
```



Warning

Note that the operators "=" and "==" are very different! Using "=" by mistake is a common error and can be difficult to debug.

5.1.2 if-else statement

An `else` clause can be added to execute an alternative set of statements if the condition is not met.

```
if ( CONDITION ) {
    STATEMENTS-TRUE;
}
else {
    STATEMENTS-FALSE;
}
```

The `if-else` statement executes either `STATEMENTS-TRUE`, if `CONDITION` is met, or `STATEMENTS-FALSE` if it is not met. It is impossible for both `STATEMENTS-TRUE` and `STATEMENTS-FALSE` to be executed on a single evaluation of `CONDITION`.

5.1.3 Compound Statements

In some circumstances it is necessary for several conditions to be satisfied for a set of statements to be executed. To achieve this use the **logical operators** to combine conditions.

```
if ( CONDITION1 [OP CONDITION2] ... ) {
    STATEMENTS;
}
```

where the expression in square brackets may appear zero or more times. For example:

```
if (a == 3 && b == 2) { }
if (a == 3 || a == 7) { }
```

Example 5.2: Finding the minimum of three numbers

```cpp
#include <iostream>
using namespace std;

int main()
{
    int n1, n2, n3;
    cout << "Enter three integer: ";
    cin >> n1 >> n2 >> n3;

    if (n1 < n2) {
        if (n1 < n3) {
            cout << "Their minimum is " << n1 << endl;
        }
        else {
            cout << "Their minimum is " << n3 << endl;
        }
    }
    else if (n2 < n3) {
        cout << "Their minimum is " << n2 << endl;
    }
    else {
        cout << "Their minimum is " << n3 << endl;
    }
    return 0;
}
```

Several conditions can be tested for using a nested `if-else` statement; for example, an `if-else` statement can be nested within the `else` part:

```cpp
if ( CONDITION_1 ) {
    STATEMENTS_1;
}
else {
    if ( CONDITION_2 ) {
        STATEMENTS_2;
    }
    else {
        STATEMENTS_3;
    }
}
```

A cleaner way to test for multiple conditions is using the `if-else-if` statement. An `else if` block is added for each additional condition to test for:

```cpp
if ( CONDITION_1 ) {
    STATEMENTS_1;
}
else if ( CONDITION_2 ) {
    STATEMENTS_2;
}
...
else {
    STATEMENTS_N;
}
```

Example 5.2 shows how we can use nested `if-else` statements.

5.1.4 Switch statement

Sometimes, when testing for different values of a *single integer variable*, a more elegant solution is to use the `switch` statement. This comprises a sequence of `case` labels for the different possible values of the variable, each with zero or more statements to execute. The values after the `case` keyword specifies the value we wish to test the variable against. The `break` statement denotes the end of the block and execution will jump to the statement immediately after the switch block. A `default` label can be specified to execute statements in the event the value does not match any of the previous values.

```
switch ( IDENTIFIER ) {
    case VALUE_1:
        STATEMENTS_1;
        break;
    ...
    case VALUE_N:
        STATEMENTS_N;
        break;
    [default:
        STATEMENTS_DEF]
}
```

An example of using the switch statement is shown in Example 5.3.



Tip

If you need to define variables inside a single `case`, enclose the statements for that case within braces to make its scope local to a single `case` and avoid a compiler error.

Example 5.3: Use of the `switch` statement for implementing a menu selection.

```
1  int vItem;
2  cout << "Select a menu item:" << endl;
3  cin >> vItem;
4  switch (vItem) {
5      case 1:
6          cout << "Selected menu item 1" << endl;
7          break;
8      case 2:
9          cout << "Selected menu item 2" << endl;
10         break;
11     default:
12         cout << "Unknown menu item!" << endl;
13 }
```

5.1.5 Static assertions

The term *assertion* is used when we test whether an expression is true or not. At runtime we would use an `if` statement for this. However, we can also make assertions about variables declared as `constexpr` using `static_assert`. For example,

```
constexpr int a = 6;
constexpr int b = 7;
static_assert(a*b == 42, "Product of a and b is not 42.");
```

will produce a compiler error if we change the values of `a` and `b` such that their product is not 42.

5.2 Iterative Structures

Iterative structures are also known as loops. They are used in programming to repeat a specific group of statements until a terminating condition is met. The two types of loops are:

- `for` loop - typically used to execute a set of statements a fixed number of times.
- `while` and `do-while` loops - used to execute statements until a condition is met.

5.2.1 for loop

The `for` loop syntax looks as follows:

```
for ( INITIALISER; CONDITION; UPDATE ) {
    STATEMENTS;
}
```

The `for` loop works as follows:

- `INITIALISER` is zero or more comma-separated expressions which are executed once before the first iteration begins.
- `CONDITION` is required and is tested at the **beginning** of each iteration. The loop is terminated if this evaluates to `false`. If it evaluates to `true`, `STATEMENTS` are evaluated.
- `UPDATE` is zero or more comma-separated expressions which are executed at the **end** of each iteration (for example, to increment a counter).



Note

The loop may never execute if `CONDITION` evaluates to `false` at the beginning of the first iteration.

The use of the `for` loop is shown in Example 5.4.

Example 5.4: Use of a for loop `for` a countdown

```
1  #include <iostream>
2  using namespace std;
3
4  int main ()
5  {
6      for (int n = 10; n > 0; --n) {
7          cout << n << ", ";
8      }
9      cout << "FIRE!" << endl;
10     return 0;
11 }
```

5.2.2 while loop

The `while` loop has the following syntax:

```
while ( CONDITION ) {
    STATEMENTS;
}
```

The `while` loop works as follows:

- `CONDITION` (the test expression) is tested at the **beginning** of each iteration.
- If `CONDITION` is `true`, the `STATEMENTS` inside the body of the while loop block are executed.
- This repeats until `CONDITION` is `false`, after which the `while` loop is terminated.



Note

The `while` loop may never execute if `CONDITION` is `false` on entry.

The `while` loop can be used, for example, to iteratively compute a result to within a given tolerance. Example 5.5 shows the use of a while loop to perform the same count-down as Example 5.4, but with additional user input.

Example 5.5: Use of a while loop for a countdown

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

int main () {
    int n;
    string vInput;

    cout << "Enter a starting number: ";
    cin >> vInput;
    n = stoi(vInput);
    if (n < 0) {
        cout << "Invalid number entered." << endl;
        return 1;
    }
    else {
        while (n > 0) {
            cout << n << ", ";
            --n;
        }
        cout << "FIRE!" << endl;
        return 0;
    }
}
```

5.2.3 do-while loop

The `do-while` loop is a variant of the `while` loop with the important difference that the body of the `do-while` loop is executed **before** the condition is checked.

The syntax of the `do-while` loop is:

```
do {
    STATEMENTS;
} while ( CONDITION )
```

The `do-while` loop works as follows:

- The `STATEMENTS` inside the body of the loop are executed.
- The `CONDITION` is checked at the **end** of each iteration. If it is `true`, the `STATEMENTS` are executed again and the process repeats.
- The loop continues until the `CONDITION` becomes `false`, at which point the loop terminates.



Note

The `do-while` loop will always execute `STATEMENTS` at least once.

The `do-while` is used, for example, to display a menu until a user chooses a valid option.

5.2.4 Loop Keywords

There are two statements which can be used within loops to change the normal flow of a program. Sometimes, it is desirable to skip the execution of a loop for a certain test condition or terminate it immediately without checking the condition. The `break` statement breaks out of the loop. It terminates the current iteration of a loop and continues execution with the statement which immediately follows the iterative structure. The `continue` statement immediately terminates the current iteration of a loop and begins the next iteration if the condition is still satisfied. Example 5.6 shows the use of the `break` and `continue` statements.

Example 5.6: Add only positive numbers entered by the user until user enters 0

```
#include <iostream>
using namespace std;

int main() {
    float number, sum = 0.0;
    while (true) {
        cout << "Enter a positive number (0 to stop): ";
        cin >> number;
        if (number > 0.0){
            sum += number;
        }
        else if (number < 0.0) {
            cout << "Only positive numbers allowed!" << endl;
            continue;
        }
        else {
            break;
        }
    }
    cout << "The sum is: " << sum;
    return 0;
}
```



Warning

`break` statements only break out of the inner-most loop of a nested loop.

Exercise 5.1

Analyse the following code, without using a computer, and determine which numbers are output to the terminal:

```
#include <iostream>
using namespace std;

int main () {
  int n;
  for (n = 20; n > 0; n--) {
    if (n % 4 == 0) {
       continue;
    }
    cout << (20 - n) << endl;
    if (n == 5) {
      break;
    }
  }
  return 0;
}
```

Exercise 5.2

Extend the program you wrote in Exercise 4.1 to add conditioning to the input, ensuring a positive radius is entered. The user should be prompted repeatedly until a valid input is given.

Exercise 5.3

Write a program which computes an approximation to π using both of the following formulae:

$$\frac{\pi}{4} \approx \sum_{k=0}^{n_1} \frac{(-1)^k}{2k+1} \qquad \frac{\pi}{2} \approx \prod_{k=1}^{n_2} \frac{4k^2}{4k^2-1}$$

and hence find n_1 and n_2 such that these approximations are accurate to five significant figures.

Which approximation converges faster?

Exercise 5.4

Write a program which, for a user-specified (and validated) positive-integer (unsigned int), n, computes n-factorial. That is,

$$n! = 1 \times 2 \times 3 \times \cdots \times n$$

Compute some values of $n!$ up to a value of $n = 20$. What problem do you notice? How might you fix the problem?

Exercise 5.5

1. Write a program which will calculate a discretised sine function on the interval $[0, 2\pi)$; this prints the values of the sine function at the points $x_i = \frac{2\pi i}{100}$ to the screen. Thus, the ith value is computed as:
$$s[i] = \sin\left(\frac{2\pi i}{100}\right) \qquad i = 0, ..., 99$$
2. Create a file output stream object and open a new file called `sine.txt` (making sure that your file stream object is valid using the `good()` function). Write the values x_i to the file, with each value on a new line. Close the file.
3. Now write both the value of x_i and the function value $s[i]$ to the file, separating them with at least one space. This is usually all that is necessary to allow appropriate programs to read the data from the files you create. Try plotting the data in `sine.txt` using Python, MATLAB or another plotting program.
4. To make the file more readable to humans, set the precision to a maximum of 5 decimal place and the width of each column to 10 characters.

Exercise 5.6

We will now read in the file we created in Exercise 5.5.

1. Create a file input stream object and open the `sine.txt` file. Read in the file and print it to the screen in the same way as described in the lecture.
2. Now create two floating point variables and use the >> stream operator to read each item on each line separately as follows:

   ```
   f_in >> var1 >> var2;
   ```

   Print these values as you read them in and verify they are correct.

Exercise 5.7

The motion of a parachutist subject to gravitational acceleration and slowed by air resistance, described in Exercise 4.3, is derived from the model

$$m\frac{dv}{dt} = mg - cv$$

1. Numerically integrate the above equation to calculate her terminal velocity (when $t \to \infty$), using the relation
$$v(t_{i+1}) = v(t_i) + \left[g - \frac{c}{m}v(t_i)\right](t_{i+1} - t_i)$$
2. Print the values of t_i and $v(t_i)$ to a file and plot her velocity as a function of time.

Exercise 5.8

Write a program which implements Simpson's rule for approximating integrals:

$$\int_a^b f(x)\,dx \approx \frac{b-a}{3n}\left(f(a) + 4\sum_{i=1}^{n/2} f(a+(2i-1)\,h) + 2\sum_{i=1}^{n/2-1} f(a+2ih) + f(b)\right)$$

Use it to approximately evaluate the integral $\int_{-1}^{1} x^2 e^{x^2}\,dx$.

Chapter 6

Functions

Objectives

In this chapter you will learn about:

- functions, how to declare them and how to define their implementation;
- how to call a function and the difference between passing arguments by value or by reference;
- function overloading;
- recursion;
- lambda expressions;

6.1 Introduction

Definition: Function

A function (or routine, or subroutine) is a set of statements which together form a functional unit within the program. A function may accept zero or more parameters and may optionally return a value.

The purpose of using functions is to break the source code into manageable chunks, which can be called from different parts of the program and therefore allows code to be reused. Consequently, this then makes testing code easier, since individual functions can be tested in isolation.

A function is defined as follows:

```
RETURN_TYPE FUNCTION_NAME ([DATATYPE IDENTIFIER [, ...] ]) {
    STATEMENTS;
    [return [RETURN_VALUE]];
}
```

The `FUNCTION_NAME` is an identifier for this function, used elsewhere when we want to call it. It can have zero or more parameters, each declared with a `DATATYPE` and an `IDENTIFIER`, which can be

used within the function. The `RETURN_TYPE` is the data type of the value returned from the routine. The `return` keyword can be used to terminate the execution of the function and optionally specify a value to be passed back to the calling routine. Note that the datatype of `RETURN_VALUE` should be `RETURN_TYPE`.

A simple function is given in Example 6.1. It can be used (called) from elsewhere in the program as follows:

```
double vMyRadius = 4.6;
double vMyArea   = computeArea(vMyRadius);
```

In this case, the function is called `computeArea` and it has a single parameter, the radius `vMyRadius` (declared as a `double`). As specified in the first line of the function, the routine also returns a `double` (the resulting area), which we store in the variable `vMyArea`.

Example 6.1: A function to compute the area of a circle.

```
1 // Computes the area of a circle, given a radius
2 double computeArea (double pRadius) {
3     const double vPi = 3.14159;
4     double vArea;
5     vArea = vPi * pRadius * pRadius;
6     return vArea;
7 }
```



Note

`main()` is really just a function. It has a special name and is the function called by the operating system when your program executes.

6.1.1 The `void` keyword

Some functions require no value to be returned. This is specified by using the `void` keyword as the `RETURN_TYPE`. Such functions need not have a `return` statement at the end. However, a `return` statement (without a value) can be used to exit the routine before the end and return to the point in the program at which the function was called. Example 6.2 shows a simple function which does not need to return a value.

Example 6.2: The `void` return-type

```
1 void printValue ( double pVal ) {
2     cout << "Value is: " << pVal << endl;
3 }
```



Note

The `void` keyword was also used in the C language to indicate a function had no parameters. For example,

```
int MyFunction(void);
```

This is not necessary, or considered standard practice, in C++. Instead, such a function should be specified simply as

```
int MyFunction();
```

6.2 Declaration and definition

As we have seen with variables, we must declare them before they can be used. The same is true of functions. If we define a function earlier on in the source code than the place from where it is called, everything compiles fine. However, with functions this can become more troublesome if there are many different functions and they call each other in a non-hierarchical manner.

If a function is called from the `main()` function, but defined after it, the compiler will report an error because within the `main()` function it is not yet aware of the existence of our function, the types of arguments which can be passed to it and the return type. In C++, the function prototype is a declaration of the function without its body to give the compiler the information it needs about the function. A function must always be declared before it can be used, but its implementation can then appear later in the file.

Definition: Function Prototype

A function prototype is a declaration of a function that specifies the function's name, data types of parameters, and return type, but omits the body of the function.

An illustration of how function prototypes are used to declare functions is given in Example 6.3. The declaration on line 2 ensures the compiler knows about `MyFunction` before it first encounters its use in line 5. The actual definition of the function body can now come at some later point in the file (lines 9-11).

Example 6.3: Function declaration

```
1  // Declaration
2  int MyFunction (double x, double y);
3
4  int main() {
5      int a = MyFunction(1.2, 5.3);
6  }
7
8  // Definition
9  int MyFunction(double x, double y) {
10     ...
11 }
```



Note

The parameter names are not needed in a declaration, only their data types, since the purpose of the names is solely to identify the parameters within the function body:

```
int MyFunction(double, double);
```

However, including them in the prototype is perfectly acceptable and acts as a useful reminder as to the purpose of the parameter.

6.3 Passing arguments to functions

Definition: Parameter and Argument

An *argument* (or actual parameter) is the value passed to a function when it is called.

A *parameter* (or formal parameter) is a declared variable in the function prototype which has an identifier and datatype, and acquires the value specified by the corresponding *argument* when the function is called.

Recall the `computeArea` function from Example 6.1.

```
double computeArea (double pRadius) {
    const double vPi = 3.14159;
    double area;
    area = vPi * pRadius * pRadius;
    return area;
}
```

This function was called from the `main()` routine as

```
int main() {
    double vMyRadius = 4.6;
    double vMyArea = computeArea(vMyRadius);
}
```

In programming, arguments refer to the data which is passed to a function while calling it; in the above example the value of `vMyRadius` is passed to the function `computeArea` as an argument. The value of the parameter `pRadius` is then initialized to this value at the beginning of the function.

6.3.1 Passing by value

In the example above the identifier `pRadius` refers to a *copy* of the data referred to by `vMyRadius` in the calling function. `pRadius` also has a scope local to the `computeArea` routine and is therefore destroyed when the function completes. This is called *Passing by Value*, since the actual value is given to the function. In this case, any changes made to `pRadius` during the execution of the functionwould be lost at the end of the routine.

If the changes to a function parameter need to affect the argument variable in the calling routine, we can use *Passing by Reference*.

6.3.2 Passing by Reference

Passing by *reference* does not create a copy of the original data. Instead, the function parameter refers to the same location in memory as the original variable used as the argument to the function.

Changes made through the function parameter identifier therefore affect this memory location and therefore the original variable. We use the *reference operator*, `&`, on the right of the datatype for those parameters we wish to pass by reference.

There are some good reasons to pass by reference:

- The size of the argument is large and duplicating it in memory would unnecessarily degrade performance.
- A function can only have a single return value. Passing by reference allows additional data to be "returned" from the function.

A simple example of passing by reference is given in Example 6.4. This routine is called in a similar way as before.

```
double vMyVal = 3.0;
tripleValue(vMyVal);
cout << "Value is now: " << vMyVal << endl;
```

Example 6.4: Passing by Reference

```
1 void tripleValue(double& pVal) {
2     pVal *= 3.0;
3 }
```

The function `tripleValue` does not return a value. Note that the value of `vMyVal` after calling `tripleValue` is now 9.0. Without the reference operator, ampersand (`&`), it would still be 3.0 since, although the function still multiplies the value by three, the result would be lost at the end of the function as it is acting on a *copy* of `vMyVal`. We can say that `vMyVal` and `pVal` are just two identifiers (views) for the same memory location.

6.4 Default values

Parameters to functions can be assigned default values. This is most useful when parameters take a 'normal' value most of the time but still allows the user to override the default value if needed. Parameters with default values must always appear **last** in the parameter list. If specifying an alternative value for a parameter, values for all parameters up to that point in the parameter list must be explicitly specified.

Example 6.5: Specifying default parameter values

```
1 void print(double pVal, string pLabel="Value: ", string pUnits="cm") {
2 cout << pLabel << pVal << pUnits;
3 }
```

To illustrate the use of default parameter values, consider the code in Example 6.5. Both `pLabel` and `pUnits` have default values which are used in the case that arguments are not explicitly given. For example:

```
print(5.0);
```

would print `Value:  5.0cm`. We could change only the label by instead writing

```
print(5.0, "My value: ");
```

which would now print `My value:  5.0cm`. However, if we wish to specify an alternative value for `pUnits`, we must also explicitly give a value for `pLabel`,

```
print(5.0, "Value: ", "m");
```

6.5 Constants in functions

Parameters of functions can be declared as `const`. This tells the compiler that the implementation of the function should not attempt to change the value of the parameter, which might help detect subtle errors.

The return value of a function can also be declared `const`. This is useful only when returning a value (declared outside of the function) by reference and becomes more relevant in the context of Classes (see Chapter 9).

If the result of a function can be evaluated at compile-time it can be declared as `constexpr` and can be used to initialise variables declared as `constexpr`.

Example 6.6: Using `const` parameters in a function declared as `constexpr`. Since the function can be evaluated at compile-time, it can be used to initialise a variable declared as `constexpr`.

```
1 constexpr int Multiply(const int a, const int b) { return a * b; }
2 ...
3 constexpr int var = Multiply(2, 3);
```

6.6 Function overloading

A function definition must be unique. A *fingerprint* for a function definition is specified by:

- The function identifier;
- The number of parameters;
- The datatypes of those parameters, including whether they are passed by value or reference;
- **NOT** the return type.

It is possible to have two different functions with the same name, as long as the parameter list is different. The *compiler* will automatically decide which function is to be called, based on the arguments given. An example of overloading the function `sqr` is given in Example 6.7. The compiler will call the integer version for the argument `5`, while it will call the double-precision version for the argument `3.14159`. The output is therefore:

```
Integer Square
25
Floating-point Square
9.86959
```

Example 6.7: Overloaded functions

```
int sqr (int pVal) {
    cout << "Integer Square" << endl;
    return pVal * pVal;
}

double sqr (double pVal) {
    cout << "Floating-point Square" << endl;
    return pVal * pVal;
}
...
cout << sqr(5) << endl;       // Calls the first version
cout << sqr(3.14159) << endl; // Calls the second version
```

6.7 Recursion

Definition: Recursion

Recursion is the definition of an operation in terms of itself.

A function that calls itself is known as recursive function. It therefore has the following structure:

```
RETURN_TYPE FUNCTION_NAME([DATATYPE IDENTIFIER [, ...] ]) {
    STATEMENTS_1
    FUNCTION_NAME([ARGUMENT [, ...] ]);
    STATEMENTS_2
}
```

Let $F_n = n!$. Then we can define this recursively as $F_n = n \times F_{n-1}$ with $F_1 = 1$. An implementation is shown in Example 6.8.

Example 6.8: Factorial of a number using recursion

```
#include <iostream>
using namespace std;

int factorial(int);

int main() {
    int n;
    cout<< "Please enter a number to find its factorial: ";
    cin >> n;
    cout << "The factorial of " << n <<" is " << factorial(n);
    return 0;
}

int factorial(int n) {
    if (n > 1) {
        return n*factorial(n-1);
    }
    else {
        return 1;
    }
}
```

Recursion is a common and elegant method of solving some problems such as computing some mathematical expressions or sorting data. Examples include:

- Computing factorials, Fibonacci series;
- Sorting: merge sort, quick sort;
- Searching: binary search.

However, while it can simplify some problems, it can complicate others. There are also dangers associated with recursion. Recursing too many times may lead to a program reaching the maximum size of the stack, which keeps track of which functions have been called by a program and in which order. Measures also have to be taken to prevent infinite recursion.



Warning
You must ensure that there is always a terminating condition.

6.8 Lambda Expressions

Definition: Lambda Expression

A lambda expression, (anonymous function) is a function definition that is not bound to an identifier and can capture variables in scope.

Lambda expressions allow us to write functions which can be easily passed as arguments to a function. Variables from the local context can be ”captured” and used in the function without being passed in explicitly as parameters.

A lambda expression has the structure

```
[ CAPTURES ] ( PARAMETER_LIST ) -> RETURN_TYPE { STATEMENTS }
```

The `CAPTURES` component determines which variables in the local scope are made available to the code in `STATEMENTS` and how. For example,

```
[&factor] (int x, int y) { return factor * (x + y); };
```

is a lambda expression which captures the variable `factor` by reference from the local context and just returns the scaled sum of its parameters. All local variables can also be captured by-reference (using just `[&]`) or by-value/copy (using `[=]`).

The available capture options are:

`[]`	Captures nothing
`[=]`	Captures all local variables by-value
`[&]`	Captures all local variables by-reference
`[x, y, &z]`	Captures only x and y by-value and z by-reference.

Generally, we do not need to specify the `RETURN_TYPE` since the compiler will work this out itself.



Note
Lambda expressions will become more useful to us later in the course, particularly in the context of the Standard Template Library (STL), see Chapter 8.7.2.

6.9 Namespaces

Large programs with potentially hundreds of functions (and Classes, see Section 9) may make it challenging to organise the code effectively. In the worst case, naming conflicts may occur between functions which perform related functionality but in a different part of the code.

Namespaces are a mechanism for organising functions and objects in C++. Most of the core C++ functions and the Standard Template Library (see Chapter 8) are within the standard namespace `std`. This means that by default, no user-defined routines will conflict with standard C++ routines.

A function can be put in a namespace by enclosing it within a `namespace` block. The use of a namespace, containing a function `mylog`, is illustrated in Example 6.9. The function can then be called with the fully-qualified name, with the namespace and function name separated by the scope operator `::`.

In very large programs or libraries of code, namespaces may also be nested if needed.

```
namespace myprogram {
    namespace core {
        int factorial(int n) {
            ...
        }
    }
}
...
myprogram::core::factorial(4);
```

If calling a function from within the same namespace, the namespace can be omitted.

6.10 Exceptions and Exception Handling

Exceptions are a mechanism for handling situations where unexpected errors occur. For example, an exception could indicate a file cannot be written to because there is no space on the disk, or a value entered by the user is unsuitable. They allow information about the error to be propagated from the location in the code where the error is identified to a place in the code where it can be handled.

When an error occurs, and the function in which it occurs cannot manage it appropriately, an exception can be *thrown*. An exception can be *caught* by a calling routine which can handle the error. If the exception propagates all the way back to the main routine and is not handled there, the program will terminate. Many standard routines in C++ will throw errors. We can therefore write code to catch these errors and handle them in a more elegant way than aborting our program.

An exception is thrown using the `throw` keyword. There can be different kinds of errors indicated by the data type of the exception being thrown. Examples of types of exceptions include:

- `std::overflow_error`: e.g. the result of a calculation exceeds the range of a datatype;
- `std::bad_alloc`: e.g. failure to allocate storage on the heap;
- `std::logic_error`: e.g. indicate the parameter to a function violate some conditions.

Exceptions are caught using a `try` block, followed by one or more `catch` blocks. The *normal* program flow is to run those statements in the `try` block. If one of those statements throws an exception, execution jumps immediately to the `catch` block with a type matching that of the exception (and the remaining statements in the `try` block are skipped).

Throwing and catching exceptions is also shown in Example 6.9.

Example 6.9: Throwing and catching exceptions.

```cpp
#include <iostream>
#include <exception>
#include <cmath>

namespace mymath {
    double mylog(double num) {
        if (num <= 0) {
            throw std::logic_error("Number must be > 0.");
        }
        return std::log(num);
    }
}

int main() {
    try {
        std::cout << mymath::mylog(1) << std::endl;
        std::cout << mymath::mylog(-1) << std::endl;
    }
    catch (const std::logic_error& e) {
        std::cout << "An error occured: " << e.what() << std::endl;
    }
    catch (const std::bad_alloc& e) {
        std::cout << "No allocations here, so should not be called!"
        << std::endl;
    }
}
```

Exercise 6.1

Write a program which allows users to convert temperatures from Celsius to Fahrenheit and back. Ensure your program checks the choice of conversion the user chooses is a valid choice. Write separate functions for each conversion direction.

Exercise 6.2

1. Write a function which takes two integers, a and b, and swaps them. Recall that for the values within the *calling* function to be swapped, they must be passed by reference to your swap function.
2. Extend the program to accept three integers as input, and uses the swap function to sort the numbers in increasing order, printing out the result.

Exercise 6.3

The deflection of a cantilever supported beam loaded loaded by a concentrated force at the free end is:

$$y = \frac{F}{6EI}\left(x^3 - 3Lx^2\right)$$

Extend your solution to Exercise 4.4 to find the deflections of either the simply supported beam or the cantilever beam, as selected by the user at runtime.

1. Write separate functions for each type of beam which calculate the deflection at a given position x.
2. Extend the program to calculate the deflections at $1\,\text{cm}$ intervals along the full length of the beam, write both the position x and the deflection $y(x)$ to a file and plot the result in your favourite plotting program.

Exercise 6.4

1. Copy the program written in Exercise 5.4 and modify it to compute the factorial in a separate function, which accepts the sole parameter n.
2. Using your factorial function, write a new function to compute the binomial coefficient of n and k (the number of ways of choosing k objects from n) given by,

$$\binom{n}{k} = \frac{n \cdot (n-1) \cdots (n-k+1)}{k \cdot (k-1) \cdots 1} = \frac{n!}{k!(n-k)!} \quad \text{if } n \geq k \geq 0$$

   for a value of n and k given by the user.
3. Add exception handling to your program. Throw an exceptions from your functions if the parameters are inappropriate and catch them in the `main()` routine.
4. Finally, put your factorial and binomial coefficient functions into a namespace of your own.

Exercise 6.5

Using recursion, write a program which computes the first 15 Fibonacci numbers. You should write a function which computes the n-th number in the Fibonacci sequence x_n, given that

$$x_n = x_{n-1} + x_{n-2} \quad \text{and} \quad x_0 = x_1 = 1$$

Exercise 6.6

(Advanced) Pascal's triangle is formed as follows

```
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
      1   5   10  10  5   1
      ...
```

- The top entry (row $n = 0$) is always 1.
- There are $n + 1$ numbers on the n-th row.
- The first and last entry on any row is always 1.
- Each entry in the triangle is computed as the sum of the two values immediately above it.

Write a program for computing the following aspects of Pascal's triangle:

1. Write a function which recursively computes the mth value on the nth row. Remember to take account of the special case that $m == 0$ or $m == n$ as this is always 1 and so it should not recurse further.
2. Write a function which prints out the nth row of Pascal's triangle.
3. Write a function which returns the sum of all the numbers on the nth row of Pascal's triangle. What do you notice about these sums?
4. The Fibonacci sequence from exercise 6.5 can also be computed using Pascal's triangle: the nth term is the sum along the nth diagonal.

   Add a function which computes the nth number in the Fibonacci sequence recursively using your function from part 1. Use it to compute the first 15 Fibonacci numbers.

   For example, the 6th number in the sequence is computed as the sum of the selected numbers in Pascal's triangle below:

   ```
   1
   1   1
   1   2   1
   1   3  [3]   1
   1  [4]   6   4   1
  [1]  5   10  10   5   1
   ```

   HINT: Starting with the nth row (and so set $i = n$), sum the $(n - i)$th element of the ith row, decreasing i each time until $i < n - i$.

Chapter 7

Arrays, Pointers and Dynamic Memory

Objectives

In this chapter you will learn about:

- one-dimensional arrays, how to declare them and use them;
- multi-dimensional arrays;
- pointers and their use;
- dynamic memory allocation and why it is useful.

7.1 C-style Arrays

Programs often have to handle more than just a handful of variables. Often we have a group of variables which are intrinsically related conceptually, share a common data type and are manipulated in exactly the same way. In these cases, an *array* is an effective data structure.

Definition: Array

An array is a fixed number of elements of the same datatype, accessed using a linear index.

Figure 7.1 illustrates how multiple elements of different datatypes are stored in memory.

7.1.1 Declaring a C-style Array

When we declare an array, we ask the operating system for a **contiguous block of memory** of the correct size to store all the elements of our array. It will allocate the memory to our program and tell us the memory address of the **first** location.

An array is declared using a similar syntax to a single variable, except we specify the number of elements we want in our array.

```
DATATYPE IDENTIFIER[SIZE] = {VALUE_LIST};
```

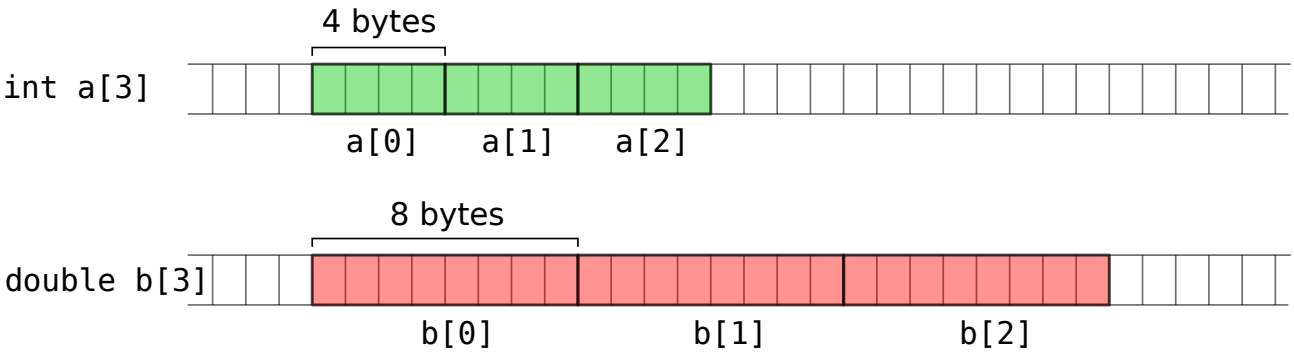


Figure 7.1: Example layout of two arrays in memory. Elements are stored contiguously. The size depends on both the number of elements and the size of each element. An `int` is typically four bytes in size on a modern computer, while a `double` is typically eight bytes.

`DATATYPE` is the data type of each individual element, `IDENTIFIER` is a name for the array and the `SIZE` in the square brackets ("[ ]") gives the integer number of elements in the array. Optionally, the contents of the array can be specified in a comma-separated list of values enclosed in curly braces ("{ }").

Example 7.1: Declaring an Array

```
1 int vMyArray[5];                          // Uninitialised array
2 int vMyArrayFilled[3] = {1,2,3};          // Initialized array
3 int vMyImplicitSizeArray[] = {0,3,6,9,12}; // Implicit array size
```

Some examples of how to declare arrays are given in Example 7.1. The first case gives us five integer memory locations. Each location can be accessed using the identifier and an index:

```
vMyArray[2] = 3;
int vThirdElement = vMyArray[2];
```



Note

Arrays are indexed from zero. The first element has index 0, the second has index 1, etc.



Warning

In the declaration of `vMyArray` in Example 7.1, all values are *uninitialised* when first declared; their value is whatever happened to be in that piece of memory before initialization.

A common error is to accidentally use an uninitialised value which will cause your program to have unpredictable behaviour.

The second and third declarations in Example 7.1 also define the initial values of the elements in the array. In the latter, the size is implied from the number of values specified in the initialisation list. Importantly, the size of the arrays we have defined must be **known** at compile-time. For instance, the following code is **not** valid C++:

```
unsigned int vSize1 = 5;
unsigned int vSize2;
cin >> vSize2;
double vMyArray1[vSize1];
```

```
double vMyArray2[vSize2];
```

However, an integer variable that is defined as constant can be used to specify the size of an array, as follows

```
const unsigned int vSize = 5;
double vMyArray[vSize];
```



Note

This section concerns C-style arrays, which are particularly useful for understanding memory management in C++.

C++ also provides a *container* called `std::array` as part of the Standard Template Library (STL) which encapsulates a C-style array and matches their performance and accessibility while providing additional functionality. See Section 8.1.

7.1.2 Using a C-style Array

Elements of an array are usually manipulated in identical ways. The simplest way to access the elements of an array is using loops as show in Example 7.2:

Example 7.2: Using a `for`-loop to manipulate an array.

```
1 const unsigned int vSize = 20;
2 double vMyArray[vSize];
3 for (unsigned int i = 0; i < vSize; ++i) {
4     vMyArray[i] = 2.0 * i;
5 }
```

To declare an array as a parameter to a function use square brackets after the identifier name:

```
void myRoutine(double pArray[]) {...}
double norm2(int pSize, double pArray[]) {...}
```

To use an array as an argument when calling such a function, we just specify the identifier for the array:

```
double testArray[] = {0.0, 1.0, 2.0, 3.0, 4.0, 5.0};
myRoutine(testArray);
cout << norm2(6, testArray) << endl;
```

7.1.3 Multi-dimensional Arrays

C++ allows multidimensional arrays, they can be thought as as ”arrays of arrays”. The general form of a multidimensional array declaration is:

```
DATATYPE IDENTIFIER[SIZE1][SIZE2]{...[SIZEN]} = {VALUE_LIST};
```

Multidimensional arrays are just a helpful abstraction, since the same results can be achieved with a simple one-dimensional array, by multiplying its indices:

```
DATATYPE IDENTIFIER[SIZE1*...*SIZEN]} = {VALUE_LIST};
```

An example of declaring and using a 2D array is given in Example 7.3.

Example 7.3: 2D Array

```
#include <iostream>
using namespace std;

int main() {
    int vMy2DArray[3][2] = { {0, 1}, {2, 3}, {4, 5} };

    // Accessing two dimensional array using nested for loops
    for(int i = 0; i < 3; ++i) {
        for(int j = 0; j < 2; ++j) {
            cout << "vMy2DArray[" << i << "][" << j << "] = "
                 << vMy2DArray[i][j] << endl;
        }
    }

    return 0;
}
```

7.2 Pointers

When an array is declared, the memory address of the first element is returned by the operating system. All elements in the array are then accessed by determining their memory location relative to this starting address. A variable which holds a memory address is known as a *pointer*.

Definition: Pointer

A pointer is an identifier whose value is a memory address. The pointer *data type* corresponds to the type of the data at the target address.

Pointers are a powerful concept inherited from C that allow direct access to computer memory. They are implicitly used when working with arrays and they are explicitly needed when using *Dynamic Memory*, discussed in Section 7.3.

A pointer is declared by prepending an asterisk "`*`" to the identifier. For example

```
int *a;
```

would declare a *pointer to an integer*.

The memory address of an identifier is found using the reference (address-of) operator "`&`". This memory address can then be assigned to a pointer.

These concepts are illustrated graphically in Figure 7.2. In this diagram the variable `x` is an `int`, while the variable `y` is a `double`. Their location in memory is obtained using the *address* operator (`&`) and stored in pointers, which have types `int *` and `double *`, respectively. The asterisk (`*`) here indicates that `a` and `b` are *pointers to an int or double*, but they do not hold an `int` or `double` themselves.

Pointers are always four bytes in size as memory addresses are essentially just integers.

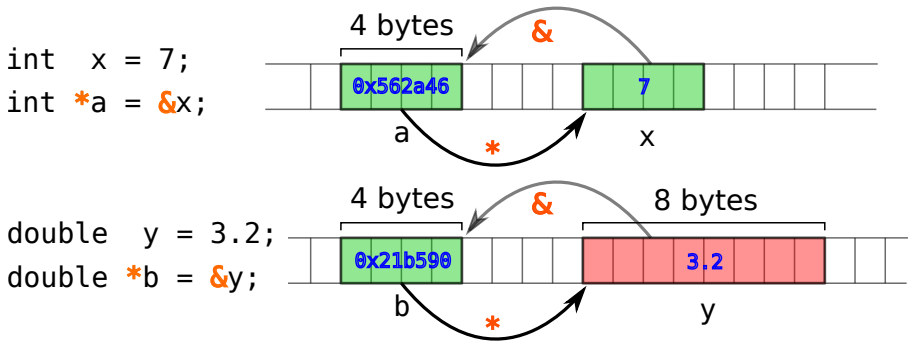


Figure 7.2: Declaring pointers to an `int` and `double`.



Note

The asterisk may be positioned next to the datatype, next to the identifier or with a space either side.

However, if multiple pointers are declared in a single statement, each identifier must be explicitly marked as a pointer:

```
int *a, *b;
```

It is recommended to declare only one pointer per statement to avoid ambiguity.



Warning

The datatype of the pointer should match the datatype of the data pointed to. For example, if `b` was of type `int *` in Figure 7.2, then dereferencing `b` would return only the first four bytes of `y`, interpreted as an integer.

The asterisk is also used as the *dereference* operator in order to retrieve the value *pointed to* by the pointer. In Example 7.4 we create a variable called `vMyData` and assign an approximation of π to it. On line 2, we declare a *pointer to a* `double`. The initial value assigned to the pointer is the memory address of `vMyData`, found by using the *address-of* operator.

The third line prints out the contents of `vMyPointer`. This is a memory address, so might be something unintelligible like:

```
0x558fbf84cc20
```

Usually we are more interested in the data at that memory address. The asterisk, on line 4, is used to *dereference* the pointer and retrieve the data at the memory location stored in the pointer – that is, the value of `vMyData`.

Example 7.4: Using pointers and dereferencing them.

```
1 double  vMyData    = 3.14159;   // vMyData variable declaration.
2 double *vMyPointer = &vMyData;  // store address of vMyData in vMyPointer
3 cout << vMyPointer << endl;     // print the address stored in vMyPointer
4 cout << *vMyPointer << endl;    // print the value at the address in
5 pointer
```

7.2.1 Assigning Pointers

In the same way that variables can hold different values, pointers can point to different memory locations during their lifetime, since they are themselves just variables which hold a memory address. They can therefore be assigned in exactly the same way as variables, as illustrated in Example 7.5:

Example 7.5: Assigning Pointers

```
double  vK1 = 0.0;          // vK1 variable declaration.
double  vK2 = 1.0;          // vK2 variable declaration.
double  vK3 = 2.0;          // vK3 variable declaration.
double *vPointer = &vK1;    // store address of vK1 in vPointer
cout << *vPointer << endl;  // print the value in *vPointer
vPointer    = &vK2;         // store address of vK2 in vPointer
cout << *vPointer << endl;
vPointer    = &vK3;         // store address of vK3 in vPointer
cout << *vPointer << endl;
*vPointer  += vK2;          // add vK2 to *vPointer
cout << *vPointer << endl;
cout << vK3 << endl;        // vK3 and *vPointer have the same value
```

7.2.2 Pointers and Arrays

The concepts of pointers and arrays are strongly related. In fact, pointers and array identifiers are interchangeable in many cases. The array identifier is conceptually a pointer to the first element of the array; that is, `vMyArray[0]` and `*vMyArray` mean the same. Other elements in the array can be accessed as if `vMyArray` was a pointer:

```
cout << *(vMyArray + 2) << endl;
```

Here we have used *pointer arithmetic* to move to a memory location relative to the original pointer. We can move to another point relative to the memory address `vMyArray` using any of the standard arithmetic operations (`+`, `-`, `++`, `--`). Example 7.6 shows how we might print the contents of an array by using pointers and pointer arithmetic.

Example 7.6: Printing an array using pointers and pointer arithmetic.

```
const unsigned int size = 5;
double vMyArray[size];
double * vMyPointer = &(vMyArray[0]);  // Could just write vMyArray
for (unsigned int i = 0; i < size; ++i) {
    cout << *(vMyPointer) << endl;
    ++vMyPointer;
}
```

7.2.3 Null pointers and Initialisation

Consider the following line, which creates a pointer:

```
double * vPointer;
```

Immediately executing the statement

```
cout << *vPointer << endl;
```

will usually print an arbitrary number, or cause a *Segmentation Fault*.

Definition: Segmentation Fault

A segmentation fault is an error which occurs when a program attempts to access a memory location which as assigned to another program, the operating system itself, or has not been allocated at all.

The issue with the lines of code above is that the pointer `vPointer` has not been initialised to point to a valid memory location. By *valid* we mean that the memory location has been allocated for use by our program by the operating system. Therefore, it must be initialised before it can be dereferenced. When it is not possible to assign a meaningful address at declaration time, it is good practice to make it a *null pointer*.

Definition: Null pointer

A null pointer is a pointer which has been assigned the value `nullptr`.

Dereferencing a null pointer also causes a **segmentation fault**. However, you can reliably test if a pointer is a `nullptr` to determine if it is safe to dereference it, for example

```
if (vPointer) {
    cout << *vPointer << endl;
}
```

7.2.4 Shared Pointers

The challenge with pointers is that in complex programs it can be difficult to keep track of their validity when objects are being shared between several parts of the program. If one part decides to destroy the object, the other part has no way to know that the pointer it has no longer points to a valid object.

To overcome this, C++11 introduced *shared pointers*. Conceptually, a shared pointer is composed of:

1. A normal pointer to some resource in memory
2. A reference counter

The reference counter keeps track of the number of shared pointers referencing a given object. Each time the shared pointer is copied (e.g. when passing it to a function by value) the count is incremented. When a shared pointer goes out of scope, the count is decremented. When the count reaches zero, the resource referenced by the pointer is destroyed. This mechanism ensures that the resource remains until the last reference to it is destroyed and also ensures the resource is deallocated which helps to avoid *memory leaks*.

Shared pointers can be dereferenced and used in much the same way as normal pointers.

To use shared pointers, you must include the `<memory>` header:

```
#include <memory>
```

A shared pointer can be created with:

```
std::shared_ptr<TYPE> IDENTIFIER { new TYPE {[INITIALIZATION-LIST]} };
```

where `IDENTIFIER` is the name of the shared pointer, `TYPE` is the datatype of the resource referenced by the pointer and the optional `INITIALIZATION-LIST` can be used when initializing that resource.

A better way is to use `std::make_shared`:

```
std::shared_ptr<TYPE> = std::make_shared<TYPE>();
```

To illustrate how shared pointers work, consider Example 7.7. We use `std::make_shared` to create a shared pointer to a single `int` with the initial value of 2. There is one reference to this resource at the moment (`x`) and so the reference count is 1 (you can use `x.use_count()` to get the count).

We then call `fun2`, passing the shared pointer by reference. Since this does not make a copy of the shared pointer, the reference count of `x` (and `y`) is still 1. We pass `y` to `fun1` by value which creates a copy of the shared pointer, incrementing the count by 1. Now each of `x`, `y` and `z` will show a reference count of 2.

When `fun1` ends, `z` goes out of scope and is destroyed, while the count is reduced by 1. At the end of `main()` the shared pointer `x` goes out of scope and is destroyed, reducing the count to zero, at which point the resource is deallocated from memory.

Example 7.7: Working with shared pointers.

```
#include <memory>

void fun1(std::shared_ptr<int> z) {    // shared pointer passed by value
    *z = 1;                            // now set to 1
}
void fun2(std::shared_ptr<int>& y) {   // shared pointer passed by reference
    fun1(y);
}
int main() {
    auto x = std::make_shared<int>(2); // initialise to 2
    fun2(x);
}
```

7.2.5 Unique pointers

Unique pointers instead do not allow the pointer to be shared, so it cannot be copied or passed to a function by value. Instead it can be *moved* from one unique pointer to another, after which the original unique pointer is empty. It should be used where strict ownership of a resource needs to be enforced. For example:

```
auto x = std::make_unique<int>(2.0);    // x is of type std::unique_ptr<int>
std::unique_ptr<int> y;                 // create an empty unique pointer
y = std::move(x);                       // pass ownership of the int to y
```

7.3 Dynamic Memory

The fixed size arrays we have seen so far are allocated memory in a reserved area assigned to your program. Fixed size arrays have a number of limitations that can lead to problems and bugs:

Exceeding the maximum (allocated) size of the array: We ideally need to be able to allocate an array whose size is determined at runtime, possibly based on user input and, most of the time, choosing a reasonable maximum size is often impossible.

Reserved memory is relatively small: Large arrays suitable for real data sets may be too large to fit within the area of memory reserved for your program.

No resizing: Using a small size may be more efficient for the typical data set, but prevents the program from running with larger data sets. If the array limits are not checked, large data sets will run over the end of an array with unpredictable consequences.

These shortcoming can be addressed by dynamically allocating an array on the *heap*, of the right size at runtime, or reallocating an array when it needs to be expanded. This is known as *Dynamic memory allocation*.

Definition: Heap

The heap is the area of memory which can be assigned to a program at runtime as required.

Definition: Dynamic Memory

Dynamic memory is memory allocated to a program during its execution. It is allocated from the heap using the `new` keyword, and must be explicitly released back to the operating system with the `delete` keyword.

To allocate a variable on the heap using dynamic memory allocation, we use the `new` operator with the following syntax:

```
DATATYPE *IDENTIFIER = new DATATYPE;
```

The `new` operator returns the address of the memory allocated, which is stored in a pointer.

For example, we could request an integer on the heap, and manipulate it, with the following statements:

```
int* vPtr = new int;
*vPtr = 7;
```

The integer can be released using the statement

```
delete vPtr;
```

which will leave the pointer as a null pointer (so it should not subsequently be dereferenced). However, using dynamic memory allocation for arrays is much more useful as the array size can be chosen at runtime.

7.3.1 Arrays

To allocate a dynamic array we first declare a pointer which will hold the memory address returned by the operating system. We then use the `new` operator to request the operating system allocates memory,

```
DATATYPE* IDENTIFIER = new DATATYPE[SIZE];
```

`DATATYPE` is the data type of the array elements, `IDENTIFIER` is a name of the **pointer** to the start of the allocated memory and `SIZE` is the number of elements in the array, which can be determined at runtime. An example showing a user-defined array allocation is given in Example 7.8. If the operating system was unable to allocate the requested memory, possibly due to insufficient memory, an exception will be thrown, which can be caught using a `try...catch` block (see Section 6.10).

In order to release a dynamic array, the `delete` operator is used to free memory when the array is no longer needed.

```
delete[] <identifier>;
```

The `delete[]` keyword is used to delete an entire array associated with a pointer. Any attempt to reference array elements will cause a *segmentation fault*. If the pointer goes out of scope (and it has not been copied elsewhere), we no longer know the location of the array in memory and consequently cannot deallocate it. This will cause a **memory leak**.

Example 7.8: Allocating Dynamic Memory

```
int N;
cin >> N;   // We would normally sanitise this!

double *vArray = nullptr;
try {
    vArray = new double[N];
}
catch (const std::bad_alloc& e) {
    cout << "Unable to allocate memory!" << endl;
    return 1;
}
```

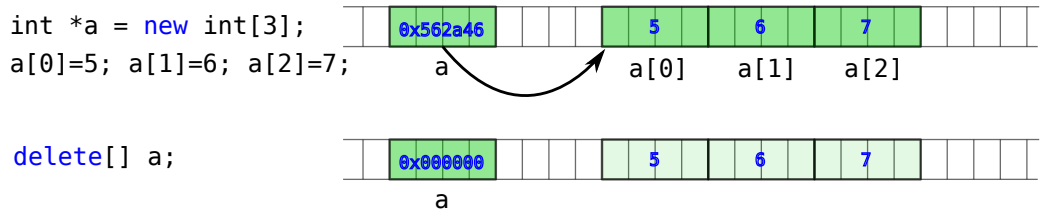


Figure 7.3: Allocation and deallocation of an array of three integers using the `new` and `delete[]` keywords.

Definition: Memory leak

Memory leak occurs when a program fails to release memory assigned to it.

Figure 7.3 illustrates the allocation and deallocation of an array of three integers. After calling `delete[] a;`, the memory is returned to the operating system. The pointer `a` is still available to our program. Unless the pointer is about to go out of scope it is good practice to set it to be a `nullptr`. This allows it to be safely tested for equality with `nullptr` at some later point to determine if it points to allocated memory.

Example 7.9 illustrates dynamic memory allocation (and reallocation) by reading a sequence of integers from the user without knowing the total number at the beginning. After reading each integer, if the array is too small, the array is reallocated; the old values are copied across, the new value appended, and the old array deallocated.

Example 7.9: Resizing an array

```
int max = 10;                       // non-const size
int *vArray = new int[max];         // initially allocated
int n = 0;
// Read numbers into the array
while (cin >> vArray[n]) {
    n++;
    if (n >= max) {
        max *= 2;                   // double the previous size
        int *vTemp = new int[max];  // create new bigger array
        for (int i=0; i<n; i++) {
            vTemp[i] = vArray[i];   // copy values to new array
        }
        delete [] vArray;           // free old array memory
        vArray = vTemp;             // now vArray points to temp array
    }
}
```

Dynamic memory allocation performance

Dynamically allocating memory on the heap is more expensive than creating a normal variable (or array) on the stack. If your program repeatedly allocates and deallocates memory on the stack, this may cause a performance problem. The code in Example 7.9 mitigates this to some extent by allocating a new array which is twice the size of the previous array, rather than being just large enough to hold the additional value.

7.3.2 Matrices

There are two ways one would typically consider storing a dynamically allocated $N \times M$ matrix. One could allocate an array of pointers to arrays of numbers (by row or column), thereby accessing them using the `array[i][j]` syntax. Alternatively, one might allocate a single array of numbers containing all the entries of the matrix row-by-row, or column-by-column and access entries using `array[i*M+j]` syntax. Example 7.10) and Example 7.11 illustrate these approaches. Although less element, the second approach is more performant and memory efficient.

Example 7.10: Creating a matrix as an array of pointers to arrays

```cpp
#include <iostream>
using namespace std;

int main(){
    int N, M;
    cout << "Enter number of rows N: ";
    cin >> N;
    cout << "Enter number of columns M: ";
    cin >> M;

    // dynamically create array of pointers of size N, one for each row
    int **vMatrix = new int*[N];

    // dynamically allocate memory (arrays) of size M for each row
    for (int i = 0; i < N; i++){
        vMatrix[i] = new int[M];
    }

    // assign values to allocated memory
    for (int i = 0; i < N; i++){
        for (int j = 0; j < M; j++){
            vMatrix[i][j] = i*M+j;
        }
    }

    // print the 2D array
    for (int i = 0; i < N; i++){
        for (int j = 0; j < M; j++){
            cout << vMatrix[i][j] << " ";
        }
        cout << endl;
    }

    // deallocate memory using delete[] operator
    for (int i = 0; i < N; i++){
        delete[] vMatrix[i];
    }
    delete[] vMatrix;

    return 0;
}
```

Example 7.11: Creating a matrix using a single array of numbers

```cpp
#include <iostream>
using namespace std;

int main(){
    int N, M;
    cout << "Enter number of rows N: ";
    cin >> N;
    cout << "Enter number of columns M: ";
    cin >> M;

    // dynamically allocate memory of size M*N
    int *vMatrix = new int[M * N];

    // assign values to allocated memory
    for (int i = 0; i < N; i++){
        for (int j = 0; j < M; j++){
            vMatrix[i*M+j] = i*M + j;
        }
    }

    // print the 2D array
    for (int i = 0; i < N; i++){
        for (int j = 0; j < M; j++){
            cout << vMatrix[i*M+j] << " ";
        }
        cout << endl;
    }

    // deallocate memory
    delete[] vMatrix;

    return 0;
}
```

Exercise 7.1

Pseudo-random numbers can be generated in C++ using the `rand()` function. (They are called *pseudo*-random since they are not truely random, but are produced by a deterministic algorithm.) The fixed, read-only constant, `RAND_MAX`, defines the range of numbers generated with a uniform distribution. Note that you may need to `#include <cstdlib>` for your code to compile.

1. Write a program which creates an array of 100 random integers;
2. Adapt this program to generate an array of 100 floating point numbers between 0 and 100;
3. Add functions which compute the minimum, maximum and mean (average) of the values.

Run your program several times. What do you notice?
Prompt the user for an integer x at the start of your program and call to `srand(x)` before any calls to `rand()`. This changes the initial condition for the algorithm used to generate the numbers. A better way to seed the random number generator is to do `srand(time(0));` – this uses the current time in seconds to seed the algorithm. You may need to `#include <ctime>`.

Exercise 7.2

In this exercise we will sort the array using the *Insertion Sort* algorithm.

1. Write a function `insert` which takes an array `a`, a position `p` (which must be strictly less than the total size of `a`), and a floating-point value `val` to insert. Assuming that `a` is already sorted up to position `p`, this function should insert `val` in the correct position (overwriting `a[p+1]`). Obviously there is nothing to do if `p=0`. Now all the values of `a` up to position `p+1` are sorted in ascending order.
2. Write a function `insertionSort` which takes an array `a`, and it's length `l`. For each element `i` of the array in order, make a call to `insert` passing `a`, `(i-1)` as the position, and the value of `a[i]` as the value to insert.

Exercise 7.3

1. Write a program which creates a pair of two-dimensional $n \times n$ matrices (start with $n = 2$ and manually specify the entries - for larger matrices you could generate them randomly). Choose n to be a power of 2.
2. Write a function, `multiply`, which, given two square $n \times n$ matrices $\mathbf{A}$ and $\mathbf{B}$, computes the product $\mathbf{AB}$ into another matrix $\mathbf{C}$. Test this works for your 2×2 case first, and then try with larger matrices.

Exercise 7.4

(Advanced) The matrix product can be computed in fewer operations by using a recursive algorithm. Each matrix is split into quarters, and the sub-blocks are multiplied together (like with 2x2 matrices). This is done recursively.
Write a function `recursiveMultiply` which does this. Your function will need the following parameters:

- The matrices $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$ (obviously arrays passed by reference).
- The size of the submatrices we are multiplying. e.g. for the first subdivision, this will be $n/2$ (hence why n was chosen to be a power of two!).
- The x and y offset of each of: the destination submatrix in $\mathbf{C}$, and of the operand matrices $\mathbf{A}$ and $\mathbf{B}$.

Then:

$$\mathbf{C}_{ij} = \sum_k \mathbf{A}_{ik}\mathbf{B}_{kj}.$$

Chapter 8

The Standard Template Library (STL)

Objectives

In this chapter you will learn about:

- the different types of containers available;
- the algorithms which can be used to manipulate these containers.

Definition: Standard Template Library

The **Standard Template Library (STL)** is a collection of templated containers and associated algorithms for manipulating those containers.

Each container is a C++ class. There are different containers for storing data which are akin to various mathematical concepts such as *set*, *list*, *vector* or *map*. Iterators, which are similar in concept to pointers, are used to move through and access these containers in a safe manner. Associated with these containers are a collection of algorithms which manipulate these containers in an efficient manner, implementing standard operations such as sorting, searching, finding the maximum element, etc.

The containers in the STL are generalised through the use of templates. This allows the containers to hold arbitrary types of data, rather than only pre-determined types (such as `int`, `double`, `std::string`, etc). Templating of classes is discussed in Section 11.

Definition: Container

A container is an object which holds multiple values of a given datatype in a way which obeys a predefined set of rules.

The rules govern the relationship between the values in a container. For example, determining how the values are sorted (or not sorted), whether they have to be unique, and how the data can be accessed. Each container also has different member functions tailored to the nature of the container which can simplify access and manipulation of the data.

8.1 Arrays

Required header: `#include <array>`

The STL `std::array` is a container which emulates a "C"-style fixed-size array. Both the datatype and size of the array object are specified as template parameters and, as with fixed-size arrays, must therefore be known at compile-time.

For an array declared as `int x[5]`, the `std::array` equivalent would be:

```
std::array<int, 5> x;
```

Some advantages of the `std::array` over a conventional array are:

- They can be directly initialised at construction using a comma-separated list of entries (*list-initialisation*):

  ```
  std::array<int, 5> x = {1, 2, 3, 4, 5};
  ```

- They can be directly initialised from another `std::array` object and assignment can be performed directly (assuming the template parameters exactly match):

  ```
  std::array<int, 5> y = x;
  y = x;
  ```

- They can be easily compared:

  ```
  std::array<int, 3> x, y;
  ...
  if (x == y) {
  ...
  }
  ```

- They explicitly know their size and this can be obtained by calling the `size()` member function.
- They have the necessary member functions such that they can be used with STL algorithms (see Section 8.7).

However, a `std::array` cannot be directly interpreted as a pointer. The expectation is that values in the array are accessed and modified using the normal array index operator "`[]`" or through iterators (see Section 8.3).



Note

The notation `{1, 2, 3, 4, 5}` actually creates an instance of another lightweight class in the STL library called `std::initialization_list<T>`. This is usually used for initialising other containers with a list of values. As discussed in Section 2.2.2, initialisation lists do not allow for *narrowing conversions* where a loss of precision occurs.

8.2 Vectors

Required header: `#include <vector>`

An STL `std::vector` is similar to a dynamically allocated array and holds ordered values. Unlike conventional arrays, a `std::vector` can be expanded or contracted after it has been created.

A vector is declared as:

```
std::vector<DATATYPE> IDENTIFIER;
```

where `DATATYPE` specifies the type of data held in the vector and `IDENTIFIER` is the name of the vector. For example, a vector of double-precision floating-point numbers could be declared as, for example,

```
std::vector<double> myvec;
```

8.2.1 Creating vectors

The `std::vector` class has a number of constructors which allow us to create vectors in different ways. Some examples of how a `std::vector` can be created are shown in Example 8.1. On line 2 we

Example 8.1: Creating vectors

```
1 unsigned int vSize = 10;
2 std::vector<double> vMyVector;            // Empty vector.
3 std::vector<float>  vSecondVector(vSize); // Vector of size vSize.
4 std::vector<char>   vString(vSize, 'z');  // Initialise with value.
5 std::vector<double> vAnother(vMyVector);  // Copy an existing vector.
```

created a vector of size zero, which can sometimes be useful if the contents is not known *a priori*; this is not possible with an array. The second form on line 3 creates a vector of a predetermined size. Note that the size does not have to be a constant and can be determined at runtime. The values in the array are initialised to a default value (e.g. 0 for `int`). On line 4 we additionally specify the initial value of each element and on line 5 we create a new vector as a copy of an existing vector.

8.2.2 Using vectors

Vectors are manipulated using the available member functions. To find the number of elements in a vector we use the `size()` function:

```
unsigned int n = vMyVector.size();
```

Element access and assignment is generally through the index operator, for example,

```
vMyVector[3] = 7.0;
```

Like arrays, all elements in a vector are stored contiguously in memory. However, unlike arrays, a vector can be resized later on. To add a single element to the end of an array, or remove the last element, we can do

```
vMyVector.push_back(7.0);
vMyVector.pop_back();
```

These implicitly resize the vector. It can also be explicitly resized:

```
unsigned int M = vMyVector.size();
vMyVector.resize(N);
```

This resizes the vector to have `N` elements. If $N > M$, extra elements are added on the end of the vector and will have the default value. If $N < M$, elements will be deleted from the end of the vector.

8.2.3 Clearing all elements

A vector can be emptied using the `clear()` function.

```
vMyVector.clear();
```

8.2.4 Inserting and Removing Elements

Elements can be inserted at arbitrary positions in the array, and similarly removed, using the `insert` and `erase` functions, illustrated in Example 8.2. These use *iterators* (discussed in Section 8.3) to specify the location of insertion/removal. After these routines, all elements after the given position are shifted along the array and any iterators marking positions after the inserted or removed element are invalidated.

STL
Example 8.2: Inserting and Removing elements from a `std::vector` using `insert()` and `erase()`.

```
1 std::vector<double> vMyVector(5, 0.0);
2 std::vector<double>::iterator vIt = vMyVector.begin() + 3;
3 vMyVector.insert(vIt, 7.0);
4 vIt = vMyVector.begin() + 1;
5 vMyVector.erase(vIt);
```

8.3 Iterators

Definition: Iterator

An iterator is an object which marks a position in a container object and allows such a container to be traversed.

Iterators are similar in concept to pointers in that they mark a position within a container. An iterator is specific to the type of container it is associated with. An iterator for a `vector<double>` would be declared as:

```
std::vector<DATATYPE>::iterator IDENTIFIER;
```

The `::` is the same *scope operator* we have already encountered in classes. This is used because iterators for a container are generally declared as classes within the container class to which they apply. Iterators and container objects of different container type or data type cannot be used together.

Iterators can also be incremented to traverse the container in exactly the same way as pointers and "dereferenced" to access the contents of the container, for example

```
std::vector<double>::iterator vIt;
...
++vIt;
cout << *vIt << endl;
```

They can also be compared.

Member functions, `begin()` and `end()`, on the container can be used to get an iterator pointing to the start and (just past the) end of the container. These can be used to move through the elements of the array, as shown in Example 8.3.



Note

The `end()` member function returns an iterator which is just past the end of the container. Trying to dereference it will produce a runtime error.
However, any suitable iterator which is strictly less than this iterator references an element of the container.

Example 8.3: Iterating over a vector

```
std::vector<unsigned int> vMyVector(10);
std::vector<unsigned int>::iterator vIt;
for (vIt = vMyVector.begin(); vIt < vMyVector.end(); ++vIt) {
    *vIt = rand();
}
```

Explicitly declaring an iterator on containers with more complex types may be cumbersome. Another way to declare an iterator is to use the `auto` keyword, available in C++11 and later standards, for inferring the variable data type from the value assigned to it during construction.

Definition: The `auto` keyword

The auto keyword is used for automatic type deduction. It can be used in place of the variable type, when initializing a variable, to tell the compiler to infer the variable's type from the assigned variable's type.

We can also use *range-based loops*, again available in C++11 and later standards, to further simplify the code. Range-based loops execute over a range of values within an array or other container.

```
for (RANGE-DECLARATION : RANGE-EXPRESSION) {
    STATEMENTS
}
```

Example 8.4 illustrates the use of these concepts. In this code, we define two range-based loops: one to populate the vector, and another to print its contents. The `RANGE-DECLARATION` defines the variable which will hold each element of our vector, specified in `RANGE-EXPRESSION` in turn as the loop `STATEMENTS` are executed. We use `auto` here to get the compiler to deduce the data type of this variable (an iterator in this case). We make `x` a reference to avoid copying the value so the assignment alters the original value in the vector. In the second loop we use `const auto& y`, since we do not want to alter the values in the array and it would be inefficient to copy them for the purpose of printing them.

Example 8.4: Iterating over a vector using `auto` and range-based loops.

```
#include <iostream>
#include <vector>
#include <cstdlib>

int main() {
    std::vector<unsigned int> vMyVector(10)
    for (auto& x : myVector) {
        x = rand();
    }
    for (const auto& x : myVector) {
        std::cout << x << std::endl;
    }
}
```

8.4 Sets

Required header: `#include <set>`

The `std::set` is an associative container which stores unique elements. Elements are referenced by their key (which, for a set, is also its value), rather than their position in the set. Unlike the mathematical concept of a set, the STL set stores elements ordered by their value. It is optimised for random access of elements, rather than sequential.

Sets are declared using

```
std::set<DATATYPE> IDENTIFIER
```

8.4.1 Unordered sets

Required header: `#include <unordered_set>`

The `std::unordered_set`, introduced in C++11, conceptually does not order elements. They are more efficient at retrieving elements, based on their value, than normal sets.

8.5 Pairs

Required header: `#include <utility>`

Pairs associate two independently-typed values together. A pair can be declared using

```
std::pair<FIRST-TYPE, SECOND-TYPE> IDENTIFIER
```

We could create pairs as follows:

```
std::pair<int, int>    p1;
std::pair<int, double> p2(42, 3.14159);
```

A pair has the two member *variables*: `first` and `second`, which return the first and second value, respectively.

```
std::cout << "The meaning of life is " << p2.first << std::endl;
```

8.6 Maps

Required header: `#include <map>`

The STL map is essentially an extension to the set and is an associative container which maps unique keys to values. The keys must be unique in the map, but the values need not be. This container is optimised for random access to elements by their key.

A map is declared using

```
std::map<KEY-TYPE, VALUE-TYPE> IDENTIFIER
```

For this container, there are two template parameters which specify the data types of the **Key** and the **Value**.

8.6.1 Working with Maps

Elements can be accessed either through using iterators, or by using the index operator `[ ]`, where the key is specified in the square brackets. Since a map is typically stored as a binary tree, the computational cost of looking up an entry by its key is $\mathcal{O}(\log N)$; in a vector this would instead be $\mathcal{O}(1)$.

An example of using a map is given in Example 8.5. Since keys are unique in a map, line 9 replaces the value set on line 7 for the key "Sally". Iterators of a map dereference to an `std::pair` object which holds the key and value. In Example 8.5 we use a range-based loop to visit each entry in the map. The variable `x` has type

```
std::map<std::string, unsigned int>::const_iterator
```

which is helpfully deduced by the compiler.

Example 8.5: Iterating through a map

```
1  #include <map>
2  #include <string>
3  #include <iostream>
4
5  int main() {
6      std::map<std::string, unsigned int> vPeopleAges;
7      vPeopleAges["Sally"] = 26;
8      vPeopleAges["James"] = 30;
9      vPeopleAges["Sally"] = 27;  // Replaces previous value for "Sally"
10     for (const auto& x : vPeopleAges) {
11         std::cout << "Name: " << x.first << ", age: " << x.second << std::endl;
12     }
13     return 0;
14 }
```

8.7 STL Algorithms

Required header: `#include <algorithm>`

STL Algorithms are intended to provide common data manipulation algorithms for use with containers. They include standard tasks such as searching, sorting, finding maximum/minimum value and counting. There are also algorithms for copying, transforming, replacing and merging elements from containers. We will examine some of these to illustrate their use but a more complete reference can be found at: http://www.cplusplus.com/reference/algorithm/

8.7.1 `generate`

The generate routine is used to populate a sequence of elements in a container using a function. The generator routine should not accept any parameters. Useful for generating an array of random numbers, as illustrated in Example 8.6.

Example 8.6: Generating random numbers with `std::generate`

```
1  int random () {
2  return rand() % 1000;
3  }
4
5  int main() {
6  std::vector<int> vNumbers(20);
7  std::generate(vNumbers.begin(), vNumbers.end(), random);
8  }
```

8.7.2 for_each

This algorithm runs a user-defined routine on each element from a range of elements in a container. The range is defined by two iterators. The user-defined function is specified by its identifier and must accept a single parameter with data type matching that of the container. This is a read-only algorithm in the sense that the container is not modified. An example of its use is given in Example 8.7.

Example 8.7: Use of the `for_each` STL algorithm.

```
1  void print_number(int i) { cout << i << endl; }
2
3  int main() {
4      vector<int> vNumbers(20);
5      ...
6      for_each(vNumbers.begin(), vNumbers.end(), print_number);
7  }
```

We could also use a lambda expression instead of a separately declared function, such as

```
for_each(vNumbers.begin(), vNumbers.end(), [](int i) { cout << i << endl; } );
```

A second example, where we can also exploit variable capture, is given in Example 8.8. The variable `divisor` is captured by the lambda expression and can remain local in scope.

Example 8.8: Finding numbers that are divisible by a specified value.

```
1  int divisor = 3;
2  vector<int> numbers { 1, 2, 3, 4, 5, 10, 15, 20, 25, 35, 45, 50 };
3  for_each(numbers.begin(), numbers.end(), [&divisor] (int y) {
4      if (y % divisor == 0) {
5          cout << y << endl;
6      }
7  });
```

8.7.3 transform

If we wish to manipulate the contents of a container, we can use the `transform` algorithm. This either: applies a unary operator to a sequence of elements in an input range, outputting the result to another sequence, or; applies a binary operator to pairs of elements in two sequences, outputting the result to a third sequence. The former is illustrated on lines 8-9 of Example 8.9, with the latter on lines 12-13. Also note we have used an *initialisation list* to populate the input vector, which is a feature available in C++11 and later.



Note

You can do an in-place transform - that is one of the input vectors can be replaced by the output.

8.7.4 Copying elements between containers

The `copy` algorithms allow elements of one container to be copied into another container. The `copy_if` algorithm adds the ability to only copy elements which satisfy a condition. The containers do

Example 8.9: Modifying containers with the `transform` algorithm

```
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> vInput = {1,2,3,4,5,6,7,8,9,10};
    std::vector<int> vOutput(vInput.size());

    std::transform(vInput.begin(),  vInput.end(),
                   vOutput.begin(), [] (int& x) { return x*3; });

    std::vector<int> vMoreInput = {2,3,5,7,11,13,17,19,23,29};
    std::transform(vInput.begin(), vInput.end(), vMoreInput.begin(),
                   vOutput.begin(), [] (int& x, int& y) { return x+y; });
}
```

not need to be of the same type, but the type of the elements within them should be.

An example of this is given in Example 8.10. In the first example we provide iterators to the start and end of the range of elements we want to copy, along with an iterator to the starting position in the destination container. In this case, one should be careful that the destination container is large enough! In the second case, we copy elements from an array container to a vector container, selecting only those with value numerically less than 5. Since we copy into a `std::vector` we use the `std::back_inserter` which essentially calls `push_back()` on the `std::vector` to insert the elements. The condition is checked using a lambda expression. Finally we use a range-based loop to print out the selected elements.

Example 8.10: Use of the `copy` and `copy_if` algorithms

```
std::array<int, 10> x = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
std::array<int, 10> y;
std::copy(x.begin(), x.end(), y.begin());

std::vector<int> t;
std::copy_if(x.begin(), x.end(), std::back_inserter(t),
             [&](int p) {
                 return (p < 5);
             });

for (auto i : t) {
    std::cout << i << std::endl;
}
```

8.7.5 Finding the min/max of a set of elements

The minimum or maximum of two elements in a container can be found with the `min(a,b)` or `max(a,b)` routines.

The `min_element()` and `max_element()` routines can be used for finding the minimum and maximum of a range of elements, as shown in Example 8.11. Note that these routines return an iterator which points to the minimum- or maximum-valued element in the range. Therefore, in the example, we need to dereference it to print the actual minimal value.

Example 8.12 illustrates the use of a number of the STL algorithms we have seen so far to scale a vector of random numbers to fit in the range $[0, 1]$.

Example 8.11: Finding min/max element using `min_element` and `max_element`

```
std::vector<int> vNumbers(20);
std::generate(vNumbers.begin(), vNumbers.end(), random);
cout << "Minimum is: "
     << *min_element(vNumbers.begin(), vNumbers.end())
     << endl;
```

Example 8.12: STL Library algorithms example

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cstdlib>
using namespace std;

int main() {
    double inf_norm = 1.0;
    vector<double> v(10);

    generate (v.begin(), v.end(), [] () {
        return double(rand())/RAND_MAX*1000.0;
    });

    for_each (v.begin(), v.end(), [] (double& x) { cout << x << endl; });

    inf_norm = *max_element(v.begin(), v.end());

    transform(v.begin(), v.end(), v.begin(), [&inf_norm] (double x) {
        return x/inf_norm;
    });

    for_each (v.begin(), v.end(), [] (double& x) { cout << x << endl; });
}
```

8.7.6 `sort`

Sorting elements requires the notion of how to compare two elements and infer an ordering of them. We define what it means for two elements i, j to be related as $i < j$ in a routine which returns a boolean value, such as:

```
bool compareFunction(int i, int j) {
    return i < j;
}
```

The `sort` STL algorithm operates on a range of elements in a container denoted by two iterators, along with a comparison function such as the one defined above. The comparison operator is optional; if none is specified the default operator $<$ for the element data type is used, assuming it is defined. Example 8.13 shows the use of the `sort` algorithm for sorting a vector in the default ascending order, followed by sorting in descending order by using a custom comparison function.

8.7.7 `binary_search`

A binary search is an efficient search algorithm which can be applied to a set of sorted elements. A binary search requires a sorted range of elements, a method of comparison, and the value sought.

Example 8.13: Using `sort` to sort a vector in ascending and descending order.

```cpp
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

bool compareFunction(int i, int j) {
    return (i > j);
}

int main() {
    vector<int> vNum(20);
    generate(vNum.begin(), vNum.end(), [] () { return rand() % 100; });

    sort    (vNum.begin(), vNum.end());
    for_each(vNum.begin(), vNum.end(), [] (int& x) { cout << x << endl; });

    sort    (vNum.begin(), vNum.end(), compareFunction);
    for_each(vNum.begin(), vNum.end(), [] (int& x) { cout << x << endl; });

    return 0;
}
```

The search proceeds as follows. The middle element of the range of elements is located and compared with the sought value to determine in which half of the range the value lies. This process is repeated until only a single element remains. True is returned if the element is matched. An example of how to use the binary search is shown in Example 8.14.

Example 8.14: Searching for a number in a vector.

```cpp
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

int main() {
    int vSearchVal = 7;
    vector<int> vNum(20);

    generate(vNum.begin(), vNum.end(), [] () { return rand() % 100; });

    sort    (vNum.begin(), vNum.end());

    for_each(vNum.begin(), vNum.end(), [] (int& x) { cout << x << endl; });

    if (binary_search(vNum.begin(), vNum.end(), vSearchVal)) {
        cout << "Found it!" << endl;
    }

    return 0;
}
```

Exercise 8.1

1. Use the STL vector container to create two vectors of type double and fixed size 100 containing random numbers. Remember that the header file `<vector>` must be included.
2. Create a third vector and, using iterators, set its values equal to the pairwise-element sum of the first two vectors and print the result to screen.
3. The `pop_back()` member function deletes the last entry in a container. Use the `pop_back()` function to delete every element of the third vector above, except the first ten elements. Finally use an iterator to print what is now contained in the vector.
4. The `push_back()` member function adds an element onto the end of an STL container, such as a vector. Use the `push_back()` function to create a vector containing numbers read from the terminal, without explicitly declaring the initial size of the vector at the start of the code. The user should be allowed to enter an arbitrary number of values until they choose to stop entering values.

Exercise 8.2

1. Create a map to associate the name of a country with its area in km^2.
2. Enter some example data into this map (found from the web) - five entries should be sufficient.
3. Use an iterator to find the total area of all the countries entered.

Exercise 8.3

A map is created and populated as follows:

```
std::map<int, int> p;
for (int i = 0; i < 500; ++i) {
    p[i] = rand() % 1000;
}
```

Which of the following approaches to printing the contents of the map is more efficient, and why?

1. ```
   for (const auto& x : p) {
       std::cout << x.first << " : " << x.second << std::endl;
   }
   ```

2. ```
   for (int i = 0; i < 500; ++i) {
       std::cout << i << " : " << p[i] << std:endl;
   }
   ```

What is the computational complexity of each approach?

Exercise 8.4

1. Create two `vector<double>`'s called `x` and `y` of the same size. Populate these arrays with random numbers using an STL algorithm. These represent the x and y coordinates of points on a 2D plane.
2. Use an STL algorithm to print the x coordinates and the y coordinates (separately).
3. Use an STL algorithm to double all the numbers in the `x` vector.
4. Use an STL algorithm to compute the distance of each point $(x[i], y[i])$ from the origin, storing the result in a new vector called `distances`.
5. Use suitable STL algorithms to find the distances of the point which is nearest and furthest from the origin.

Exercise 8.5

Implement the insertion sort algorithm from Exercise 7.2 using vectors and iterators. The user should be able to enter an arbitrary number of values to be sorted **at run-time**. You should use the `insert()` and `erase()` member routines to reposition values in the vector.

Chapter 9

C++ Classes

Objectives

In this chapter you will learn about:

- the use of classes as complex data types;
- data encapsulation and use of access specifiers;
- how to define a class and its implementation;
- how to define and use constructors and the destructor;
- how to overload operators for a class;
- the concept of static functions;
- forward declarations and when they can be used;
- friendship and when it is appropriate.

C++ classes are a syntactic mechanism for encapsulating data and creating user-defined data types. They are the foundations of object-oriented programming, which enables us to model real-world objects in our code. We model not just the data which is stored – which may be a collection of related pieces of data of fundamental data types or other classes – but also the operations that may be performed on that data.

Definition: Class

A C++ class is a programming construct which encapsulates and manipulates data; it is the blue-print, or design, for an object.

Definition: Object

An object is an instance of a *Class* which has real memory allocated.

A *C++ Class* can be thought of as a blueprint for creating **objects** which all behave in a similar way. A class may have zero or more properties (data members) which enable objects to behave slightly

differently. For example, we may create a class for working with complex numbers (see Exercise 9.1) and define the operations (such as addition, subtraction, modulus) which can be performed on such a number.

Another example might be an aeroplane class, encapsulating properties such as wing length, chord length, number of passengers, etc. Properties may themselves be other class types, so our aeroplane class may have one or more engines, with each engine having properties such as maximum thrust, weight, fuel consumption, etc. We might then define member functions which operate on these properties, such as to calculate the lift, or calculate total fuel cost for a particular route. An airline might then instantiate one aeroplane object for each real plane, and use them in a program to optimise its flight schedule over a fixed set of routes to maximise profits.

A central concept is that of *encapsulation*. This is the process of protecting the data within a class, so they can only be modified by functions defined within the class, known as *member functions*. Encapsulation is a very powerful concept in terms of robust programming as it allows us, as a class developer, to control exactly how the encapsulated data can be modified. This is particularly useful as projects become larger and classes are shared between developers.

Before we delve into defining and using classes, we briefly look at data structures (from the C language) upon which the concept of a class is based.

9.1 C Data Structures

Structures, or `struct`s, originate from C and group associated data together to create a user-defined data type. All members of a struct are visible from outside the struct. We access members of struct using the . (dot) operator.

Example 9.1: A `struct` with a number of data members.

```
1 struct Person {
2     string name;
3     int    age;
4     double height;
5 };
```

An example data structure for storing information about a person is shown in Example 9.1. This is a `struct` declaration; it describes what should be held by an *instance* of this struct, but does not in itself have any memory associated with it.



Note

The declaration of a `struct` should end with a semi-colon!

To use our struct, we can create instances of it by writing

```
Person student1;
Person student2;
```

There is real memory associated with the variables `student1` and `student2`. We see that the `Person` data structure is in fact just a data type, like `int` or `double`. However, we can read or modify the member data of a `struct` instance by writing, for example,

```
student1.name = "Steve";
cout << student2.age << endl;
```

We can also copy all the data from one instance of a `struct` to another:

```
student2 = student1;
```

We can also create arrays of students, in the same way as we might create arrays of fundamental data types:

```
Person students[2];
students[0].name = "Joe";
students[1].name = "Dave";
```

Finally, data structures can also be nested, as shown in Example 9.2.

Example 9.2: A `struct` with another `struct` nested within.

```
1  struct Person {
2      string name;
3      int    age;
4      double height;
5      struct Address {
6          int number;
7          string street;
8          string city;
9          string country;
10     } address;
11 };
12
13 Person me;
14 me.address.city = "London";
```

9.2 C++ Classes vs. C Data Structures

A `class` is similar to a C data structure in that it represents a complex user-defined datatype. However, there are some important differences between a C++ `class` and a C `struct`:

- Data is always visible from outside a C `struct`, while it is **hidden** within a `class` by default.
- We can define functions within a `class`, which is not allowed within a C `struct`.
- A C++ `class` allows for inheritance and polymorphism, (more on this later) which a C `struct` does not.



Note

There is also a `struct` syntax on C++. The `struct` syntax in C++ is essentially the same as a `class`, except that properties are visible from outside the `struct` by default. The syntax is maintained in C++, in part, for backwards compatibility with older C codes. It is recommended to use only classes in C++ codes.

9.3 Defining Classes

A class can be defined in a similar way to a data structure:

```
class Point {
    double x;
    double y;
};
```

Remember, as with `struct`, it must be terminated by a semi-colon. Instances of our `Point` class can be created in a similar way to data structures, for example

```
Point p1;
```

However, we cannot now do

```
p1.x = 1.0;
```

because all members of a class are not accessible from outside of the class by default. To make members visible outside the class we use *access specifiers*.

9.3.1 Access Specifiers: Public and Private

Encapsulation of data is achieved through access specifiers. These are:

- `Public:` member is accessible from outside the class.
- `Private:` member is only accessible by routines within the class.
- `Protected:` member is accessible to routines within the class and routines in derived classes (see Section 10.1).

We could explicitly declare our data members as private by writing

```
class Point {
    private:
        double x;
        double y;
};
```

In general, data should be kept private. *Member functions* should be created to allow access to the data in a controlled and validated way. While such functions are often trivial, this approach allows the class to *validate* changes to the data and ensures *data integrity*.



Warning

By default, all members of a class are *private*. It is still good practice to explicitly mark members as private.

9.4 Class Member Functions

Definition: Member Function

A member function is a function which is declared to be part of a *class*.

A member function is always run within the *context* of an object. It has access to all variables defined in the class. We can add a member function to our class by writing it within the class. We can also make it public so it can be called from code outside of our class.

```
class Point {
    public:
        double GetX() {
            return x;
        }
...
};
```

This function is particularly trivial. As classes become larger and the member functions more complex, defining all the functions within the class becomes unmanageable. We can therefore separate the definition of the function from its declaration by putting the implementation outside the class and using the *scope operator* (`::`) to specify it belongs to our class

```
double Point::GetX() {
    return x;
}
```

Remember that because `GetX()` is within the scope of the `Point` class, it has access to all its private data. Finally, we can call our member function on an instance of our class using

```
Point p1;
p1.GetX();
```

Objects are often allocated on the heap using the `new` operator, and thereafter accessed through a pointer. We can then use the *structure dereference operator*, `->`, to access member functions:

```
Point* p1 = new Point;
p1->GetX();
```

This is equivalent to first dereferencing the pointer and then using the structure/dot operator:

```
Point* p1 = new Point;
(*p1).GetX();
```

You might notice the ambiguity in the above example. We have instantiated an object and then ask for the value of `x` before we have set its value. Ideally, we should initialise `x` to a default value when the object is instantiated; this is the role of a *constructor*.

9.5 Class Constructors

Definition: Constructor

A constructor is a special member function which initialises the variables in an object as it is created.

A constructor always has the same name as the class and it may take zero or more parameters. Constructors do not have a return-type, not even `void`.

Defining constructors is technically optional, since if they are not defined the compiler will create several default constructors for you. However, it is usually best to define them ourselves to avoid ambiguity and subtle bugs in the code. In particular, they are necessary when an object needs to allocate memory on the heap for the data it holds.

The constructors created by the compiler, if they are not user-defined, are the *default constructor* and the *copy constructor*. In C++11, the compiler may also create a *move constructor* if the copy constructor is not defined.

9.5.1 Default constructor

The default constructor takes exactly zero parameters. For example,

```
Point::Point() {
...
}
```

Usually it is used to initialise variables when an object is instantiated with a statement such as

```
Point p1;
```

9.5.2 Copy constructor

This constructor takes a constant reference to an instance of an object of the same type. For example:

```
Point::Point(const Point& pSrc) {
...
}
```

This allows an object to be instantiated from another, for example

```
Point p2 = p1;
```

9.5.3 Move constructor

The move constructor provides the ability to move resources without the expense of copying them. In moving a resource its ownership will be transferred from an existing object to the new object. Therefore, unlike the copy constructor, it may leave the source object unchanged, or it might leave it in a very different state.

For objects with only fundamental data types, a move will usually be the same as a copy. However, if a class holds an array as a member variable, the move constructor might instead transfer ownership of the allocated memory to the new object, rather than allocating a new array and copying all the elements, as we would typically do in a copy constructor. Move constructors therefore generally provide a performance advantage. The `std::unique_ptr` discussed in Section 7.2.5 uses the same concept to transfer ownership of an object referenced by a pointer.

A *move constructor* is defined in a similar way to a *copy constructor*, but with two ampersands. For example, a `Matrix` class, which uses an array to store its values, might define copy and move constructors as:

```
Matrix::Matrix(const Matrix& pSrc) {
    mData = new double[pSrc.size()];
    memcpy(mData, pSrc.mData, pSrc.size());
}
Matrix::Matrix(Matrix&& pSrc) {
    mData = std::move(pSrc.mData);
    pSrc.mData = nullptr;
}
```

Note:

- We have used `std::move` to indicate that the `mData` member may be *moved*. In this case, `mData` is a pointer, so the new object would simply point to the same memory of the original `pSrc` and no allocation would occur.
- We explicitly set the old objects pointer to a `nullptr` to avoid having two objects operating on the same memory.

9.5.4 User-defined constructors

We can, of course, define our own constructors appropriate to our class usage. For our `Point` class, we could define a constructor which allows explicit assignment of the values of `x` and `y`:

```
Point::Point(const double& pX, const double& pY) {
    x = pX;
    y = pY;
}
```



Note

We are not obliged to pass parameters to a class constructors as const-references. However, particularly for complex data types, it is more efficient to do so.

9.5.5 Compiler-generated constructors

If you do not specify any constructors, the compiler will generate basic constructors for you, which generally initialise the member variables to default values. However, if you provide one constructor, the compiler will no longer by default generate any of the other constructors. Rather than needing to manually implement all standard constructors, we can explicitly tell the compiler to generate implementations where we are happy for it to do so. This is achieved by using `= default;` after the constructor declaration. In Example 9.3, we choose to implement the main constructor ourselves, but ask the compiler to generate a move constructor for us (as it is trivial, given the member variables).

9.5.6 Deleted constructors

Sometimes, we wish to prevent an object being constructed in a certain way and therefore wish to explicitly exclude it from being implemented manually or generated by the compiler. This can be achieved using `= delete;` after the constructor declaration. In Example 9.3 we wish to prevent objects of our class being copied and so mark the copy constructor as deleted.

Example 9.3: Default and deleted constructors

```
1  class Point {
2      Point(double pX, double pY);
3      Point(const Point& p) = delete;
4      Point(Point&& p) = default;
5
6      private:
7          double x;
8          double y;
9  };
```

9.5.7 Constructor initialisation lists

Initialisation lists are an efficient way of setting the initial values for member variables in an object when it is constructed. In Example 9.4, we could use an initialisation list to set the values of `x` and `y`:

```
Point::Point() : x(0.0), y(0.0) {
    // Nothing left to do
}
```

The colon (`:`) denotes the start of the initialisation list, which consists of a comma-separated list of member variables and their values in parentheses. Before the body of the constructor runs, the values of `x` and `y` have already been initialised.

Little benefit gained for simple classes such as our `Point` class; they are much more useful when initialising objects which use inheritance (see Section 10.1) or data members references.

9.5.8 Default values for class member variables

Default values can be given for member variables which will be used if they are not otherwise set in the constructor initialisation list. In Example 9.4, we could provide default values for the coordinates, thereby avoiding the need to specify them in the default constructor. They can be specified using either an equals initialisation:

```
private:
    double x = 0.0;         // Data
    double y = 0.0;
```

or a brace initialisation:

```
private:
    double x {0.0};         // Data
    double y {0.0};
```

9.6 Class destructors

Definition: Destructor

The destructor performs the reverse of a constructor and implements any *clean-up* operations, such as deallocating memory, necessary for the object.

Like constructors, the destructor always has the same name as the class, but with a preceding tilde (`~`). There is only ever one destructor in a class and it always takes exactly zero parameters. A destructor is called in two cases:

- When an object goes out of scope, for example

```
for (int i = 0; i < 100; i++) {
    Point p1;
    // do something
} // p1 goes out of scope at the end of each iteration and destructor called
```

- When an object, allocated on the heap is explicitly deallocated, for example

```
Point* p1 = new Point;
...
delete p1;
```

9.7 Putting it all together

A more complete example of our `Point` class is given in Example 9.4. Some points to note:

- The `x`, `y` coordinates are `private` and cannot be accessed by any part of the code apart from those defined within the scope of the class.
- `GetX()` and `GetY()` allow us to access the coordinates.
- `SetX()` and `SetY()` allow us to change the coordinates.

Here we have used the constructor to initialise the values of `x` and `y` to 0.0; if we did not do this, they might adopt the value of whatever happened to be in the referenced memory location beforehand, depending on the operating system. A more elegant way to initialise these values is to use an initialisation list.

Example 9.4: Class definition

```
class Point {
    public:
        Point(double px, double py);    // User-defined constructor
        ~Point();                       // Destructor
        double GetX();                  // 'Getter' functions
        double GetY();
        void SetX(double pVal);         // 'Setter' functions
        void SetY(double pVal);
        double Distance();

    private:
        double x;                       // Data
        double y;
};

Point::Point() {
    x = 0.0;
    y = 0.0;
}

Point::~Point() {
    // Nothing to do for this class
}

double Point::Distance() {
    return sqrt(x * x + y * y);
}
...
```

9.8 Operator Overloading

Recall the different operators defined in Chapter 3. These include arithmetic, logical, and relational operators, among others. We are able to use them with our own class definitions by defining what they mean in the context of objects of our class.

Definition: Operator Overloading

Operator Overloading is the definition of standard operators in the context of a class.

You can overload *almost every* operator. Most notably, you cannot overload the scope operator `::`. One might then ask what operators *should* be overloaded in a class? In general, we should only overload operators which make sense conceptually for the class. If our class is mathematically

oriented (e.g. a complex number class), arithmetic operators generally make sense. However, if the class describes an aeroplane then multiplication of one aeroplane by another makes little sense.

Furthermore, we should always define operators in a way which makes sense intuitively. For example, do not overload the multiplication operator to perform addition and vice versa, as this will confuse users of your class.

Operator Overloading has the following syntax:

```
RETURN-TYPE CLASS-NAME::operator<SYMBOL> (OPERAND) {
    ...
}
```

The `RETURN-TYPE` is the type of value returned (usually of type `CLASS-NAME`, `bool`, or `void`). The `CLASS-NAME` is the name of the class, `operator<SYMBOL>` specifies which operator is being overloaded and `OPERAND` optionally specifies the second operand (in the case of binary operators).

Examples 9.5, 9.6 and 9.7 show how to overload some operators.

9.8.1 Return types

Different operators clearly must have different return values. For example, comparison operators should return `true` or `false`. The array-index and parentheses operators may return any appropriate type. Arithmetic operators usually return an object of the same type as the class **by value**; it is therefore essential that a copy constructor is defined. In contrast, operators involving assignment return **a reference to itself** using the `this` keyword. This therefore allows us to do `a = b = c`. This is illustrated in Example 9.5.

Definition: Keyword `this`

The `this` keyword returns a *pointer* to the object under the context of which it is called.

Example 9.5: Assignment Operator Overloading (`operator=`)

```
1  class PositivePoint {
2  ...
3      PositivePoint& operator=(const PositivePoint& pSrc);
4  };
5  ...
6  PositivePoint& PositivePoint::operator=(const PositivePoint& pSrc) {
7      mX = pSrc.mX;
8      mY = pSrc.mY;
9      return *this;
10 }
```

An example of defining the operator which tests for equality is given in Example 9.6. Now, for two `PositivePoint` objects `a` and `b` we can do

```
if (a == b) {...};
```

Here the compiler interprets the expression `(a == b)` as if we had written out in full the expression

```
( a.operator== ( b ) );
```

the return value of which is a `bool`. So here we can see that the operators are really just short-hand syntax for calling specific member functions.

Example 9.6: Equality Operator Overloading (`operator==`)

```
class PositivePoint {
    ...                                          // current object is first operand
    bool operator==(const PositivePoint& pSrc);  // pSrc is second operand
    ...                                          // passed as a constant reference
};

bool PositivePoint::operator==(const PositivePoint& pSrc) {
    if (mX == pSrc.mX && mY == pSrc.mY) {
        return true;
    }
    return false;
}
```

Finally, we illustrate the implementation of arithmetic operators with the addition operator in Example 9.7. Note that this operator returns an instance of `PositivePoint` by value; we cannot return it by reference since the return value is a temporary object which will go out of scope at the end of the `operator+` routine.

Example 9.7: Addition Operator Overloading (`operator+`)

```
class PositivePoint {
    ...
    // Must return a PositivePoint object by value
    PositivePoint operator+(const PositivePoint& pSrc);
};
...
PositivePoint PositivePoint::operator+(const PositivePoint& pSrc) {
    PositivePoint ret;
    ret.mX = mX + pSrc.mX;
    ret.mY = mY + pSrc.mY;
    return ret;
}
```

9.8.2 Time-saving with Operators

Most classes can potentially allow for the implementation of a dozen or more operators. Arithmetically-oriented classes are especially prone to this with not only arithmetic operators (`+`, `-`, `/`, `*`) but also compound assignment operators (`+=`, `-=`, `/=`, `*=`, `++`, `--`). There are even quite a few relational and comparison operators `==`, `!=`, `<`, `>`, `<=`, `>=`. Most of these basically duplicate very similar operations, so we can take advantage of this fact to minimise code duplication and ensure our operators are consistent with each other.

Example 9.8 shows how to overload the `operator!=` using `operator==`; and the `operator<=` by combining `operator<` and `operator==` logical operators.

9.8.3 Self-assignment

There are some aspects when using operators with which we need to be a little careful; self-assignment is one of these. Consider the code below which implements the assignment operator for a matrix class.

Example 9.8: Operator shortcuts

```
bool PositivePoint::operator!=(const PositivePoint& p) {
    return !operator==(p);
}

bool PositivePoint::operator<=(const PositivePoint& p) {
    return operator<(p) || operator==(p);
}
```

```
Matrix& Matrix::operator=(const Matrix& pSrc) {
    delete[] mData;
    mData = new double[pSrc.size()];
    memcpy(mData, pSrc.mData, pSrc.size());
}
```

The following code then instantiates an instance of the `Matrix` class and attempts self-assignment:

```
Matrix a;
...          // Populate matrix
a = a;
```

On the surface our assignment operator implementation looks reasonable. We essentially copy the data from the object on the right-hand side into storage on the left-hand side. Since the matrices may be different sizes, we must reinitialise the storage. However, this code will fail when self-assignment is attempted. While this seems contrived here, more complex codes may indirectly lead to self-assignment.

The reason for this failure is that in deallocating the storage of the assigned-to object, we are also deallocating the storage of the assigned-from object, and therefore we lose the data we wish to copy. Of course, in the case of self-assignment the reallocation and copying is pointless, so we can easily resolve this by checking if the address of the parameter `pSrc` in memory is the same as the result of evaluating `this`. A corrected form of the assignment operator, which protects against self-assignment, is shown in Example 9.9. We could further optimise performance by avoiding the reallocation if the sizes of the operands are equal.

Example 9.9: Self-assignment resolved

```
Matrix& Matrix::operator=(const Matrix& pSrc) {
    if (this == &pSrc) {       // Compare location in mem of the LHS
        return;                // (this) and the RHS (pSrc)
    }                          // If equal, we do nothing.

    delete[] mData;
    mData = new double[pSrc.size()];
    memcpy(mData, pSrc.mData, pSrc.size());
}
```

9.8.4 Move assignment operator (C++11)

In C++11 we can explicitly define a *move assignment* operator, such that the LHS *takes ownership* of the `mData` array of Example 9.9 without reallocation. This is denoted with the double-ampersand in the data type of the parameter and can be explicitly called using `std::move`.

```
Matrix& Matrix::operator=(const Matrix&& pSrc);
...
```

```
m1 = std::move(m2);
```

Move-assignment is quite useful when returning complex temporary objects by value from a member function; rather than performing a deep copy of all the data in the temporary object and then deleting the original as it goes out of scope, we can move it to the return-value object directly to avoid potentially costly reallocations.

9.9 Non-member Operators

Suppose we have a complex number class where we can do:

```
Complex a;
if (a == 5.0) {...}
```

As we saw in the previous section, this uses the member function

```
bool Complex::operator==(double pVal);
```

However, what we cannot yet do is write `if (5.0 == a)`. Now the left-hand side of the equality operator is a floating point number, for which there is no member function taking an object of type `Complex`!

We can additionally define operators on classes *outside* a class as shown in Example 9.10. Such functions are not implicitly associated with a class definition and so they take two parameters: the left-hand operand and the right-hand operand. Now when the compiler encounters `if (5.0 == a)`, the compiler can match such a function.

Example 9.10: Non-member operators

```
1  class Complex {
2      ...
3      bool operator==(double pVal);
4  };
5
6  bool operator==(double pVal, const Complex& pSrc);
7  ...
8  bool operator==(double pVal, const Complex& pSrc) {
9      return (pSrc == pVal);
10 }
11 ...
12 if (5.0 == a) {...}   // compiler translates to: operator==(5.0,a)
```

9.10 Static members

It is often useful to have a member variable or routine which is shared among all instances of a class.

Definition: `static` member

A member declared as `static` indicates there is only one unique copy of that member common to all instances of the class.

A `static` variable can be thought of conceptually like a global variable but restricted to a specific class. We can declare a static variable with, for example,

```
class MyClass {
    static int mySharedVariable;
};
```

A `static` class member cannot be initialised more than once, so it is not initialised in a constructor of a class. Instead it must be initialised separately outside the class definition.

```
int MyClass::mySharedVariable = 0;
```

Just as static member variables can be declared, so too can static member functions. We just need to include the `static` keyword before the function prototype in the class declaration.



Warning

Since static members exist irrespective of whether the class has been instantiated, only static member variables can be used in a static member function!

In Example 9.11 we use a `static` variable and `static` member function to make an *instance counter*. As each instance of `PositivePoint` is created, the `mInstanceCount` variable is incremented by 1 and as each instance of `PositivePoint` goes out of scope or is explicitly destroyed, the `mInstanceCount` variable is reduced by 1. The class routine `getInstanceCount()` is also declared static and it can therefore be called without needing an instance of `PositivePoint` by doing

```
PositivePoint::getInstanceCount();     // 0 instances
PositivePoint vRectangle[4];
PositivePoint vDot;
vDot.getInstanceCount();               // 5 instances
```

Example 9.11: Instance counter

```
1  class PositivePoint {
2      public:
3          PositivePoint();
4          ~PositivePoint();
5          static unsigned int getInstanceCount();
6      private:
7          static unsigned int mInstanceCount;
8  };
9
10 unsigned int Point::mInstanceCount = 0;
11
12 PositivePoint::PositivePoint() {
13     mInstanceCount++;
14 }
15
16 PositivePoint::~PositivePoint() {
17     mInstanceCount--;
18 }
19
20 unsigned int PositivePoint::getInstanceCount() {
21     return mInstanceCount;
22 }
```

9.11 Header and Source files

When working with all but the most trivial of codes, it is a good idea to split code up into multiple source files. It is considered good practice to put each class declaration in its own header file, with the corresponding implementation in a source file; these files are then generally named to match the class. The practicalities of compiling and linking our split up code are discussed in Section C.3.

9.11.1 Header Guards

On larger projects this can lead to header files which include other header files, and so on. Potentially a header file could be included multiple times which would result in our classes being declared multiple times, producing a compile error. To avoid this we can wrap the entire contents of each header file with a pre-processor conditional statement, for example

```
#ifndef CLASS_POSITIVEPOINT
#define CLASS_POSITIVEPOINT
class PositivePoint {
...
};
#endif
```

9.11.2 Forward declarations

To reduce the number of chain-included files the compiler must process, classes may be declared by means of a **forward declaration** within header files, if they are only used in the context of a pointer or a reference within the including file and no class members are referenced. This is instead of including the corresponding header file in which the class is defined. This is possible because in these circumstances the compiler does not need to know the details of the implementation, only that the class exists. The full header file for the class should still be included in the source files where the class is used.

Definition: Forward declaration

A forward declaration indicates that a particular class exists without describing its complete definition.

To forward declare a class called `CLASS-NAME` rather than including the corresponding header file, we simply write

```
class CLASS-NAME;
```

A concrete example of using forward declarations is given in Example 9.12.

9.12 Friendship

Friends to a class are functions or other classes which are permitted to directly access the private members of that class. A friend is specified using the `friend` keyword. Friendship is **not** transitive; if A is a friend of B, and B is a friend of C, then A is not automatically a friend of C.

Example 9.12: Forward Declarations

Line.h

```
#ifndef CLASS_LINE
#define CLASS_LINE
class PositivePoint;        // Forward declaration
class Line {
    ...
    PositivePoint * p1;     // Only pointers to PositivePoint appear
    PositivePoint * p2;     // in Line.h
};
#endif
```

Line.cpp

```
// full header as compiler needs to know about class members
#include "PositivePoint.h"
...
double Line::getLength() {
    double x = p2->getX() - p1->getX();
    double y = p2->getY() - p1->getY();
    return sqrt(x*x + y*y);
}
```



Note

The concept of friendship contradicts the concept of encapsulation. Friendship should be used sparingly and there are only a few cases where it is needed.
The need for friendship is sometimes due to performance considerations; for example, to directly access arrays. However, in many other cases the need for friendship is often an indication of poor code design decisions.

One valid use of friendship is that use of non-member operator overloading. The friendship of a function to a class is specified in the class definition, as illustrated in Example 9.13. In this case, making the non-member operator a friend of the class allows it direct access to the private variables. This allows it to evaluate equivalence of objects directly where data is not otherwise publicly exposed.

Example 9.13: Friendship

```
bool operator==(double pVal, const Point& pSrc);

class Point {
    ...
    friend bool operator==(double pVal, const Point& pSrc);
};

bool operator==(double pVal, const Point& pSrc) {
    return (pVal == pSrc.mX)
}
```

The friendship of a class to another class is similarly specified in the class definition. For example:

```
class Point {
    ...
    friend class Line;
```

```
};
```

This allows every member function in `class Line` to access the private data of `class Point`.

Exercise 9.1

Create a class called *Complex* which stores the real and imaginary parts (as `double`s) of a complex number $z = a+ib \in \mathbb{C}$. Remember, the data should be **private**. Now, add functionality to your class as follows:

- Appropriate constructors and a destructor.
- Methods to get and set the real and imaginary parts.
- Overload the stream operator << to allow, for an object `c` of your Complex type,

  ```
  cout << c << endl;
  ```

- Methods to calculate the argument and magnitude.
- Overload assignment and comparison operators appropriately.
- Overload relevant mathematical operators to allow complex number arithmetic with objects of type `Complex`.
- Create non-member operator functions to allow statements of the type:

  ```
  Complex c, d;
  ...
  if (5 == c) {d = 2.0 * c};
  ```

Create a `main()` function which tests the functionality of your class.

Exercise 9.2

Create a class called `Vector` which encapsulates an arbitrary-sized vector. It should hold a *private* pointer to an array for storing the values of the vector.

- Provide a constructor which accepts an integer parameter specifying the size of the vector.
- Provide appropriate copy and move constructors and destructor.
- Overload the `operator[]` function to accept an integer parameters, `i` and return the value at position i in the vector.
- Overload arithmetic operators for scalar arguments and the assignment operator.
- Add a member function which calculates and returns the L_1 norm and L_2 norm of the vector.
- Add a member function which accepts another `Vector` object as a parameter and calculates and returns the dot-product of itself with this vector.

Exercise 9.3

Create a class called `Matrix` which encapsulates a dense square matrix. It should hold a *private* pointer to an array for storing the values of the matrix.

- Provide a constructor which accepts an integer parameter specifying the size of the matrix.
- Provide appropriate copy and move constructors and a destructor.
- Overload the `operator()` function to accept two integer parameters, `i` and `j` and return the value at position (i, j) in the matrix.
- Overload arithmetic operators for scalar arguments.
- Create a member function which calculates and returns the Frobenius norm of the matrix.
- Overload comparison operators, based on the Frobenius norm.
- Overload the `operator*` function to accept a Vector object (created in Exercise 9.2) as an argument, validate it is of the correct size and perform matrix-vector multiplication.

Chapter 10

C++ Classes - Polymorphism and Inheritance (Optional)

Objectives

In this chapter you will learn about:

- the concept of class inheritance;
- polymorphism and the use of virtual functions;
- multiple inheritance.

10.1 Inheritence

Inheritence allows the creation of classes which are derived from other classes - they become a specialisation of another class. The specialised class is known as the *derived class*, while the class from which it inherits is called the *base class*. Additional members can be added to the derived class to extend the functionality of the base class. A derived class is defined using the syntax

```
class CLASS-NAME : ACCESS-SPECIFIER BASE-CLASS-NAME {
};
```

An example of defining a base and derived class is given in Example 10.1. In this example all members of the base class which are `public` or `protected` are accessible to the derived class.

10.1.1 Inheritence Access

The `ACCESS-SPECIFIER` has different implications for the access level of the members in the base class. There are only two options for `ACCESS-SPECIFIER` in the context of inheritance:

- `public`: Members of the base class which were public in the base class remain publicly accessible in the derived class.
- `private`: Members of the base class which were public become *private* in the derived class.

Example 10.1: Definition of a base class and derived class.

```
class BaseClass {
...
};

class DerivedClass : public BaseClass {
...
};
```

Usually, `public` is the sensible choice, but sometimes `private` is more appropriate if the derived class needs to manage access to the base class members itself.

10.1.2 Protected Member Access

Protected access should be granted to members of a class which should be kept private in general, but remain available to any classes which derive from that class. Protected access is transitive.

An example is given in Example 10.2, in which we give `protected` access to the variable `mCorners`. This member variable is still not accessible from outside the `Polygon` and `Triangle` classes, but just allows `Triangle` to directly access corners rather than having to use the public-interface function `getPoint`. Transitivity means that a class `EquilateralTriangle`, derived from `Triangle` would also retain access to `mCorners`.

Example 10.2: Example: Polygons

```
class Polygon {
    public:
        Point getPoint(unsigned int vIdx);
    protected:                          // Instead of 'private'
        std::vector<Point> mCorners;
};

class Triangle : public Polygon {
    public:
        double getArea();
};
```

10.1.3 Inheritence and Constructors

Constructors exist to initialise class members. A derived class cannot initialise *private* members of its base class as it does not have access to them; in fact the constructor of a derived class should only initialise members of the derived class. We therefore must use the constructor of the base class to initialise members of the base class and this should be done before initialising the members of the derived class (since they may depend on the values of the base class).

We could call the base class constructor from our derived class constructor implementation as

```
BASE-NAME::BASE-NAME (BASE-PARAMS);
```

Here the default scope is that of the derived class so we need the `BASE-NAME::` in order to tell the compiler we want the constructor in the base class.

A better way is to call this is as the first item in the initialisation list of the derived class constructor

```
DERIVED-NAME::DERIVED-NAME (DERIVED-PARAMS)
    : BASE-NAME (BASE-PARAMS),
      DERIVED-INIT-LIST {
}
```

Therefore base class members are all initialised first, followed by any elements of the derived class, followed by any statements in the constructor.

A more illustrative concrete example is given in Example 10.3.

Example 10.3: Initialising base class from derived class constructor.

```
1  class MyBaseClass {
2      public:
3          MyBaseClass(int x)          // Base class constructor
4              : mX(x) {}              // Initialise base class member
5      private:
6          double mX;
7  };
8
9  class MyDerivedClass : public MyBaseClass {
10     public:
11         MyDerivedClass(int x, double a)    // Derived class ctor
12             : MyBaseClass(x),       // <-- Explicitly call base ctor
13               mA(a) {}              // Init other derived class members
14     private:
15         double mA;
16 }
```

10.2 Polymorphism and Virtual Functions

Definition: Polymorphism

Polymorphism of objects means that those objects may behave differently in different contexts.

A pointer to a derived class is *type-compatible* with a pointer to its base class. Recalling the classes defined in Example 10.2, we could write

```
Triangle* T = new Triangle();
Polygon* P = T;
T->getPoint(0); // This line calls the same function in
P->getPoint(0); // the Polygon base class as this line.
```

We instantiate a triangle, which we can use through both a pointer to a `Triangle` and a pointer to a `Polygon` because the former is derived from the latter. The last two lines both call the `getPoint()` member function of the `Polygon` base class.

However, what if we want to specialise behaviour of this function in a derived class? If we just define our new implementation of `getPoint()` in `Triangle` then

```
T->getPoint(0)
```

would call our new specialised implementation, but

```
P->getPoint(0)
```

would still call the version of `getPoint()` in `Polygon`.

Definition: Virtual function

A function declared as `virtual` in a base class can be redefined/specialised (but having an identical function prototype) in a derived class.

When a virtual function is called through a pointer to the base class, the specialised function is executed.



Note

Constructors cannot be made virtual.

Virtual functions are what allows objects to behave according to their instantiated (derived) type, even when accessed through a base-class pointer. This allows us to implement polymorphism. An example of using virtual functions to effect polymorphism is shown in Example 10.4. Which version of `getPoint()` is called in each case?

The version of the `getPoint()` function which is called does not depend on the datatype of the pointer!

- `A` is a `Polygon` so runs `Polygon::getPoint()`.
- `B` is a `Triangle` so runs `Triangle::getPoint()`.
- `C` is **still** a `Triangle`, so runs `Triangle::getPoint()`.

However, note that we could not call the `C->getArea()`. Why?

Example 10.4: Polymorphism using virtual functions.

```
1  class Polygon {
2      virtual Point getPoint(unsigned int idx) {...} // Impl 1
3  };
4  class Triangle : public Polygon {
5      virtual Point getPoint(unsigned int idx) {...} // Impl 2
6      double getArea();
7  };
8
9  Polygon*  A = new Polygon();
10 Triangle* B = new Triangle();
11 Polygon*  C = B;
12 cout << A -> getPoint(0) << endl;
13 cout << B -> getPoint(0) << endl;
14 cout << C -> getPoint(0) << endl;
```

10.3 Virtual destructors

Suppose we create two triangle objects

```
Triangle* T = new Triangle();
Polygon*  P = new Triangle();
```

Suppose we now call `delete T;`. `T` is of type `Triangle*`, so `Triangle::~Triangle` is called. Since `Polygon` is a base class C++ goes ahead and calls `Polygon::~Polygon` in order to destruct the base class.

Now if we call `delete P;` then, since `P` is of type `Polygon*` then `Polygon::~Polygon` is called directly, in the same way as normal member functions. The problem is that `P` is really a `Triangle` but the derived class destructor is never called! This could result in memory not being correctly deallocated causing a memory leak.

The solution is to make `Polygon::~Polygon` virtual. `delete P;` now calls `Triangle::~Triangle` (which in turn calls `Polygon::~Polygon` as above). This is exactly same concept as normal virtual functions.

Therefore, if you are creating a base class, or you think the class might one day be used as a base class, you should make the destructor virtual.

10.4 Pure virtual functions

```
Polygon* P = new Triangle();   // Recall getArea() not in Polygon
cout << P->getArea() << endl;  // Cannot do this!
```

Suppose we do not know how to calculate the area of an arbitrary polygon from a list of its corners, but only for specific shapes such as triangle, rectangle, etc. As such we would not ever wish to specifically instantiate a `Polygon` object, only objects derived from it. For the purposes of creating more general algorithms using our polygon class we would still like to be able to call `getArea()` on these *specific* polygons through a base class pointer. We need to add a `getArea()` function in `Polygon` (but which we do not know how to implement) and make it `virtual`. It makes no sense to instantiate just a `Polygon` and we need to insist every derived class implements `getArea()`.

We therefore make the `Polygon::getArea()` function *pure virtual* and `Polygon` an abstract base class.

Definition: Abstract class

An abstract class is a class which is specifically designed to be used as a base class and contains one or more pure virtual functions.

Definition: Pure virtual function

A pure virtual function is a virtual function that has no implementation and is required to be implemented by all non-abstract derived classes

A pure virtual function is declared

```
virtual DATATYPE FUNCTION-NAME(FUNCTION-PARAMS) = 0;
```

The `= 0` is what denotes the function as pure virtual.

For our `Polygon::getArea()` function we would write

```
class Polygon {
    virtual double getArea() = 0; // The "= 0" indicates pure virtual.
};
```

The class `Polygon` now defines an *interface*.

10.5 Multiple Inheritence

Definition: Multiple Inheritence

Multiple inheritence allows a class to derive from more than one class.

Suppose we have another class, `FilledPolygon` (also derived from `Polygon`). We can inherit functionality from both `Triangle` and `FilledPolygon` in order to defined a `FilledTriangle`, by listing both classes as base classes:

```
class FilledTriangle : public Triangle, public FilledPolygon {
};
```

If two or more base classes have a member with the same name, this is fine since their scope is different. However, if you wish to use one of those members, you must identify it with its *scope*, for example, `FilledPolygon::getArea()`. Remember to initialise all base classes before other members. One thing we need to be careful of with multiple inheritance is the *Diamond Problem*.

10.5.1 The "Diamond Problem"

The previous example had a problem: both `Triangle` and `FilledPolygon` were derived from the `Polygon` base class. Referencing members of `Triangle` or `FilledShape` was fine since they were unique with respect to scope. However, C++ follows independent inheritence routes and so referencing a member of `Polygon` causes a problem: Does the compiler want

- `Polygon::<member>` via `Triangle`
- `Polygon::<member>` via `FilledPolygon`

This is known as the **Diamond Problem**. The solution is to use *virtual inheritence* of `Polygon` so the compiler takes care to create implementations for `Polygon` members only once. We modify the declaration of the derived classes `Triangle` and `FilledPolygon`:

```
class Triangle : public virtual Polygon {
}

class FilledPolygon : public virtual Polygon {
}
```

Now, when we create a `FilledTriangle` we are guaranteed only to get one instance of `Polygon`.

10.5.2 Constructors and Multiple Inheritence

In the same way as we called the base class constructor from the initialisation list, we do the same for multiple inheritence. For our `FilledTriangle` class we would write

```
FilledTriangle::FilledTriangle
        : Triangle(),
          FilledShape() {
}
```

The two items in the initialisation list initialise the members of the base classes before the derived class is initialised.

Exercise 10.1

Extend your solution to Exercise 9.3 to create derived matrix types for symmetric and general banded matrices.
Hint: Try to keep as much functionality in the base `Matrix` class as possible, overriding functions only when needed.

Chapter 11

Templates (Optional)

Objectives

In this chapter you will learn about:

- templates and generic programming;
- how to use templating with functions and classes;
- how to apply template specialisation.

Templates are a C++-language construct which allows the writing of type-generic code which is independent of data type. This idea is known as *generic programming* or *meta-programming*. This allows classes and algorithms to be written in a type-agnostic way, rather than copying and customising the algorithm for each data type we need. The compiler then does the hard work of dynamically generating compiled forms of the algorithm for each data type used in a program, and only those data types which are used.

We have already seen templates in action in the Standard Template Library (STL), see Section 8. The `vector` class is actually a templated class, which allows us to create vectors of any arbitrary data type, as long as certain operations are defined for that data type. When we write

```
std::vector<double> x;
```

we tell the compiler that we want it to generate a `std::vector` class which operates on double-precision floating-point numbers. The data type in the angled brackets, e.g. `<double>`, is known as a template parameter. More complex classes might have multiple template parameters. For example, the STL `std::map` has two parameters – one defining the data type of the *key* and one defining the data type of the *value* in the map. Template parameters can also be non-type value which can be determined at compile time, such as an integer.

Both functions and classes can be templated. Furthermore, we can create *specializations* of templated classes. These allow us to modify the behaviour of our function or class to behave in a specific way for certain data types. This is often used to handle edge-cases. Meta-programming can be an extremely powerful design tool, when used appropriately. It is used extensively in the STL and can be used to develop various code design patterns.

11.1 Function templates

Let us consider a simple function which prints the area of a shape. Without using templates and assuming `Triangle` and `Rectangle` are not polymorphic and do not share a common ancestor, we might have:

```
void printArea(const Rectangle& pItem) {
    cout << pItem.getArea() << endl;
}
void printArea(const Triangle& pItem) {
    cout << pItem.getArea() << endl;
}
Triangle T;
Rectangle R;
printArea(T);
printArea(R);
```

We note there is considerable duplication in the function `getArea()`. We could instead use templates to remove this duplication, as shown in Example 11.1. This function should now be usable for any class which has a function `getArea()`. When it is compiled, the compiler essentially creates the same machine code as when we had written out the function for each data type explicitly.

Example 11.1: Templated `printArea()` function can be used on any data type (class) which has a `getArea()` member function.

```
1 template<class T>
2 void printArea(const T& pItem) {
3     cout << pItem.getArea() << endl;
4 }
5
6 Triangle T;
7 Rectangle R;
8 printArea<Triangle>(T);
9 printArea<Rectangle>(R);
```

There are three new items of syntax in this example. On line 1,

```
template<class T>
```

indicates that the function which follows should be templated. There will be exactly one template parameter, which will be denoted as `T` in the function. The template extends for one code block only – i.e. the end of the function.

On line 2 we have modified the function prototype, for example for the specific type `Triangle`,

```
void printArea(const Rectangle& pItem)
```

to accept a parameter of generic type `T`, as specified through the template parameter,

```
void printArea(const T& pItem) {
```

While we have used the template parameter in the declaration of the function parameter, it could equally well be used in the return type of a function, if that were appropriate. The compiler will be able to generate code for any data type for which the `getArea()` function is defined. If the function does not exist, we would receive a compile error.

Finally, in lines 8 and 9, we call our templated `printArea<T>()` function on each of our `Triangle` and `Rectangle` classes, explicitly telling the compiler the data type we are using.



Note

Here the compiler could *implicitly deduce* the template parameter from the data type of the function parameter. For example, we could just write:

```
printArea(T);
```

11.1.1 Templated Insertion Sort

As a further example, let us now consider our insertion sort algorithm implemented using templated functions instead so that it can operate on any data type for which the *less-than* operator, and assignment operator is defined. Our insertion sort outer level algorithm might have a function prototype such as

```
void insertionSort(float a[], int length)
```

We could template this as

```
template<class T>
void insertionSort(T a[], int length)
```

We have called our template parameter `T`, which replaces the data type of the array (previously `float`). We therefore also replace all occurrences of `float` throughout the algorithm.

Calling the routine requires the datatype parameter to be specified:

```
insertionSort<float>(vMyArray, vMyLength);
```

We similarly apply templating to the `insert()` function in the same manner. The completed example is given in Example 11.2. Note that on line 3 we explicitly use the *less-than* operator to compare two values of our templated type. On line 4 we use the assignment operator on the templated type. If the specific type given when we *instantiate* the template does not define these operators, a compile error will occur.

Example 11.2: Templated version of the Insertion Sort algorithm

```
1  template<class T>
2  void insert(T a[], int p, T val) {
3      while (p > 0 && val < a[p-1]) {
4          a[p] = a[p-1];
5          p--;
6      }
7      a[p] = val;
8  }
9
10 template<class T>
11 void insertionSort(T a[], int length) {
12     for (int i = 0; i < length; i++) {
13         insert<T>(a,i,a[i]);
14     }
15 }
16
17 int main() {
18     float a[n];             // could choose double, int, long, etc
19     ...
20     insertionSort(a, n);    // compiler infers template parameter
21 }
```

11.2 Class templates

Templating can be applied to classes in a similar way to functions. We declare the template parameters immediately before the class declaration and replace the appropriate concrete data type with our template variable throughout the declaration. Example 11.3 illustrates this.

Lines 1-9 show the class declaration; we have declared the template parameters in line 1 and used the template parameters to appropriately declare member functions and private data to operate on the arbitrary data type. Lines 11-14 and 16-19 are implementations for two of the member functions (the constructor was trivial and implemented inline in the class declaration). Recall that each template declaration only applies to the statement/block immediately following it. We therefore need to precede each function definition with the template parameter declaration.

Example 11.3: A templated wrapper class around a `std::vector`

```
1  template<class T>
2  class MyVector {
3      public:
4          MyVector (vector<T> source) {data = source;}
5          T getmax ();
6          T operator[](int index);
7      private:
8          vector<T> data;
9  };
10
11 template<class T>
12 T MyVector<T>::getmax() {
13     return *max_element(data.begin(), data.end());
14 }
15
16 template<class T>
17 T MyVector<T>::operator[] (int index) {
18     return data[index];
19 }
```

To use our new class we can simply instantiate it, much like any other class we might write or one of the STL classes, but with the addition of the template parameter defining the concrete data type to use. An illustration of its use is given in Example 11.4.

11.3 Template specialisation

In some cases we must do something specific for particular data types. This could be for a range of reasons, including algorithm performance optimisations which are only possible on certain data types, or the need to use an alternative algorithm.

In Example 11.4 we created a templated function to generate random numbers. Specifically, the random number was computed using

```
return (T)(rand()) / RAND_MAX * 1000;
```

This works well with floating-point numbers, but invariably evaluates to zero if `T = int`. One approach to fixing this is to use template specialization for this data type. To specialise a function template,

```
template <class T>
T myfunc() {}
```

Example 11.4: Use of the `MyVector` templated class.

```
template <class T>
T random_num() {
    return (T)(rand()) / RAND_MAX * 1000;
}

int main () {
    vector<double> x(10,0.0);
    vector<float> y(20,0,0);
    generate(x.begin(), x.end(), random_num<double>);
    generate(y.begin(), y.end(), random_num<float>);

    MyVector<double> v(x);
    MyVector<float> w(y);

    cout << "Max: " << v.getmax() << endl;
    cout << "First: " << v[0] << endl;
}
```

we remove the parameter from the template declaration and insert the concrete type we are specialising to after the function name (and everywhere else where we used the template parameter in the function definition):

```
template <>
int myfunc<int>() {}
```

A template specialisation for the random number generation function might therefore look like the code shown in Example 11.5. If the template parameter is `int` we use the second version of the routine. All other datatypes use the first.

Example 11.5: Specialisation of the `random_num` function for the `int` data type.

```
// Generic template
template <class T>
T random_num() {
    return (T)(rand()) / RAND_MAX * 1000;
}

// Template specialisation
template <>
int random_num<int>() {
    return rand() % 1000;
}
```

We can also specialize classes in a similar way using, for example, `int` as the data type specialisation:

```
template<class T>
class MyVector {
};

template<>
class MyVector<int> {
};
```

A more complete example is given in Example 11.6.

Example 11.6: Class template specialisation

```
template <class T>
class Container {
    public:
    Container(T arg) : data(arg) {}
    T Get() {return data;}
    void Increase() {++data;}
    private:
    T data;
};

template <>
class Container<char> {
    public:
    Container(char arg) : data(arg) {}
    char Get() {return data;}
    void Uppercase() {data += 'A' - 'a';}
    private:
    char data;
};

int main() {
    Container<int> a(5);
    Container<double> b(3.5);
    Container<char> c('d');
    a.Increase();
    b.Increase();
    c.Uppercase();
    cout << a.Get() << ", " << b.Get() << ", " << c.Get() << endl;
}
```

11.3.1 Partial class specialisation

Things become more interesting when a templated class has multiple template parameters. If we choose to specialise the class template in only a subset of those parameters, this is called *partial specialisation*. We generalise what we have seen for single parameter template specialisation:

```
template<class T1, class T2>
class MyClass {};

template<class T>
class MyClass<char, T> {};
```

The base definition of `MyClass` has two template parameters. Only the first parameter is fixed in the specialisation. Note that we did not need to retain the same template parameter names in the specialisation, although one could retain the names of the non-specialised parameters for clarity. An example of partial specialisation of classes is shown in Example 11.7.



Note

Function templates cannot be partially specialised, as per the C++ standard. Only full specialisation is permitted.

Example 11.7: Partial specialisation

```
template <class T1, class T2>
class Container {
    public:
        Container(T1 arg1, T2 arg2) : data(arg1), tag(arg2) {}
        T1 Get() {return data;}
        void Increase() {++data;}
    private:
        T1 data;
        T2 tag;
};

template <class T2>
class Container<char, T2> {
    public:
        Container(char arg1, T2 arg2) : data(arg1), tag(arg2) {}
        char Get() {return data;}
        void Uppercase() {data += 'A' - 'a';}   // Note: no Increase() function.
    private:
        char data;
        T2 tag;
};

int main() {
    Container<int,    string> a(5,   "My container");
    Container<char,   int>    b('d', 1);
    a.Increase();
    b.Uppercase();
    cout << a.Get() << ", " << b.Get() << endl;
}
```

11.4 Template Metaprogramming

Template metaprogramming can also allow us to move computation from *runtime* to *compile-time*, or to generate code, thereby improving runtime performance. As an example, consider the factorial function:

```
long int factorial(long int n) {
    if (n == 1) return 1;
    return n * factorial(n-1);
}
```

Calling this function with a constant (known at compile time) as

```
cout << factorial(10) << endl;
```

will put 10 entries on the stack, require 10 multiplications, subtractions and comparisons and at least 20 assignments. However, we could have computed this value at compile time!

11.4.1 Compile-time evaluation

Instead, we use meta-programming to enable the compiler to pre-compute the value at compile-time. We can rewrite the factorial function using class templates and use `static const` member variables to force the compiler to perform computations at compile-time. This approach is shown in Example 11.8.

Example 11.8: Factorials at compile-time

```
template< int i >
class Factorial{           // General case
    public:
    static const int RESULT = i * Factorial<i-1>::RESULT;
};

template< >
class Factorial< 1 >{      // Specialise for the case i == 1
    public:
    static const int RESULT = 1;
};
```

The recursive nature of the factorial function is still evident in the code. The single template parameter is just an integer. The template argument of every instance of the `Factorial` class can be computed at compile time since `RESULT` is declared `static const`.

In our program, we can now write

```
cout << Factorial<10>::RESULT << endl;
```

which compiles to the same code as if we had just written:

```
cout << 3628800 << endl;
```

What happens when we compile the code in Example 11.8?

1. The compiler instantiates Factorial$<10>$.
2. It must initialise the static const int RESULT member.
3. To do this it instantiates Factorial$<9>$.
4. This continues until it instantiates the specialisation Factorial$<1>$ and the template recursion stops.
5. Working back up the stack, RESULT is evaluated in each class.
6. Since RESULT is static, const and inline, the compiler optimises away the class structure and inserts the value into the cout line.

11.4.2 Loop unrolling

Consider the loop

```
const int size = 500;
std::vector<double> x(size);
std::generate(x.begin(), x.end(), random_num<double>);
double sum = 0;
for (int i = 0; i < size; ++i) {
    sum += x[i];
}
cout << sum << endl;
```

where we have used our random number generation function from earlier. This requires 1000 additions (including incrementing the counter) and 500 comparisons. Example 11.9 shows how this could be *unrolled* using template meta-programming. By unrolling the loop we eliminate the operations associated with the counter.

Instead of the `for` loop, we instead write

Example 11.9: Loop unrolling using template meta-programming.

```
template< typename T, int i >
class LOOP {
    public:
    static inline void EXEC(T& sum, std::vector<T>& x) {
        LOOP< T, i-1 >::EXEC(sum, x);
        sum += x[i-1];
    }
};

template < typename T >
class LOOP< T, 0 > {
    public:
    static inline void EXEC(T& sum, std::vector<T>& x){}
};
```

```
LOOP<double, size>::EXEC(sum, x);
```

In a similar manner to Example 11.8, the compiler expands the code to

```
sum += x[0];
sum += x[1];
...
sum += x[size-1];
```

This reduces the number of arithmetic operations, so it should be faster. However, the generated code would be larger (in this case from 17KB it increases up to 110KB); this may potentially negatively impact performance of the code execution.

Template meta-programming can be very powerful when used wisely. Ultimately, the only way to assess true real-world impact on performance is to benchmark code.

11.5 Design Patterns

Definition: Design pattern

A design pattern is an abstraction to structure system design. They provide a generic approach to common design problems.

There are in general three categories of design patterns: *creational*, *structural* and *behavioural*. Some examples of each of these are:

- Creational patterns
  - *Abstract Factory*: Decide at run-time what type of object to instantiate based on user input or configuration data.
  - *Singleton*: Ensure a given class is only ever instantiated once.
- Structural patterns
  - *Bridge*: Separate a class interface from its implementation.
  - *Decorator*: Attach additional functionality to an object dynamically.
- Behavioural patterns

- *Iterator*: Generic traversal of data containers (see STL!).
- *Observer*: Notify dependents when an object changes state.

11.5.1 Abstract Factory Pattern

To illustrate the idea of a design pattern, we look at the first example above – the *Abstract Factory Pattern*.

Consider a base class with some derived classes, declared as follows:

```
class PizzaBase {...};
class PizzaMargherita : public PizzaBase {...};
class PizzaHawaiian   : public PizzaBase {...};
```

In this example, the particular data members and their implementation are not important and are therefore not shown. We can now define a factory as follows:

```
class PizzaFactory {
    public:
        PizzaBase* MakePizza(const std::string name) {
            if (name == "Margherita") {
                return new PizzaMargherita;
            }
            else if (name == "Hawaiian") {
                return new PizzaHawaiian;
            }
            return 0;
        }
};
```

This class contains a single (static) function which will return a pointer to a new instance of a specific Pizza type, based on a string provided as a parameter. If the string “Margherita” is passed as a parameter, the returned object will be a `PizzaMargherita`. If the string “Hawaiian” is passed, it will be a `PizzaHawaiian`. Note that the return type is a pointer to a `PizzaBase` and so we would need our pizza classes to be polymorphic.

We can now instantiate a new instance using our factory:

```
PizzaFactory F;
PizzaBase* mypizza = F.MakePizza("Hawaiian");
```

What have we achieved? We have decoupled the `Pizza` class creation from the client code. This means that we only need include `PizzaBase.h` and our factory code wherever we need a pizza. This also means that we only need to update one place in the code if we want to add a new type of pizza, and subsequently only recompile the factory code (and the new pizza class).

We could generalise this more and further decouple the code, but this is left as an exercise.

Exercise 11.1

Extend your solution to Exercises 9.1 to use templating to generalise the Complex class to arbitrary (fundamental numerical) data types. One could therefore instantiate the following, for example:

```
1 Complex<double> x;
2 Complex<int>    y;
3 Complex<float>  z;
```

Exercise 11.2

Create the `PizzaBase`, derived classes and `PizzaFactory` from the notes.

1. Add a `virtual` member function to each pizza class called `PrintToppings()`, which writes the list of ingredients to the terminal.

   Instantiate your different pizza types using the factory and test the correct list of toppings is produced when this function is called through the base-class pointer.

2. Extend each derived pizza class to include a `Create()` function which returns a base class pointer (`PizzaBase*`) to a new instance of that derived class.

3. Generalise the factory to support an arbitrary list of derived classes:

   - Extend the pizza factory to include a `std::map` which maps a string describing the name of the pizza to a pointer to a function (*function pointer*) which will create that pizza.
   - Add a `Register()` function to the factory, taking a string name and function pointer as parameters, which adds to the map.
   - Update the `MakePizza()` function to use the map, creating the derived class instance through the function pointer, rather than hard-coding the types.
   - After creating the instance of the pizza factory, populate it by registering each derived class.

Exercise 11.3

Generalise the factory developed in Exercise 11.2 using templates to allow the use of arbitrary-typed keys and any class hierarchy.

Part II

Supplementary Material

Appendix A

Unix/Linux Operating System

A.1 Overview

Unix and Linux are both operating systems. They are the software that controls the hardware of a computer and manages the software applications run by one or more users. Unix is an commercial operating system developed in the 1970s by AT&T. It introduced the concept of a hierarchical filesystem that we are familiar with today, but also the idea of having many small programs which can be connected together to perform more complex tasks. Later versions of Unix supported multiple users and applications running concurrently through time-sharing of the CPU.

Linux is a Unix-like operating system built on the same principles as Unix. Unlike Unix, it is *open-source* meaning it is free to download and use. The core of the Linux operating system is the Linux kernel, initially released in 1991 by Linus Torvalds (hence the name). Since then many flavours – or *distributions* – of Linux have been developed through contributions from thousands of programmers around the world. The Linux environment is highly customisable, efficient, robust and secure, and is used for a wide array of purposes. For example, over 80% of the world's websites run on servers which use Linux, as do most of the fastest supercomputers. In the past decade, it has gained increasing popularity as a desktop operating system, particularly through the Ubuntu distribution.

The computing systems provided for this course run the Debian Linux operating system, on which Ubuntu is based.

A.2 Desktop environments

Modern Linux distributions support desktop environments. These enable applications to provide an intuitive graphical interface for users. Most Linux graphical environments (and indeed Microsoft Windows and Mac OSX environments) provide common features:

- A *desktop* with icons for quick access to common applications or files;
- A *taskbar* showing the running applications;
- An *applications menu* for launching applications.

Figure A.1 highlights these features on the Linux environment used in this course.

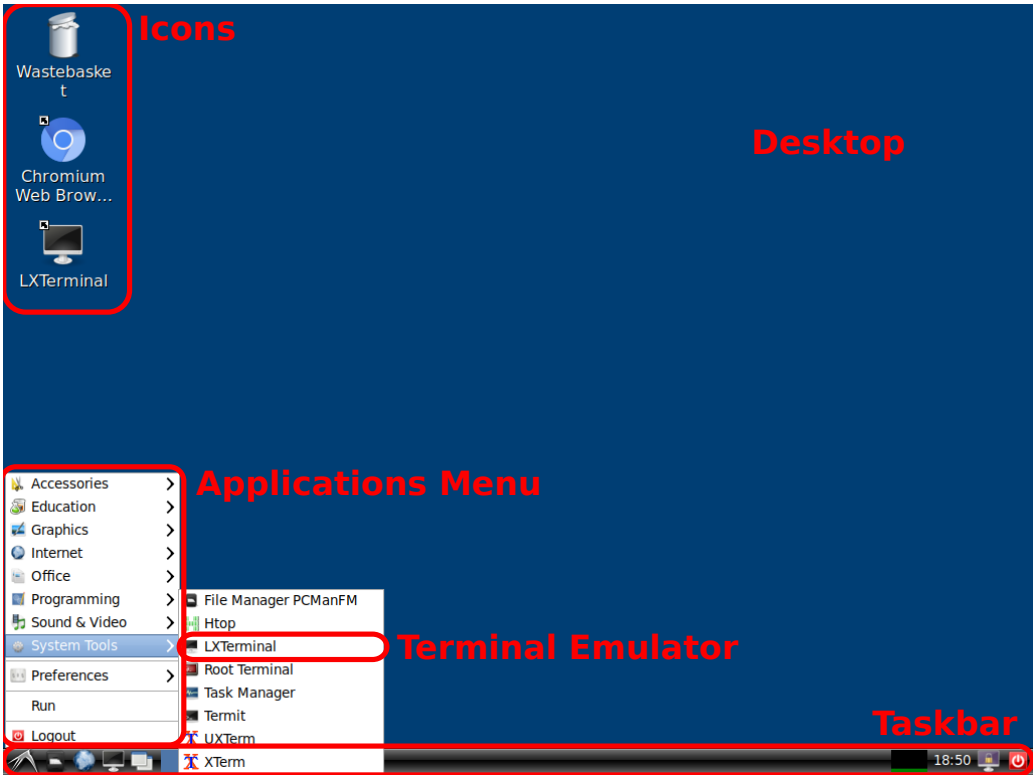


Figure A.1: Key components of a graphical environment.

A.3 Command-line

A command-line interface enables instructions to be given to the operating system through textual commands. These commands may include accessing or modifying files, executing applications and controlling hardware devices. While initially quite daunting to the novice user, with experience the command-line interface can be more powerful and efficient than the graphical environment. We will briefly introduce some of the basic operating system commands in this section.

A.3.1 The Shell

A command-line interface is provided through a *Terminal emulator*. An example is LXTerminal (Figure A.2), which can be launched from the applications menu, as shown in Figure A.1. Initially the terminal shows a *prompt* and awaits the user to type a command. This is in fact produced by a *command-interpreter*, or *shell*, which is a program which will process any input given at the prompt, performing the requested operations or running another program as needed. The format of prompt can vary (and be customised), but the default shown here has the form `user@host:path$`

- `user` - the id of the user under which commands will run. The permissions given to the user will affect which commands can be run and on which files.
- `host` - the name of the computer
- `path` - the current *working directory*

Every program running will have a working directory which is the default place it will look to for reading and writing files. The tilde (∼) symbol is a short-hand notation for the user's home directory.

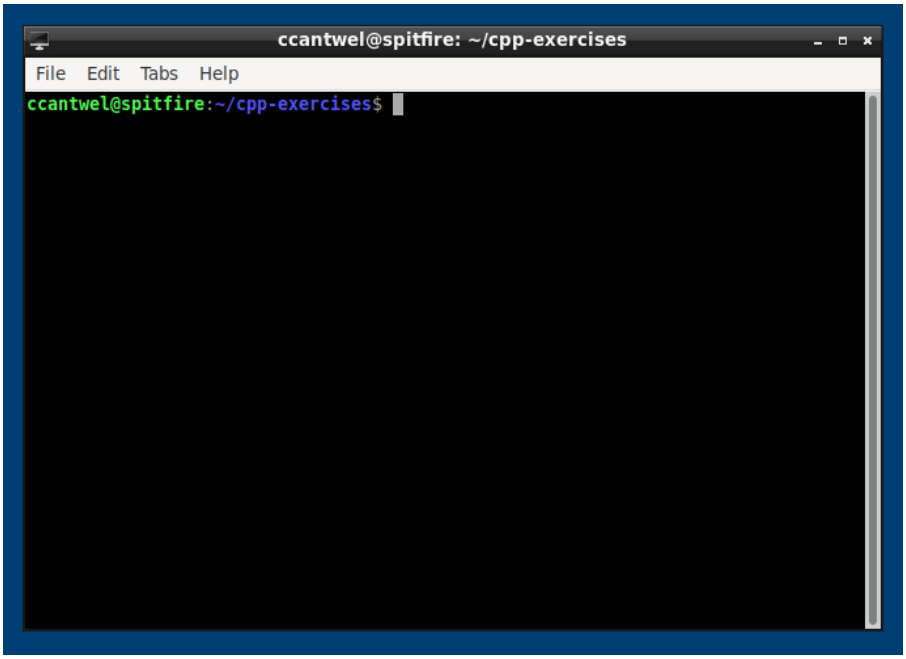


Figure A.2: LXTerminal application



Note
We use the `$` symbol to indicate the shell prompt going forward.

A.3.2 Commands and Arguments

The first word entered at a prompt is taken to be the command to be run. This may be an executable program, or a built-in command of the command-interpreter. In order to customise the behaviour of a command, the user can specify zero or more arguments (separated by spaces) after the command. For example, the following is the command to move a file, where the source and destination filenames are the arguments given:

```
mv myfile.txt ~/mydirectory
```

The shell would replace the ~ with the user's home directory location and then run the `mv` command with the two arguments given. In the case of `mv` requires two or more arguments or it will return an error.

A.3.3 Getting help – man-pages

There are many commands, some of which have many possible arguments. To find out about a particular command, there is the `man` program, which displays the manual for a given command. For example,

```
man ls
```

shows a description of the `ls` command and all the possible arguments to customise the command's behaviour. These are known as man-pages.

A.3.4 Filesystems

There is only a single filesystem hierarchy on a Linux operating system (known as the root filesystem, or `/`). Other filesystems (e.g. disks or network drives) can be attached to a specific location in the hierarchy so we can access them. For example user home directories are on a network filesystem (i.e. the data actually is stored on a different computer on the network) but are attached at the child folder `/home`. Levels in the hierarchy are separated by the `/` symbol. So the files for a user `abc123` would be found in the hierarchy at `/home/abc123`.

A.3.5 Filesystem commands

These commands can be used to browse and manage files in the filesystem hierarchy using the shell.

- `ls`
  Displays a list of the files in the current working directory.
- `cp <source> <dest>`
  Copies the file or directory `<source>` to `<dest>`.
- `rm <file>`
  Deletes a file.
- `pwd`
  Prints the current working directory.
- `cd <dir>`
  Changes the current working directory.
- `mkdir <dir>`
  Creates a directory.
- `du`
  Calculate the size of a file or directory.
- `cat <file>`
  Prints (con`cat`enates) a text file to the terminal.
- `less <file>`
  Display a text file in the terminal, allowing scrolling.

You can find out more details on how to use these commands using the man-pages. Example A.1 illustrates basic usage of some of these commands. The first thing to note is that the hash symbol (#) is used to indicate the start of a comment. As well as interactive usage, the shell can also run sequences of commands from a file and comments can be used to help explain the purposes of commands.

The command `ls -l` displays a *long* listing of files, which includes various details about file attributes:

- the first 'abc123' refers to the user which owns the file.
- the second 'abc123' refers to the group which owns the file (in this case it has the same name as the user and is the user's default group).
- `-rw-r--r--` indicates file permissions. If the first character is a 'd', then this is a directory. The remaining characters are read in groups of three, each indicating whether the file can be read (r), written (w) or executed (x) by the associated user, group or anyone else. In Example A.1, the file is readable and writeable by the user 'abc123' but only readable by anyone else.

```
$ cd ~          # Change to the user's home directory (~ is optional)
$ pwd           # Show the current working directory
/home/abc123
$ ls            # List files
countries.csv
$ ls -l         # List files with details
-rw-r--r-- 1 abc123 abc123   782 Nov 15 10:39 countries.csv
$ mkdir data    # Make a directory
$ mv countries.csv data/
$ cd data
$ pwd
/home/abc123
$ ls -l
-rw-r--r-- 1 abc123 abc123   782 Nov 15 10:39 countries.csv
```

Example A.1: Example usage of some filesystem commands. The dollar ($) symbol represents the shell prompt. Text to the right of the hash (#) symbol is a comment and is not processed by the shell.

- the 782 is the number of bytes in the file.
- Nov 15 10:39 was when the file was last modified.

A.3.6 Text manipulation

Commands that can be used to manipulate text include:

- `echo`
  Repeat a string to the terminal.
- `printf`
  Format strings of characters and numbers.
- `head`
  Display the first N lines from a file.
- `tail`
  Display the last N lines from a file.
- `grep`
  Only display lines matching a given string.
- `cut`
  Select fields or columns from lines of text.
- `paste`
  Concatenate lines from multiple files side-by-side.
- `sed`
  Find and replace sub-strings within a file.
- `sort`
  Sorts lines in a file.
- `wc`
  Counts characters, words (with `-w`) or lines (with `-l`) in a file.

```
$ echo "This is a string"
This is a string
$ head -n 5 countries.csv
Country,Area(km^2)
Albania,28748
Andorra,468
Armenia,29743
Austria,83858
$ grep "Luxembourg" countries.csv
Luxembourg,2586
$ cut -f 1 -d, countries.csv
...
$ sort -k 2 -t, -n -r countries.csv
...
$ sed -e 's/,/: /' countries.csv
```

Example A.2: Illustrative usage of text manipulation commands. The ... indicates output is not shown. The `cut` command extracts the first column of values, assuming a comma as the delimiter. The `sort` command shown orders the rows of the file by land area from largest to smallest. The `sed` command replaces commas with a colon and a space.

Example A.2 shows how these commands can be used to manipulate a comma-separated text file (`countries.csv`, available from Blackboard Learn). The sort example shown orders the rows of the file by the land area column from largest to smallest. The `sed` command is used here with the '-e' (expression) argument to find and replace sequences of characters. The forward-slash separate the components: 's' indicates search and replace; ',' is the string to find; ': ' is the string to replace each instance of the search string.



Tip

Use the man-page to find out about the arguments of the `cut` and `sort` commands.

A.3.7 Text editors

Text editors are interactive tools for editing text files (such as source code) within a shell. Some of the more popular choices include:

- `nano`
- `vim`
- `emacs`

The `nano` editor is relatively simple to use, while `vim` and `emacs` are more powerful (but more complex to learn to use).

A.3.8 Pipes and redirection

By default programs executed from the terminal receive their input from the keyboard and their output is sent to the terminal to be displayed. These are known as *standard input* and *standard output*, respectively. There is a third stream known as *standard error*, which is where programs can write error messages. This is usually also connected to the terminal to be displayed.

These can of course be changed. We can use *redirection operators* to change from where a program receives input, and to where it sends its output. For example,

```
$ echo "C++ is awesome and redirection is cool"
C++ is awesome and redirection is cool
$ echo "C++ is awesome and redirection is cool" > message.txt
$ cat message.txt
C++ is awesome and redirection is cool
```

The first line prints the message to the terminal, since this is the default place to send output. The second command redirects the output to a file instead (therefore nothing is printed to the terminal).

Redirection becomes really powerful when we redirect the output from one command to the input of another. For example:

```
$ cat message.txt | xargs -n 1 | sort | uniq | grep s
```

This reads the text from the file `message.txt` and passes it to the `xargs` program, which splits the text into separate words, after which they are sorted and only one of each word is retained (note 'is' appears only once). Finally, only those words containing an 's' are printed. The output looks like:

```
awesome
is
```

We can even combine this with file redirection:

```
$ xargs -n 1 < message.txt | sort | uniq | grep s > words.txt
```

The first command reads from `message.txt` and the last command writes out to `words.txt`.

Example A.3 gives a less-contrived example and pipes together three commands to extract and format the land area of Luxembourg from the text file `countries.csv`. The `xargs` command runs the command which follows it for each word of input, appending the word as an argument to the command.

```
1 $ grep "Luxembourg" < countries.csv | cut -f 2 -d, | xargs printf "%'.f km^2"
2 2,586 km^2
```

Example A.3: Extracting and formatting the area of Luxembourg from a data file.

Exercise A.1

Download the file `FCDO_Geographical_Names_Index.csv` from Blackboard Learn. This contains the UK Government list of country letter codes, names, official names and citizen names. The first line contains the column headings and columns are separated by commas.

Use shell commands and redirection, as necessary, to:

1. List the country names beginning with a 'G'.
2. Sort the file by citizen name.
3. Find the number of countries in the list.
4. Print the country code for 'Croatia' to the terminal.
5. How many official country names begin with 'The'?
6. How many words are in the official name of 'Sri Lanka'?
7. How many non-whitespace characters are in the official name of 'Sri Lanka'?

Appendix B

List of C++ File Types

Extension	Type	Description
.cpp	C++ Source file	Contains implementations of functions, classes, etc. Exactly one such file in the project must contain the `main()` routine.
.h	Header file	Contains function declarations, class definitions, etc. Used to define prototypes which are then included in one or more C++ source files.
.hpp	Header file	These files are often used to distinguish header files containing templated code from those containing non-templated code.
.so	Shared library	Libraries contain collections of compiled code which can be reused by other programs. There are often one or more header files associated with a library, which declares the available functions. Shared libraries use dynamic runtime linking, and the object code is not included in the executable.
.a	Static library	Like a Shared Library, but the functions and classes used by a program are copied into the programs executable. A static library is therefore only required during compilation.
.o	Object code	These files are the result of compiling a source code file to machine code.
.s	Assembly code	This is the result of halting the compiler part-way through the compilation process before running the Assembler.

Appendix C

The C++ Compiler and Linker

Objectives

In this chapter you will learn about:

- how to use multiple source files to better organise large codes;
- the range of options which can be used to modify the behaviour of the compiler;
- how to use and link to external libraries of code;
- the C++ linker and its purpose.

This chapter discusses the compiler/linker. Typically, as is the case with the GNU C++ compiler `g++`, these two processes are handled by the same executable. One can perform the entire compilation and linking process for an application using a single `g++` command. However, there are benefits to breaking up the process into stages for larger projects.

The `g++` compiler/linker can be invoked as follows in order to compile and link multiple C++ source files into an executable.

```
g++ file1.cpp file2.cpp
```

This will first perform compilation of each source file into a corresponding object file, before linking these together (with some system libraries) to produce an executable called `a.out`. We now explore in more detail what happens during the compilation and linking processes and discuss some of the available optional arguments to the `g++` tool.

C.1 The Compilation Process

The compilation of C++ source code into an executable code which can be directly run by the computer's processor is a complex process. It involves several stages in which the human-readable text, often combined with other codes, is interpreted and translated into machine-readable binary instructions that can be executed directly by the Central Processing Unit (CPU).

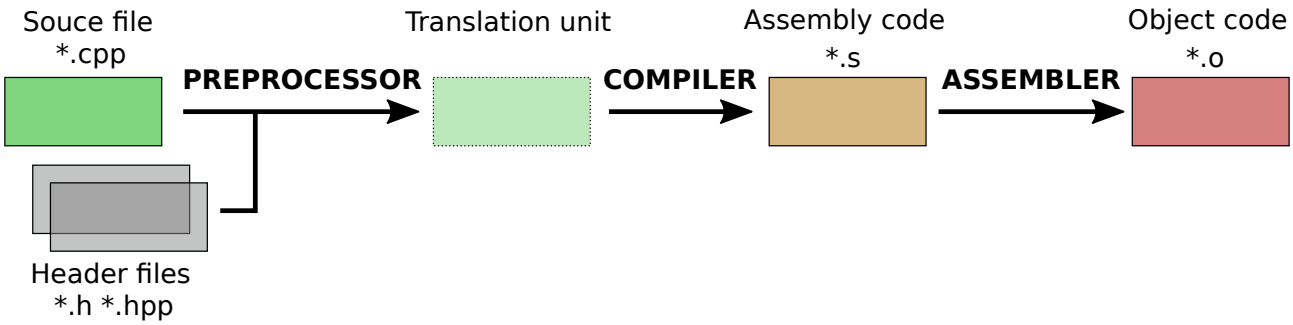


Figure C.1: The compilation process consists of three distinct stages. Filename extensions shown are conventions used on Linux-based systems.



Note

Only C++ source files are passed to the compiler. Header files are loaded implicitly through the use of `#include` commands in the source file.

Conceptually, this process is broken down into three stages, which is illustrated in Figure C.1, although in general (i.e. particularly outside of this chapter) we refer to the three stages together as *compiling*:

- **Preprocessing**: the preprocessor actions preprocessor statements and merges in header files to create a single *translation unit*.
- **Compilation**: the compiler translates this translation unit into assembly code instructions.
- **Assembling**: the assembler takes the assembly instructions produced by the compiler and creates *relocatable object code*.

We are very rarely interested in anything but the end result of this process (the object code).

C.1.1 Preprocessing

In practice, the preprocessing step is usually executed automatically before the compilation process begins. The main tasks undertaken during preprocessing include:

- Inserting the contents of header files in place of the `#include` preprocessor statements.
- Expanding macros such as `#define` preprocessor statements.
- Remove code within conditional `#if`-`#endif` preprocessor statements if the condition evaluates to `false`.

Even the simplest of programs usually results in many thousands of lines of code due to included header files.

To see the result of the pre-processing step, one can ask `g++` to stop after this step by using the `-E` argument, for example:

```
g++ -E file1.cpp -o file1.ii
```

The `-o file1.ii` argument instructs the `g++` to write the expanded source code to file instead of stdout.

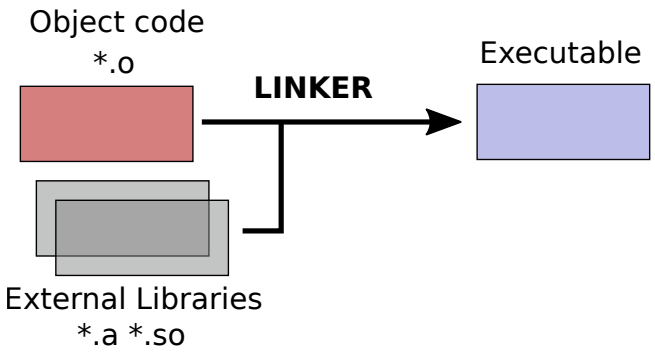


Figure C.2: The linking process puts together the individually compiled translation units with external libraries to create an executable.

C.1.2 Compilation

The compiler translates C++ instructions from the translation unit into low level assembly code. Assembly code (or symbolic machine code) consists of instructions which are specific to the target CPU. These are typically mnemonics. Many simple statements in C++ will result in several low-level statements in assembler.

To see the result of the compilation process, use the `-S` argument:

```
g++ -S file1.cpp
```

The resulting file `file1.s` then contains assembly code for the translation unit.

C.1.3 Assembly

Finally, the assembler converts the assembly code into machine code instructions which can be directly understood by the processor and which is created in such a way that it can be put together with other object code (relocatable) by the linker.

To stop after this step we could use

```
g++ -c file1.cpp
```

C.2 The Linking Process

Linking is the process of connecting the compiled object code generated from the different source code files into a single executable. The source code may make reference to *symbols* (such as functions) provided by external libraries. It is also the job of the linker to make sure these libraries and required symbols are available and that the executable knows the position in the library to go to when the symbol is required. The linking process is illustrated in Figure C.2.

Unless requested to stop at an earlier point in the compilation process using one of the options discussed above, the linking process will automatically be performed to generate executable code.

To perform the linking process on previously generated machine code (files typically with the `*.o` extension), one or more of these files are specified on the command-line, instead of the source code files. For example,

```
g++ file1.o file2.o
```

will link the code in `file1.o` and `file2.o` and generate an executable called `a.out`.

Rather than accepting the default output name of `a.out`, the name of the executable can be specified using the `-o` command-line argument, for example,

```
g++ -o myprog file1.o file2.o
```

C.3 Compiling multiple source files

As a program becomes larger and larger it is common practice to separate the program source code into several (if not many) files. This helps keep the program code organised and manageable, and therefore easier to work with. There are two important types of files we should use: source files (.cpp) and header files (.h). See Section B for more information on the purpose of these file types.

We start by moving functions into separate files. The choice on how to organise functions and classes in files is down to the programmer, but by convention we usually aim to collate related functions together, or restrict ourselves to one class per source file (naming the source file the same as the class). Furthermore, we extract function prototypes and class declarations into corresponding header files. An example of this is shown in Example C.1.

Example C.1: Splitting up code into multiple files.

`myfunction.h`

```
1 void tripleValue( double & );
```

`myfunction.cpp`

```
1 #include "myfunction.h"
2 void tripleValue( double & pVal) { pVal *= 3.0; }
```

`main.cpp`

```
1  #include <iostream>
2  #include "myfunction.h"
3
4  int main(){
5      double vMyVal = 4.0;
6      std::cout << "Initial value is: " << vMyVal << std::endl;
7      tripleValue(vMyVal);
8      std::cout << "Triple value is:  " << vMyVal << std::endl;
9      return 0;
10 }
```

We can now compile the source files separately and link everything together. Note that the header file is not compiled, although it is included in both `main.cpp` and `myfunction.cpp` by the preprocessor.

```
# Compile main code
g++ -Wall -O2 -o main.o -c main.cpp
# Compile myfunction
g++ -Wall -O2 -o myfunction.o -c myfunction.cpp
# Link everything together
g++ -o myprog main.o myfunction.o
```

C.4 Compiler/Linker options

Compilers can take many arguments which influence how the code is compiled. Some of these are essential, such as specifying the libraries needed by the program, while others might "tune" the compilation process and resulting code, include additional information to assist with debugging (identifying why code does not work as expected) or provide warnings about possible problems with the code.

It is important to remember that the way a code is compiled can make a big difference in performance.

C.4.1 Specifying Libraries

To tell the linker that an external library will provide some of the functionality needed by a program, these must be listed as arguments on the command-line. Library names on Linux typically start with `lib` and either have a `.a` or `.so` extension. The former indicates the library is designed for static linking, while the latter for dynamic linking. Often a library with both extensions is available on the system. The part between the `lib` and the extension is the *basename*. The compiler will choose to link dynamically by default if only the basename is provided with the `-l` option. Alternatively the full path to the library can be provided on the command-line, without using the `-l` option.

For example:

```
g++ -o myprog file1.o file2.o -lblas
```

or

```
g++ -o myprog file1.o file2.o /usr/lib/libblas.so
```



Note

Dynamic linking just adds a reference to the dynamic library file (`*.so`) in the executable. When the executable is run, the dynamic library must also be available and is loaded by the operating system.

Static linking actually *copies* the used symbols from the library file into the resulting executable; this allows the executable to be moved to another system (of the same CPU type, of course) and run without requiring the library files. However, the executable is of course potentially much bigger in size.

C.4.2 Library dependencies and ordering

Some libraries depend on other libraries and all must be specified for the linking process to complete. Furthermore, the order in which libraries are specified on the command-line matters. When the compiler encounters a symbol (such as a function) in a translation unit or library which does not exist in that unit or library, it looks for it in the libraries which follow using the ordering specified on the command-line. It will not look at libraries specified earlier.

For example, the LAPACK library uses routines from the BLAS library. Therefore, both libraries must be specified and the LAPACK library must be specified *before* the BLAS library:

```
g++ -o myprog file1.o file2.o -llapack -lblas
```

C.4.3 Search Paths

The majority of applications, which perform anything but the most trivial of tasks, rely on external code that is made available in pre-compiled libraries and/or external headers. The compiler will search through a number of standard locations when these files, for example `/usr/lib` for libraries and `/use/include` for headers on Linux systems.

If a library is located elsewhere, the compiler must be told additional paths in which to search. This is accomplished with the `-L` option, placed before the library name itself:

```
g++ -o myprog file1.o file2.o -L/path/to/my/lib -lmylib
```

Similarly, when compiling, if header files are located in a non-standard path, the compiler can be instructed to search these paths using the `-I` option:

```
g++ -c -o file1.o -I/path/to/my/headers file1.cpp
```

These can be combined if compiling and linking together:

```
g++ -o myprog -I/path/to/my/headers -L/path/to/my/lib file1.cpp -lmylib
```

C.4.4 Warnings

The compiler and linker is a very complex and powerful tool. In the process of converting C++ code to machine code, it must decipher each and every symbol in the code and the relationships between them. As a consequence, it can identify places in the code where mistakes may have been made, there are ambiguities over what is intended, or the resulting code may be sub-optimal.

The options below control how much of this information is displayed.

Option	Description
`-Wall`	turns on all warnings, including those below
`-Wcomment`	displays nested comments warnings
`-Wformat`	displays incorrect use of format strings in `printf`
`-Wunused`	displays any variables which are declared but not used
`-Wimplicit`	displays functions which are used without declaration (header is missing)

Some additional warning flags which may be useful include:

Option	Description
`-W`	common programming errors (e.g. logical expressions which can never be true)
`-Wconversion`	conversion errors (e.g. passing the wrong type to a function)
`-Wshadow`	redeclaration of a variable

For more details see https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html

C.4.5 Optimisations

If instructed, the compiler will try to optimise the code to try and produce a faster or smaller executable. As a rule, one should initially compile without optimisation flags, check that the executable is correct and then try different optimisation levels.

There are four levels of optimisation:

Option	Description
`-O0`	no optimisation at all, no code rearrangement
`-O1`	common optimisations with no space trade-off, no instruction scheduling
`-O2`	turns on instruction scheduling but does not increase exec size or memory requirement; good for production level
`-O3`	turns on options that make executable larger, e.g. function inlining

For example, we can compile our earlier example with some optimisations enabled:

```
g++ -O2 -c -o file1.o -I/path/to/my/headers file1.cpp
```

C.4.6 Language standard

C++ is an evolving standard and its specification has been defined several times over the years in different *standards*. GCC supports the original ISO C++ standard published in 1998, as well as the 2011 and 2014 revisions. Some newer language features are only available in the later revisions of the standard.

To tell the compiler which standard to adhere to, we use the `-std` argument. For example:

```
g++ -std=c++11 -o myprog file1.cpp
```

The options are

- `-std=c++98`, or `-std=c++03` - Original standards
- `-std=c++11` - 2011 revision (default for our compiler version)
- `-std=c++14` - 2014 revision
- `-std=c++17` - 2017 revision

Most of what we will cover is common to all the standards. A few features are only available in the 2011 and later revisions.

C.5 Compiler and Linker Errors

C.5.1 Undefined References

If the linker cannot find a symbol it reports an *undefined reference* and an error such as the following is shown:

```
/tmp/cc49I6tc.o: In function ‘main()’:
test.cpp:(.text+0x9fe): undefined reference to ‘dgbmv_’
```

We might have forgotten to link against the library which provides the function (in this case BLAS), listed the libraries in the wrong order on the command-line, or declared a function in our code and not defined its implementation.

Exercise C.1

Download the files `compiler-main.cpp`, `compiler-norm.cpp` and `compiler-norm.h` from Blackboard Learn. This program uses the BLAS library (`/usr/lib/libblas.so`) and a manual routine to compute the norm of a randomly generated vector of numbers.

Open a terminal and change to the directory in which you downloaded the files.

1. Use the compiler and an appropriate command-line argument to pre-process the `compiler-main.cpp` file. Examine the output to observe the effect of preprocessor statements.
2. Use appropriate compile flags to compile each of the files separately into object code.
3. Use the compiler to link the object code into an executable.
4. Does this code meet the C++98 standard? Use the compiler to test it.
5. Execute the code; notice that it just produces zero. Use compiler warning flags to help you identify the errors in the code.
6. When the code produces a non-zero result and both algorithms match, examine the effect of using optimisation flags on the runtime of the code, using the Linux `time` command.